

Dieses Dokument ist vom 23.03.2026. Die aktuelle Version des Dokuments kannst du im moodle oder [direkt von GitHub herunterladen](#). Dieses Dokument wird ständig aktualisiert.

Dieses Dokument enthält 328 Fragen und Aufgaben unterschiedlicher Größe und andere Ressourcen zum Thema Deklarative Programmierung. Die Inhalte dieses Dokuments sollen dir helfen, dein Verständnis über Haskell und Prolog zu prüfen und zu stärken.

Die Aufgaben sind grob in drei Schwierigkeitsstufen eingeteilt:

- **STUFE 1** Aufgaben zur Überprüfung grundlegender Kenntnisse. Die Lösung erfolgt durch direktes Anwenden bekannter Regeln oder Verfahren und erfordert kein vertieftes Nachdenken.
- **STUFE 2** Aufgaben, bei denen grundlegende Kenntnisse angewendet und durch eigene Überlegungen ergänzt werden müssen. Es sind erste Ideen oder einfache Lösungsstrategien notwendig.
- **STUFE 3** Komplexe Aufgaben, die ein vertieftes Verständnis voraussetzen. Die Lösung erfordert mehrere Gedankenschritte, das Verknüpfen verschiedener Inhalte sowie eigenständigen Lösungsstrategien.

Herausforderndere Aufgaben sind als Challenges gekennzeichnet. Sie erfordern häufig das Verständnis mehrere Konzepte und führen teilweise zusätzliche Inhalte ein, die speziell für deren Bearbeitung relevant sind. Tests, deren Bearbeitung voraussichtlich mehr als 10 Minuten dauern, sind mit ☹️ markiert, Challenges bei 30min.

In dieses Dokument haben es bereits sehr viele Selbsttests geschafft. Das kann überwältigend wirken. Um dem etwas entgegenzuwirken und um dir einen ersten Lernpfad zu zeigen, haben wir eine Auswahl von Aufgaben mit Tier-Emojis hervorgehoben. Weiter sind Aufgaben mit den **Level Up** und **Deep Dive** markiert. Erstere gehen über die Lerninhalte der Vorlesung hinaus – könnten dir aber helfen, die eigentlichen Lerninhalte der Vorlesung nochmal besser zu verstehen. Letztere solltest du, je nachdem wie viel Zeit du bereit zu investieren, nur dann machen, wenn du bereits mit gutem Verständnis dabei bist. Wir hoffen, das hilft dir bei deiner Orientierung in diesem Dokument.

Für die Selbsttests wird es absehbar keine Lösungen geben. Stattdessen möchten wir dich ermutigen, deine Lösungen mit anderen Mitstudierenden oder Mitarbeitenden zu diskutieren, solltest du offene Fragen haben – oder du promptest verantwortungsvoll das LLM deiner Wahl. An jeden Abschnitt ist eine Checkliste zur Selbstevaluation angehängt. Wenn du auf einer geeigneten Bewertungsskala (z.B. Schulnoten) für dich feststellst, dass du weiterhin Schwierigkeiten hast, melde dich gerne, damit wir dir helfen können.

Die Inhalte dieses Dokuments sind nicht vollständig und nicht fehlerfrei. Wir machen Fehler! Es kann sein, dass Modulinhalt nicht durch Selbsttests, Referenzen oder Selbstevaluationen abdeckt werden und es nie werden – insbesondere fehlen derzeit noch viele Lernziele in den Selbstevaluationen. Betrachte dieses Dokument immer als „work in progress“ und ziehe auch immer andere Quellen zum Lernen heran.

Wenn du Anmerkungen oder weitere Ideen oder Quellen für Inhalte für dieses Dokument hast, dann schreibe uns gerne über z.B. mattermost an – oder [erstellt ein issue oder stellt eine PR auf GitHub](#).

# Inhaltsverzeichnis

|                                                         |     |
|---------------------------------------------------------|-----|
| Funktionale Programmierung .....                        | 3   |
| Ausdrücke und einfache Funktionen .....                 | 3   |
| Datentypen .....                                        | 6   |
| Polymorphismus .....                                    | 7   |
| Pattern Matching .....                                  | 14  |
| Automatisches Testen .....                              | 16  |
| Funktionen höherer Ordnung .....                        | 16  |
| Abstrakte Datentypen .....                              | 25  |
| Typklassen und Überladung .....                         | 30  |
| Lazy Evaluation .....                                   | 35  |
| Sequenzen .....                                         | 42  |
| List Comprehensions .....                               | 43  |
| Ein- und Ausgabe .....                                  | 44  |
| Funktoren, Applicatives und Monaden .....               | 46  |
| Automatisches Testdatengenerierung .....                | 60  |
| Logische Programmierung .....                           | 66  |
| Prolog, Prolog! .....                                   | 66  |
| Elementare Programmiertechniken .....                   | 73  |
| Rechnen in der Logikprogrammierung .....                | 77  |
| Negation .....                                          | 88  |
| Der Cut-Operator .....                                  | 89  |
| Prolog-Arithmetik und nicht-relationale Prädikate ..... | 90  |
| Prädikate höherer Ordnung .....                         | 93  |
| Kapselung des Nichtdeterminismus .....                  | 93  |
| Logik-Puzzles .....                                     | 94  |
| Hinweise zu Tests und Challenges .....                  | 97  |
| Weitere Ressourcen .....                                | 99  |
| Häufige Bearbeitungsfehler in Klausuren .....           | 100 |
| Klausur 1.PZ WS25/26 .....                              | 100 |
| Appendix .....                                          | 102 |

# Funktionale Programmierung

## Ausdrücke und einfache Funktionen

### Referenzen

- Skript: Funktionale Programmierung, Ausdrücke und Funktionen
- [Introduction – Learn You a Haskell for Great Good!](#)
- [Starting Out – Learn You a Haskell for Great Good!](#) („Ready, set, go!“ und „Baby’s first functions“)
- [Syntax in Functions – Learn You a Haskell for Great Good!](#) („Where!?“ und „Let it be“)
- [Pure function – Wikipedia](#)

### TEST 1

Beziehe Stellung zu der Aussage „Alles ist ein Ausdruck“ in Haskell?

### TEST 2

Was bedeutet es, wenn eine Funktion keine Seiteneffekte hat? Warum ist die Abwesenheit von Seiteneffekten wünschenswert, sofern es möglich ist?

xkcd: Haskell

### TEST 3

Haskell ist eine streng getypte Programmiersprache. Was bedeutet das?

### TEST 4

Wenn du eine Schleife in Haskell umsetzen möchtest, auf welches Konzept musst du dann zurückgreifen?

### TEST 5

Wie können wir Ausdrücke bedingt auswerten?

### TEST 6

Welche Vorteile und Nachteile haben streng getypte Programmiersprachen?

### TEST 7



In imperativen Programmiersprachen sind Variablen Namen für Speicherzellen, deren Werte zum Beispiel in Schleifen verändert werden können. Als Beispiel betrachte die Funktionen `clz` und `popcnt`.

```
def clz(n):  
    k = 0  
    while n > 0:  
        n /= 2  
        k += 1  
    return 64 - k
```

```
def popcnt(n):  
    k = 0  
    while n > 0:  
        if n % 2 == 1:  
            k += 1  
        n /= 2  
    return k
```

In Haskell sind Variablen keine Namen für Speicherzellen. Wie können wir dieses Programm in Haskell umsetzen? Wo wandert das `k` hin oder wo wandert im Allgemeinen der Zustand hin?

### TEST 8



Um ein weiteres Glücksspiel neben Blackjack in diesem Modul vorzustellen, implementieren wir in diesem Test einen einarmigen Banditen. Diese soll einen initialen Zustand haben und als Ergebnis ein Tripel und einen neuen Zustand liefern. Sowohl der Zustand als auch das Tripel soll aus Ganzzahlen bestehen.

- Überlege dir welche Signatur die Funktion haben muss. Dafür kannst du die Überlegungen aus [Test 7](#) fortführen.
- Sei  $s$  der initiale Zustand, dann soll das Tripel

$$((s + 5) \bmod 3, (s + 7) \bmod 3, (s + 11) \bmod 3)$$

sein und der neue Zustand  $s + 11$ . Implementiere eine Funktion `slotMachine` mit deinem überlegten Typ. Du hast gewonnen, wenn alle Zahlen des Tripels gleich sind.

---

Casinos lieben diesen Spiel-Automaten.

### TEST 9

Auf was müssen wir achten, wenn wir eine rekursive Funktion definieren? Die Antwort ist abhängig von dem, was die Funktion berechnen soll. Denke über die verschiedenen Möglichkeiten nach und gebe Beispiele an.

### TEST 10

Gegeben sei das folgende Haskell-Programm.

```
even :: Int → Bool
even 0 = True
even n = odd (n - 1)

odd :: Int → Bool
odd 0 = False
odd n = even (n - 1)
```

- Berechne das Ergebnis von `odd (1 + 1)` händisch.
- Wie sieht der Auswertungsgraph für den Ausdruck `odd (1 + 1)` aus?
  - Welcher Pfad entspricht deiner händischen Auswertung?
  - Welcher Pfad entspricht der Auswertung, wie sie in Haskell stattfindet?
  - Welcher Pfad entspricht der Auswertung, wie sie in Python sinngemäß stattfindet?

### TEST 11

Es wird als sauberer Programmierstil angesehen, Hilfsfunktionen, die nur für eine Funktion relevant sind, nicht auf der höchsten Ebene zu definieren. Mithilfe welcher Konstrukte kannst du diese lokal definieren?

### TEST 12



Das Potenzieren einer Zahl  $x$  (oder eines Elements einer Halbgruppe) mit einem natürlich-zahligen Exponenten  $n$  ist mit  $\mathcal{O}(\log n)$  Multiplikationen möglich. Dafür betrachten wir

$$x^n = \begin{cases} (x^{\frac{n}{2}})^2 & \text{falls } n \text{ gerade,} \\ x \cdot x(x^{\frac{n-1}{2}})^2 & \text{sonst.} \end{cases}$$

Implementiere eine Funktion, die diese Variante des Potenzierens umsetzt.

---

Das Verfahren ist als Binäre Exponentiation bekannt. (^) ist so in Haskell implementiert (siehe `powImpl`).

### TEST 13

Gegeben ist folgender Ausdruck.

```
let v = 3
    w = 5
    x = 4
    y = v + x
    z = x + y
in y
```

Welche Belegungen der Variablen werden tatsächlich berechnet, wenn wir `y` ausrechnen?

#### TEST 14

Ist der folgende Ausdruck typkorrekt?

```
if 0 then 3.141 else 3141
```

#### TEST 15

Definiere eine Funktion, die kein Ergebnis liefert.

### Selbstevaluation

Ich bin in der Lage, ...

- einfache Funktionen selbstständig zu definieren,
- typkorrekte Ausdrücke zu definieren und händisch auszuwerten, und
- erste grundlegende Konzepte der funktionalen Programmierung zu erklären wie
  - pure functions,
  - Rekursion und
  - streng getypte Programmiersprachen.

## Datentypen

### Referenzen

- Skript: Funktionale Programmierung, Datentypen
- [Introduction – Learn You a Haskell for Great Good!](#)
- [Starting Out – Learn You a Haskell for Great Good!](#) („An intro to lists“)
- [Making Our Own Types and Typeclasses – Learn You a Haskell for Great Good!](#) („Algebraic data types intro“)
- [Constructor – HaskellWiki](#)

#### TEST 16

Wie werden algebraische Datentypen in Haskell definiert?

#### TEST 17

Was ist charakterisierend für Aufzählungstypen, Verbundtypen und rekursive Datentypen? Gebe Beispiele für jeden dieser Typarten an.

#### TEST 18

Benenne null-, ein- und zweistellige Konstruktoren aus der Haskell Prelude.

### TEST 19

Gegeben ist der Typ `IntList` mit `data IntList = Nil | Cons Int IntList`. Weiter kann mithilfe der Funktion

```
lengthIntList :: IntList → Int
lengthIntList Nil = 0
lengthIntList (Cons _ xs) = 1 + lengthIntList xs
```

die Länge einer solchen Liste berechnet werden. Du möchtest nun auch die Längen von Listen berechnen, die Buchstaben, Booleans oder Gleitkommazahlen enthalten. Was stört dich am bisherigen Vorgehen? Kennst du ein Konzept mit dessen Hilfe du mit weniger Arbeit an dein Ziel kommst?

### TEST 20

Wie ist die Funktion `lengthIntList :: IntList → Int` aus dem vorherigen Test definiert?

### TEST 21

Du hast einen Datentypen definiert und möchtest dir Werte des Typen nun z.B. im GHCi anzeigen lassen. Was kannst du tun, um dieses Ziel zu erreichen?

### TEST 22

Wie definieren wir Funktionen?

---

Frank ist traurig, wenn du insbesondere diese Frage nicht beantworten kannst.

### TEST 23

Gebe ein Listendatentypen an, für den es nicht möglich ist, kein Element zu enthalten.

---

In Haskell heißt dieser Datentyp `NonEmpty` und ist definiert in `Data.List.NonEmpty`.

### Selbstevaluation

Ich bin in der Lage, ...

- (monomorphe) algebraische Datentypen zu definieren,
- Funktionen induktiv über den Datentypen zu definieren und
- Listen zu verwenden und kenne wichtige Funktionen auf Listen.

## Polymorphismus

### Referenzen

- Skript: Funktionale Programmierung, Polymorphismus
- [Making Our Own Types and Typeclasses – Learn You a Haskell for Great Good!](#) („Type parameters“, „Recursive data structures“)
- [Recursion – Learn You a Haskell for Great Good!](#) („Type parameters“, „Recursive data structures“)

#### TEST 24

Wie sieht eine Datentypdefinition in Haskell im Allgemeinen aus?

#### TEST 25

Welchen Typ haben

- `(:)` und `[]`,
- `Just` und `Nothing`,
- `Left` und `Right`?

---

Datenkonstruktoren sind „first-class values“ bzw. Funktionen und haben entsprechend einen Typen. Das ist unter anderem von besonderer Bedeutung für die natürliche Faltung (die wir später kennenlernen).

#### TEST 26

Was ist parametrischer Polymorphismus?

#### TEST 27

Welche Typkonstruktoren des kinds `* → *` oder `* → * → *` kennst du?

#### TEST 28

Welche kinds haben jeweils `Either` und `Either a`?

#### TEST 29

Beim Programmieren in Haskell vernachlässigen wir redundante Syntax. Gibt es in Haskell einen Unterschied zwischen `f 1 2` und `f(1, 2)`?

#### TEST 30

Welches Konzept erlaubt es uns, dass wir Funktionen auf Listen nicht für jeden konkreten Typen angeben müssen?

#### TEST 31

Wie gewinnt man aus einem Typkonstruktor einen Typ?

#### TEST 32

Visualisiere `[1, 2, 3]` als Termbaum, wie du es in der Vorlesung kennengelernt hast. Zur Erinnerung: Die inneren Knoten sind Funktionen und die Blätter Werte, die nicht weiter ausgerechnet werden können.

### TEST 33

Ist `[32, True, "Hello, world!"]` ein valider Haskell-Wert? Warum ja oder nein?

### TEST 34

Was ist der Unterschied zwischen einem Typ, einem Datenkonstruktor und einem Typkonstruktor?

### TEST 35

Gegeben ist

```
data Pair a b = Pair a b
```

Wie unterscheidet sich der Typ von

```
data Pair a = Pair a a
```

### CHALLENGE 1

#### Hinweis 1

- Der größte gemeinsame Teiler (ggT) zweier Ganzzahlen kann mithilfe des euklidischen Algorithmus berechnet werden. Implementiere das Verfahren.

$$\text{gcd}(x, y) = \begin{cases} |x| & \text{falls } y = 0, \\ \text{gcd}(y, x \bmod y) & \text{sonst.} \end{cases}$$

- Alternativ kann der ggT auch berechnet werden, indem wir das Produkt des Schnittes der Primfaktorzerlegung der beiden Zahlen betrachten, also

$$\prod (\text{PF}(x) \cap \text{PF}(y)),$$

wobei PF die Menge der Primfaktoren der gegebenen Zahl (mit entsprechenden Mehrfachvorkommen) beschreiben soll. Implementiere diesen Ansatz.

## CHALLENGE 2



Die Ableitung einer Funktion  $f : \mathbb{R} \rightarrow \mathbb{R}$  kann mithilfe des Differenzenquotienten  $\frac{f(x+h)-f(x)}{h}$  für kleines  $h$  approximiert werden. Eine andere Methode zur Berechnung der Ableitung ist symbolisches Differenzieren und ähnelt dem, wie wir händisch Ableitungen berechnen. Eine Funktion sei dargestellt durch den folgenden Typ:

```
data Fun = X          -- x      (Variable x)
          | E          -- e      (Euler's constant)
          | Num Double -- c      (Constant)
          | Ln Fun     -- ln     (Natural logarithm)
          | Fun :+: Fun -- f + g (Addition)
          | Fun :-: Fun -- f - g (Subtraction)
          | Fun *: Fun  -- f * g (Multiplication)
          | Fun :/: Fun -- f / g (Division)
          | Fun <.: Fun -- f o g (Composition)
          | Fun ^.: Fun -- f ^ g (Exponentiation)

-- Example
f :: Fun
f = (E ^.: X) <.: (X *: X) -- (e^x) o (x * x) = e^(x^2)

-- Example
g :: Fun
g = let x = X
     in x2 = x *: x
     x3 = x2 *: x
     in x3 :+: x2 :+: x :+: Num 1.0 -- x^3 + x^2 + x + 1
```

- Gebe die Funktion  $\sigma(x) = \frac{1}{1+e^{-x}}$  als Wert des Typs `Fun` an.
- Implementiere eine Funktion `($$) :: Fun -> Double -> Double`, die eine gegebene Funktion in einem gegebenen Punkt auswertet.
- Implementiere eine Funktion `derive :: Fun -> Fun`, die eine gegebene Funktion ableitet. Die Funktionen müssen nach dem Ableiten nicht vereinfacht werden.

Hier findest du eine [Zusammenfassung der Ableitungsregeln](#).

## CHALLENGE 3

Deep Dive

In Franks Einführung in die Algorithmik hast du verschiedene Varianten des `mergesort`-Algorithmus kennengelernt. Eine davon hat ausgenutzt, dass in einer Eingabeliste bereits nicht-absteigend sortierte Teillisten vorkommen können, um den Algorithmus zu beschleunigen. Implementiere diese Variante in Haskell.

Für den Anfang kannst du annehmen, dass die Eingabelisten vom Typ `[Int]` sind. Wenn wir Typklassen behandelt haben, kannst du `Ord a => [a]` nutzen – oder du nutzt letzteres und behandelst es erstmal so, als wäre es ersteres.

In der Haskell base-library wird `sort` aus `Data.List` vergleichbar implementiert: [Data.List.sort](#).

#### CHALLENGE 4

Entwickle einen Datentyp **Ratio**, um rationale Zahlen

$$\frac{p}{q} \in \mathbb{Q}, \quad p \in \mathbb{Z}, q \in \mathbb{N}, p \text{ und } q \text{ teilerfremd}$$

darzustellen. Implementiere die Operationen: Addition, Subtraktion, Multiplikation, Division. Implementiere weiter eine Funktion, die die rationale Zahl als reelle Zahl mit einer festen Anzahl von Nachkommastellen darstellt.

Später kannst du auch hier die jeweiligen Typklassen verwenden, um die arithmetischen Operationen zu überladen.

#### TEST 36

Wie können wir es hinkriegen, dass die invalide Liste `[32, True, "Hello, world!"]` ein valider Haskell-Wert wird? Mithilfe welches Hilfstypen kriegen das hin? (Die Liste müssen wir dafür unter Umständen umschreiben.)

#### TEST 37

Du hast bereits viele Funktionen kennengelernt, die in der Haskell **base**-library implementiert sind. Anstatt eine konkrete Liste dieser Funktionen anzugeben, möchten wir dich motivieren, folgende Dokumentationen verschiedener Module anzuschauen.

- [Prelude](#)
- [Data.List](#)

Wenn du merkst, die Implementierung einer bekannten Funktion fällt dir ad hoc nicht ein, nehme dir Zeit und überlege, wie du sie implementieren könntest.

#### TEST 38

Hier ist eine fehlerhafte Implementierung eines Datentyps für einen knotenbeschrifteten Binärbäumen.

```
data Tree a = Empty | Node Tree a Tree
```

Was ist der Fehler?

### TEST 39

In imperativen Programmiersprachen (hier Java) iterieren wir über Listen oft in folgender Form.

```
List<Integer> a = new ArrayList<>();
a.add(3); a.add(1); a.add(4); a.add(1); a.add(5);

List<Integer> b = new ArrayList<>();
for (int i = 0; i < a.size(); i++) {
    b.add(2 * a.get(i));
}
```

Wenn wir diesen Code naiv in Haskell übersetzen, könnten wir z.B. folgenden Code erhalten.

```
double :: [Int] -> Int -> [Int]
double xs i | i < length xs = 2 * xs !! i : double xs (i + 1)
            | otherwise     = []
```

Das wollen wir niemals so tun.

- Was gefällt uns nicht?
- Wie unterscheiden sich die Laufzeiten?
- Optimierte die Funktion `double`, sodass sie lineare Laufzeit in der Länge der Liste hat.

Dein Ergebnis sollte Haskell-idiomatisch sein.

### TEST 40

Die `(!!)`-Funktion ist unsicher in dem Sinne, dass sie für invalide Listenzugriffe einen Fehler wirft – also z.B. für `xs !! (-1)` oder `xs !! k` mit `k > length xs`. Die Funktion `(!?) :: [a] -> Int -> Maybe a` ist eine sichere Variante von `(!!)`. Sie macht den Fehlerfall explizit durch die Wahl des Ergebnistypen. Wie fängt der Ergebnistyp diesen Fehlerfall auf? Implementiere diese Funktion.

---

Diese Funktion ist auch bereits vorimplementiert: `(!?)` in `Data.List`.

### TEST 41

Gegeben sei der Datentyp für knotenbeschriftete Binärbäume

```
data Tree a = Empty | Node (Tree a) a (Tree a).
```

Mithilfe einer Pfadbeschreibung können wir uns durch so einen Baum navigieren. Diese Beschreibung soll durch eine Liste von Werten vom Typ `data D = L | R` dargestellt sein.

Implementiere eine Funktion `(!?) :: Tree a -> [D] -> Maybe a`, die die Beschriftung des Knotens zurückgibt, der durch die gegebene Pfadbeschreibung gefunden wird. Hier sind kleine Beispiele:

- `Node Empty 3 Empty !? [] = Just 3`
- `Node Empty 3 Empty !? [L] = Nothing`
- `Node Empty 3 Empty !? [R] = Nothing`
- `Node (Node Empty 1 (Node Empty 2 Empty)) 3 Empty !? [L, R] = 2`

---

Da Datenkonstruktoren in Haskell nicht überladen werden können, können hier wir leider nicht `Left` und `Right` verwendet werden, solange die Datenkonstruktoren des `Either`-Typs im scope sind.

#### TEST 42

`(++) :: [a] → [a] → [a]` wird verwendet, um zwei Listen aneinanderzuhängen. Wenn wir eine Funktion induktiv über den Listentypen definieren wie z.B. `square :: [Int] → [Int]`, die jeden Listeneintrag quadrieren soll, dann können wir das wie folgt tun.

```
square :: [Int] → [Int]
square [] = []
square (x:xs) = [x * x] ++ square xs
```

Die Funktion ist zwar korrekt aber nicht Haskell-idiomatisch, d.h., eine Person, die Erfahrung im Programmieren von Haskell ist, würde dies nicht so schreiben. Was müssten wir an der Funktion ändern, damit sie idiomatisch wäre.

#### TEST 43

Die Funktion `show` kann genutzt werden, um Werte eines beliebigen Datentyp in eine String-Repräsentation zu überführen. Warum kann `show` nicht als Funktion vom Typ `a → String` implementiert sein?

#### TEST 44



Da viele von euch Neulinge sind, wenn es um das Programmieren in Haskell geht, brauchen wir eine neue Sammlung von Phrasen, die wir nutzen können, um natürlich-sprachliche Anweisungen in Haskell zu formulieren. Finde „Programmierschablonen“ für folgende Anweisungen:

- „für jedes Element einer Liste, tue...“
- „solange ein Wert etwas erfüllt, tue...“
- „für jedes Element einer Liste, das eine Bedingung erfüllt, tue...“
- „überprüfe ein Element existiert, das...“
- „überprüfe für alle Elemente, ob...“
- „betrachte alle Kombinationen zweier Listen, für jede Kombination tue...“

Versuche so viele, wie möglich zu finden, die verschiedene Konzepte des Moduls aufgreifen.

Als Beispiel für die erste Anweisung, können wir z.B. eine induktiv definierte Funktion implementieren, die vom Muster her so aussehen könnte

```
f [] = e          -- Wenn wir keine Elemente mehr betrachten können,
                  -- dann e
f (x:xs) = g x (f xs) -- tue g in Abhängigkeit von x und (f xs)
                  -- f xs betrachtet die Elemente, die wir noch nicht
                  -- verarbeitet haben
```

`e` und `g` sind hier Platzhalter für die Dinge, die wir tun wollen.

#### TEST 45

Manche Funktionen lassen sich nur partiell sinnvoll definieren. Implementiere die Funktionen

- `head, last :: [a] → a`, die das erste und letzte Element einer Liste zurückgeben,
- `init, tail :: [a] → [a]`, die die ersten und letzten  $n - 1$  Elemente einer  $n$ -elementigen Liste zurückgeben,
- `(!!) :: [a] → Int → a`, das ein Element an einer bestimmten Position in einer Liste zurückgibt,
- `minimum, maximum :: Ord a ⇒ [a] → a`, die das Minimum bzw. Maximum in einer Liste zurückgeben,
- `fromJust :: Maybe a → a`, dass das Element aus einem **Just**-Wert herausholt,

und überlege dir, für welche Elemente des Definitionsbereichs du keine sinnvolle Regel angeben kannst. Überlege es dir ebenso für die Funktionen `div, mod :: Int → Int → Int` und `read :: Read a ⇒ String → a`, die du nicht implementieren musst.

#### Selbstevaluation

Ich bin in der Lage, ...

- (parametrischen) Polymorphismus zu erklären,
- polymorphe Datentypen zu definieren, und
- polymorphe Funktionen zu definieren.

## Pattern Matching

#### Referenzen

- Skript: Funktionale Programmierung, Pattern Matching
- [Syntax in Functions - Learn You a Haskell for Great Good!](#) („Pattern matching“, „Guards, guards!“, „Case expressions“)

#### TEST 46

In Programmiersprachen wie Java greifen wir auf Daten komplexer Datentypen zu, indem wir auf Attribute von Objekten zugreifen oder getter-Methoden verwenden. Wie greifen wir auf Daten in Haskell zu?

## CHALLENGE 5

### Hinweis 1

In Haskell sind Listen als einfach-verkettete Listen implementiert. Das macht sie ungeeignet für Operationen, die wahlfreien Zugriff in konstanter Laufzeit benötigen. Darüber hinaus sind Listen auch nicht mutierbar. Das führt dazu, dass Operationen, die eine Liste verändern, häufig lineare Laufzeit in der Länge der Liste haben – mit Ausnahme der `(:)`-Operation.

Ziel dieser Challenge ist es, eine Datenstruktur zu entwickeln, die eine (amortisiert) konstante `append`-Operation hat. Diese ist bekannt als **Queue**. Sie soll durch `data Queue a = Q [a] [a]` dargestellt werden. Die Idee ist es, eine (linke) Liste vorzuhalten, die eine (amortisiert) konstante `dequeue`-Operationen erlaubt, und eine andere (rechte), die eine konstante `enqueue`-Operationen erlaubt. Das heißt, fast alle dieser Operationen benötigen konstante Laufzeit und konstant wenige können lineare Laufzeit haben.

Implementiere die Funktionen

- `empty :: Queue a`, die eine leere Queue erzeugt,
- `front :: Queue a → a`, die das erste Element in einer queue zurückgibt,
- `isEmpty :: Queue a → Bool`, die bestimmt, ob eine queue leer ist,
- `enqueue :: a → Queue a → Queue a`, die ein Element an das Ende einer queue anhängt,
- `dequeue :: Queue a → Queue a`, die das erste Element in einer queue entfernt.

Die Implementierung soll dabei folgende Invariante erfüllen: Eine queue ist genau dann leer, wenn die `dequeue`-Liste leer ist. Diese Invariante kannst du z.B. mit einer Hilfsfunktion erzwingen – oder du passt bei der Implementierung deiner Funktionen auf. Falls es dir für den Anfang einfacher fällt, ignoriere die Invariante erstmal.

Wenn sogar `length xs ≥ length ys` für eine queue `Q xs ys` gewährleistet wird, ist die queue nochmal schneller. Dafür muss man die Längen der Listen immer vorhalten. Mehr darüber findest du in [Simple and efficient purely functional queues and dequeues](#) von Chris Okasaki lesen. Falls dich funktionale Datenstrukturen allgemein interessieren, sei dir [seine Doktorarbeit](#) empfohlen.

## CHALLENGE 6

### Level Up

In den Übungsaufgaben hast du einen Suchbaum ohne Höhenbalancierung implementiert. Die Rotationen für einen AVL-Baum lassen sich durch das pattern matching in Haskell vergleichsweise elegant implementieren - erinnere dich z.B. an die Implementierung aus Einführung in die Algorithmik, die recht verbos ist.

Die Höhe eines Teilbaums kann z.B. als weiteres Attribut im Knoten gespeichert werden. Eine ineffizientere Variante ist es, die Höhe mit einer Funktion wiederkehrend zu berechnen. Letztere Variante ist für den Anfang übersichtlicher.

Implementiere eine Funktion `rotate :: SearchTree a → SearchTree a`, die einen Teilbaum an der Wurzel rebalanciert, sollte der Teilbaum unbalanciert sein. Diese Funktion kannst du dann nutzen, um die gängigen Operationen auf Suchbäumen anzupassen.

[Rebalancierung eines AVL-Baum.](#)

## Selbstevaluation

Ich bin in der Lage, ...

- Pattern Matching zu nutzen.

## Automatisches Testen

### Referenzen

- Skript: Funktionale Programmierung, Automatisches Testen

#### TEST 47

Formuliere QuickCheck-Eigenschaften, die die Funktionen

- `isElem :: Int → SearchTree Int → Bool`,
- `toList :: SearchTree Int → [Int]`,
- `insert :: Int → SearchTree Int → SearchTree Int` und
- `delete :: Int → SearchTree Int → SearchTree Int`

erfüllen sollen. `isElem` überprüft, ob eine Ganzzahl in gegebenen Suchbaum enthalten ist. `toList` konvertiert einen Suchbaum in eine Liste. `insert` fügt eine Ganzzahl in einen Suchbaum ein. `delete` löscht eine Ganzzahl aus einen Suchbaum.

Wie kannst du die Suchbaum-Eigenschaft spezifizieren? Zum Spezifizieren der Eigenschaften benötigst du möglicherweise weitere Funktionen.

#### TEST 48

QuickCheck-Eigenschaften werden mit zufällig generierten Werten getestet. Hin und wieder kommt es vor, dass diese Werte Vorbedingungen erfüllen müssen, damit wir Eigenschaften von Funktionen testen können. Wie können wir das erreichen?

#### TEST 49

Wie können wir für eine beliebige Eingabe verifizieren, dass die Ausgabe korrekt ist? Wieso können wir mithilfe dieses Ansatzes in den meisten Fällen nicht die Korrektheit einer Funktion zeigen?

#### TEST 50

Welchen Nachteil hat die Prüfung von Vorbedingungen mit  $(\Rightarrow)$ ? Wie können wir diese beheben?

### Selbstevaluation

Ich bin in der Lage, ...

- Eigenschaften mit QuickCheck zu spezifizieren
- ... mit Kombinatoren wie z.B.  $\Rightarrow$ , und
- Vor- und Nachteile des eigenschaftsbasierten Testens zu diskutieren.

## Funktionen höherer Ordnung

### Referenzen

- Skript: Funktionale Programmierung, Funktionen höherer Ordnung
- [Higher Order Functions – Learn You a Haskell for Great Good!](#)

#### TEST 51

Was sind Funktionen höherer Ordnung?

#### TEST 52

Wie definieren wir Lambda-Abstraktionen bzw. anonyme Funktionen?

#### TEST 53

Warum ist der Typ  $(a \rightarrow b) \rightarrow c$  nicht identisch zum Typ  $a \rightarrow b \rightarrow c$ ? Welcher andere Typ ist identisch zu letzterem?

#### TEST 54

Mit welchen Konzepten gehen die Linksassoziativität der Funktionsapplikation und die Rechtsassoziativität des Typkonstruktors  $(\rightarrow)$  gut Hand in Hand?

#### TEST 55

Zu welchen partiell applizierten Funktionen verhalten sich folgende Funktionen identisch?

- `succ :: Int → Int` (die Inkrementfunktion)
- `pred :: Int → Int` (die Dekrementfunktion)
- `length :: [a] → Int`
- `sum :: [Int] → Int`
- `product :: [Int] → Int`

#### TEST 56

Sectioning ist das Erstellen einer neuen Funktion durch teilweises Anwenden eines Operators. Zum Beispiel entspricht `(+ 1)` der Inkrementfunktion und `(* 2)` der Verdopplungsfunktion. Warum ist aber `(- 1)` keine gültige section? Welche Rolle spielt `subtract` in diesem Kontext?

#### TEST 57

Was ist partielle Applikation?

#### TEST 58

Was ist Currying?

#### TEST 59

Welche Funktionen höherer Ordnung hast du kennengelernt im Kontext der generischen Programmierung? Was ist das Ziel dieser Funktionen?

### TEST 60

Gebe `map` und `filter` unter der Unterverwendung von `foldr` an.

### TEST 61

Gegeben seien folgende Funktionen:

- `rgbToHsv :: RGB → HSV`, die eine Farbe von einer Darstellung in eine andere konvertiert, und
- `hue :: HSV → Float`, die den Farbwert einer Farbe im Wertebereich  $[0^\circ, 360^\circ)$  im HSV-Farbraum zurückgibt.

Du bekommst als Eingabe einen Bild, das hier als Liste von `RGB`-Werten dargestellt ist. Jeder `RGB`-Wert korrespondiert zu einem Pixel. Schreibe eine Funktion, die berechnet, wie viele blaue Pixel das Bild hat. Hier bezeichne eine Farbe als blau, wenn ihr Farbwert zwischen  $200^\circ$  und  $250^\circ$  (inklusive) liegt. Nutze für die Definition der Funktion sowohl `map` als auch `filter`.

### TEST 62

Level Up

Mit Funktionen höherer Ordnung können wir Kontrollstrukturen aus der imperativen Programmierung definieren. Hier ist eine mögliche Definition einer bedingten Wiederholung.

```
while :: (a → Bool) → (a → a) → a → a
while p f x | p x      = while p f (f x)
             | otherwise = x
```

In [Test 7](#) haben wir zwei Funktionen gesehen, die diese Kontrollstruktur verwenden. In Haskell können wir `clz` mithilfe von `while` wie folgt definieren.

```
clz :: Int → Int
clz n = snd (while cond step (n, 0))
  where
    cond (n, k) = n > 0
    step (n, k) = (n `div` 2, k + 1)
```

Oder alternativ so

```
clz :: Int → Int
clz = go 0
  where
    go k n | n > 0      = go (k + 1) (n `div` 2)
           | otherwise = 64 - k
```

Beide Funktionsdefinitionen sind semantisch äquivalent. Argumentiere unter verschiedenen Aspekten, warum die eine Implementierung besser als die andere sein könnte – z.B. in Hinsicht auf Lesbarkeit, Idiomatik und Wartbarkeit.

### TEST 63

Was sind sections im Kontext von Funktionen höherer Ordnung?

### TEST 64

Welche der Faltungsfunktion auf Listen ergibt sich aus dem Verfahren zur Erzeugung von Faltungsfunktionen?

#### TEST 65

Was ist der Unterschied zwischen `foldl` und `foldr`? Wann liefern `foldl` und `foldr` das gleiche Ergebnis?

#### TEST 66

Wie gewinnen wir aus `foldr` die Identitätsfunktion auf Listen? In den Übungen hast du gelernt, wie man Werte anderer Typen falten kann. Wie gewinnt man aus diesen Funktionen die Identitätsfunktionen auf den jeweiligen Typen?

#### TEST 67

Gegeben sind folgende Datentypen

- `data Tree a = Empty | Leaf a | Branch (Tree a) (Tree a),`
- `data Rose a = Node [Rose a].`

Welche Typen haben die jeweiligen Datenkonstruktoren und wie führen wir diese in die Signatur der jeweiligen Faltungsfunktion über? Wo benötigen wir rekursive Aufrufe der jeweiligen Faltungsfunktionen?

#### TEST 68

Betrachte die Funktion

```
f :: [a] -> b
f []      = e
f (x:xs) = g x (f xs)
```

Nach diesem induktiven Muster sind viele Funktionen auf Listen implementiert. Nehme als Beispiel die Funktion `sum :: [Int] -> [Int]`.

```
sum :: [Int] -> Int
sum []      = 0
sum (x:xs) = x + sum xs
```

Dieses Muster haben wir in `foldr` abstrahiert. Wo wandern die jeweiligen Bestandteile der abstrakten Funktion `f` hin, wenn wir `f` mithilfe von `foldr` definieren. Was passiert insbesondere mit dem rekursiven Aufruf von `f`?

#### TEST 69

Wie kannst du mithilfe von Faltung viele Elemente in einen Suchbaum einfügen oder lösen? Implementiere

- `insertMany :: [Int] -> SearchTree Int -> SearchTree Int` und
- `deleteMany :: [Int] -> SearchTree Int -> SearchTree Int.`

Du kannst davon ausgehen, dass du die Einfüge- und Löschfunktion für einzelne Elemente bereits hast.

### TEST 70

Wir können `map :: (a → b) → [a] → [b]` mithilfe von `foldr` wie folgt implementieren:

```
map f xs = foldr (\x ys → f x : ys) [] xs
```

Vereinfache den Lambda-Ausdruck mithilfe von Funktionen höherer Ordnung.

### TEST 71

Wenn wir die Listenkonstruktoren in `foldr` einsetzen, erhalten wir die Identitätsfunktion auf Listen, also

```
foldr (:) [] :: [a] → [a].
```

Wenn wir das Gleiche mit `foldl` und angepassten `(:)` machen, also

```
foldl (flip (:)) [] :: [a] → [a],
```

dann erhalten wir nicht die Identitätsfunktion auf Listen. Warum und was bekommen wir stattdessen heraus?

### TEST 72

Es gibt viele andere hilfreiche Funktionen höherer Ordnung in der Haskell Prelude. Eine von diesen ist `zipWith :: (a → b → c) → [a] → [b] → [c]`. Sie verknüpft jeweils zwei Elemente aus den jeweiligen Listen unter der gegebenen Funktion.

- Implementiere `zipWith` mithilfe von `map`, `uncurry`, `zip`.
- Implementiere `zip` mithilfe von `zipWith`.
- Implementiere das Prädikat `isSorted` mithilfe von `zipWith`.

### TEST 73 ☹

Deep Dive

Das Pendant zum Falten ist das Biegen mit `foldr` ist `unfoldr` (aus `Data.List`). Anstatt eine Liste von Werten zu falten, können wir mit `unfoldr` aus einem Wert eine Liste erzeugen. Die Funktion hat den Typ `(b → Maybe (a, b)) → b → [a]`.

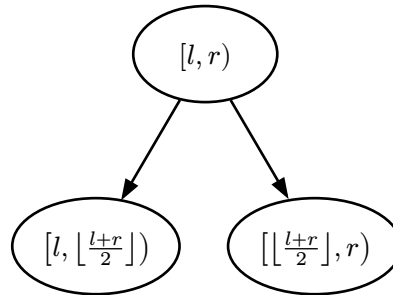
- Überlege dir anhand des Typs, wie diese Funktion implementiert sein könnte. Implementiere sie anschließend.
- Können wir `foldr (+) 0 [3, 1, 4, 1, 5]` mit `unfoldr` rückgängig machen? Das heißt, können wir die Eingabeliste rekonstruieren, ohne Annahmen darüber zu machen, wie die Eingabeliste entstanden ist?
- Berechne die Binärdarstellung einer natürlichen Zahl mithilfe von `unfoldr`. (Das LSB soll an erster Stelle der Ausgabeliste stehen.) Implementiere dies als Funktion `bits :: Int → [Int]`.
- Implementiere `map` mithilfe von `unfoldr`.

## TEST 74

## Deep Dive

In diesem Test wollen wir einen beliebigen Wert zu einen Baum entfalten. Implementiere eine Funktion `unfoldTree :: (b → Maybe (b, a, b)) → b → Tree a`, die eine Funktion nimmt, die ein Wert von Typ `b` nimmt und in eine Knotenbeschriftung und zwei weitere Wert vom Typ `b` aufspaltet, oder das Entfalten stoppt, indem ein `Nothing` zurückgegeben wird. Bäume sind durch den Datentyp `data Tree a = Empty | Node (Tree a) a (Tree a)` definiert.

Nutze die definierte Funktion, um einen Binärbaum zu erzeugen, welche die Zerlegungen eines diskreten Intervalls darstellen soll. Entnehme die Art der Zerlegung dem Diagramm.



## CHALLENGE 7

Gegeben sei die Faltungsfunktion `foldTree :: (r → a → r → r) → r → Tree a → r` für einen knotenbeschrifteten Binärbaum gegeben durch `data Tree a = Empty | Node (Tree a) a (Tree a)`.

Eine Reihe von Funktionen, die du bereits für Listen kennengelernt hast, lassen sich auch auf Bäume übertragen.

Implementiere die Funktionen

- `any :: (a → Bool) → Tree a → Bool` und `and :: (a → Bool) → Tree a → Bool`,
- `elem :: Int → Tree Int → Bool` und `notElem :: Int → Tree Int → Bool`,
- `toList :: Tree a → [a]`,
- `null :: Tree a → Bool` (überprüft, ob der Baum leer ist),
- `length :: Tree a → Int`,
- `maximum :: Tree Int → Int` und `minimum :: Tree Int → Int`, und
- `sum :: Tree Int → Int` und `product :: Tree Int → Int`.

Diese Funktionen lassen sich auf alle faltbaren Datentypen verallgemeinern. Dies wird mithilfe der Typklasse `Foldable` festgehalten.

### TEST 75

Gegeben seien die Funktion

```
f :: a → b
g :: a → b → c
```

sowie die Kompositionsfunktion `(.) :: (b → c) → (a → b) → a → c`.

In der Typdefinition von `(.)` scheint das erste Argument, eine einstellige Funktion zu sein. Ist der Ausdruck

`g . f`

trotzdem typkorrekt, obwohl `g` eine zweistellige Funktion ist? Wenn ja, wie werden die Typvariablen – insbesondere das `c` der Komposition – unifiziert?

### TEST 76

Gegeben sei der Datentyp `data Tree a = Empty | Node (Tree a) a (Tree a)` und die Faltungsfunktion `foldTree :: r → (r → a → r → r) → Tree a → r`.

Vergewissere dich, dass die Implementierung der folgenden Funktion, die alle Beschriftungen durch den gleichen Wert ersetzt, korrekt ist.

```
replace :: b → Tree a → Tree b
replace x = foldTree Empty (const . flip Node x)
```

Zeige, dass `const . flip Node x = \l y r → Node l x r` ist.

Es gibt Programmierende, die einen sogenannten punktfreien Stil bevorzugen. Weil es teilweise unterhaltsam sein kann, sich darüber Gedanken zu machen, was eine äquivalente punktfreie Funktion zu einem Lambda-Ausdruck sein könnte, steht hier eben `const . flip Node x` statt `\l _ r → Node l x r`. Falls du damit herumspielen möchtest, findest du hier [eine Seite](#), auf der du Haskell-Code in punktfreien Haskell-Code übersetzen kannst.

### TEST 77

Welche Funktion verbirgt sich hinter `foldr ((+) . f) []` und was ist ihr Typ?

### TEST 78

Versuche in den folgenden Ausdrücken, Teilausdrücke schrittweise durch bekannte Funktionen zu ersetzen oder gegebenenfalls zu vereinfachen.

- `foldr (\x ys → f x : ys) [] (foldr (\x ys → g x : ys) [] xs)`,
- `map (\_ → y) xs`,
- `foldr (\x ys → if x `mod` 2 == 1 then x - 1 : ys else ys) [] xs`,
- `foldl (\ys x → x : ys) [] xs` und
- `flip (curry snd) x`.

„Your scientists were so preoccupied with whether or not they could, that they didn't stop to think if they should.“ Jenseits solcher kleinen Verständnisfragen gilt weiterhin, dass wir verständlichen Code schreiben wollen. Solche Ausdrücke wie `flip (curry snd) x` sind häufig schwieriger zu verstehen – auch wenn es unterhaltsam ist, sich solche Ausdrücke auszudenken.

### CHALLENGE 8

Sei  $m, n \in \mathbb{N}$ . Gegeben seien

- die Identitätsfunktion auf  $\{0, \dots, n-1\}$  mit  $\pi_0 : \{0, \dots, n-1\} \rightarrow \{0, \dots, n-1\}, x \mapsto x$  und
- eine endliche Folge von Paaren  $((a_i, b_i))_{i \in \{1, \dots, m\}}$  mit  $a_i \in \{0, \dots, n-1\}, b_i \in \{0, \dots, n-1\}$  für alle  $i \in \{1, \dots, m\}$ .

Wir definieren  $\pi_{i,j}$  als

$$\pi_{i,j}(k) = \begin{cases} i & \text{falls } k = j \\ j & \text{falls } k = i \\ k & \text{sonst} \end{cases}$$

für alle  $k \in \{0, \dots, n-1\}$ .

Wir betrachten die Paare als Vertauschungen der Bilder der Abbildung  $\pi_0$ , d.h.,

$$\pi_i = \pi_{a_i, b_i} \circ \pi_{i-1} \text{ für } i \in \{1, \dots, m\}.$$

- Implementiere eine Funktion `swaps :: Int -> [(Int, Int)] -> [Int]`, die  $\pi_m$  mithilfe von Listen berechnet. Der erste Parameter bestimmt die Menge  $\{0, \dots, n-1\}$  und der zweite die Folge.
- Implementiere eine Funktion `swaps :: Int -> [(Int, Int)] -> Int -> Int`, die  $\pi_m$  mithilfe von Funktion berechnet. (Hier ist der erste Parameter unter Umständen redundant.)
- Welche Vor- und Nachteile haben die jeweiligen Ansätze im Vergleich?

### TEST 79

Eta-reduziere die folgende Ausdrücke:

- `sum xs = foldr (+) 0 xs`,
- `add a b = a + b` und
- `\x ys -> (:) x ys`.

### TEST 80

Implementiere die Funktion `insert :: Int -> a -> Map Int a -> Map Int a`, die ein Schlüssel-Wert-Paar in eine `Map Int a` einfügt. Die `Map` ist wie folgt repräsentiert `type Map k v = k -> v`.

### TEST 81

Level Up

Wir haben `foldr :: (a -> b -> b) -> b -> [a] -> b` als natürliche Faltungsfunktion kennengelernt, die einen Ausdruck erzeugt, der rechts geklammert ist. Zum Beispiel gilt

$$\text{foldr } (+) \ 0 \ [1, 2, 3] = 1 + (2 + (3 + 0)).$$

Das gleiche Ziel können wir mit anderen Typen verfolgen. Implementiere eine Funktion `foldr :: (a -> b -> b) -> b -> Tree a -> b` für einen blattbeschrifteten Binärbaum `data Tree a = Leaf a | Tree a :+: Tree a`, die den gleichen Ausdruck erzeugt. Zum Beispiel soll

$$\text{foldr } (+) \ 0 \ ((\text{Leaf } 1 \text{ :+: Leaf } 2) \text{ :+: Leaf } 3) = 1 + (2 + (3 + 0))$$

gelten.

Das `foldr` für `Tree a` entspricht nicht der natürlichen Faltung auf diesem Typen.

## TEST 82

Gegeben sei folgendes Python-Programm.

```
from dataclasses import dataclass
from typing import Generic, TypeVar

class Foldable():
    def foldr(self, f):
        pass

    def sum(self):
        return self.foldr(lambda x: lambda ys: x + ys)(0)

    def toList(self):
        return self.foldr(lambda x: lambda ys: [x] + ys)([])

    def __len__(self):
        return self.foldr(lambda _: lambda s: 1 + s)(0)

    def __contains__(self, y):
        return self.foldr(lambda x: lambda z: z or x == y)(False)

    # ...

T = TypeVar('T')

class Tree(Generic[T], Foldable):
    def foldr(self, f):
        def foldr_with_f(e):
            match self:
                case Empty():
                    return e
                case Node(l, x, r):
                    y = f(x)(r.foldr(f)(e))
                    z = l.foldr(f)(y)
                    return y
            return foldr_with_f

@dataclass
class Empty(Tree[T]):
    pass

@dataclass
class Node(Tree[T]):
    left: Tree[T]
    value: T
    right: Tree[T]

tree = Node(Empty(), 3, Node(Node(Empty(), 7, Empty()), 4, Empty()))
print(tree.sum()) # 14
print(tree.toList()) # [3, 7, 4]
print(len(tree)) # 3
print(3 in tree, 9 in tree) # True False
```

In diesem Programm werden viele Konzepte verwendet, die du im Haskell-Kontext kennengelernt hast – aber wahrscheinlich bisher nicht in Python gesehen hast. In diesem Test geht es darum, diese Konzepte im Python-Programm zu identifizieren.

Wo findest du

- Funktionen höherer Ordnung,
- pattern matching,
- algebraische Datentypen (Typkonstruktoren, Datenkonstruktoren),
- parametrischen Polymorphismus,
- ad-hoc Polymorphismus (Typklassen bzw. Überladung) und
- lokale Definitionen.

Typannotationen in Python sind nicht sonderlich elegant. Deshalb sind nur die angegeben, um den parametrischen Polymorphismus zu identifizieren und data classes anständig zu nutzen.

Data classes und match statements brauchst du dir jenseits dieses Tests nicht anschauen (wenn es dich nicht weiter interessiert). Es soll in dem Test nur darum gehen, die Haskell-Konzepte zu erkennen. In [Bemerkung 3](#) kannst du das gleiche Programm in Java sehen.

### TEST 83

In das folgende Python-Programm hat sich ein bug hineingeschlichen.

```
text = 'Ja, ja, ich back mir \'nen Kakao!'
say_words = []

for word in text.split():
    say_words.append(lambda sep: print(word, end=sep))

for say_word in say_words[:-1]:
    say_word(' ')
say_words[-1]('\n')
```

Dieses Programm gibt sieben Mal „Kakao!“ aus. Erkläre wie dieses Verhalten zustande kommt? Wie kannst du den bug beheben? Kann der gleiche Fehler in Haskell passieren?

### TEST 84

Schau dir die Selbsttests dieser Sektion erneut an und versuche, geeignete in Python zu lösen.

### Selbstevaluation

Ich bin in der Lage, ...

- Funktionen höherer Ordnung zu erkennen, zu definieren und zu nutzen,
- wichtige Funktionen höherer Ordnung zu definieren wie
  - `map`, `filter`, `foldr` und `foldl`
  - `(.)`, `($)`, `flip`, `curry` und `uncurry`
- das Zusammenspiel aus Currying und partieller Applikation zu erklären,
- natürliche Faltungsfunktionen für gegebene Datentypen zu definieren,
- Funktionen höherer Ordnung in Python zu nutzen und damit verbundene Fallstricke zu erklären.

## Abstrakte Datentypen

### Referenzen

- Skript: Funktionale Programmierung, Abstrakte Datentypen

#### TEST 85

Was ist ein abstrakter Datentyp? Was sind die Bestandteile eines abstrakten Datentyps?

#### TEST 86

Wie definieren wir die Semantik der zu einem abstrakten Datentyp gehörenden Operationen? Wie definieren wir sie insbesondere nicht?

#### TEST 87

Wieso ist das sofortige Nutzen einer Gleichheit auf einem abstrakten Datentypen problematisch? Was sollte man stattdessen tun?

#### TEST 88

Zur Spezifikation der Semantik nutzen wir Gesetze, die bestimmen, wie verschiedene Operationen miteinander interagieren. Dafür benötigen wir verschiedene Werte oftmals unterschiedlicher Datentypen. Wo kommen diese her und wie sind sie quantifiziert?

#### TEST 89

Welche Eigenschaften sollten die für einen abstrakten Datentypen formulierten Gesetze erfüllen, damit sie eine sinnvolle Semantik beschreiben?

#### TEST 90



Die Pomodoro-Technik ist eine Zeitmanagement-Methode, die einen Kurzzeitwecker nutzt, um Arbeits- und Pauseabschnitte einzuteilen. Um ein Informatik-Studierenden-Klischee zu bedienen, entscheidest dich den Timer selber zu bauen. Da du dich nicht planlos in die Implementierung stürzen möchtest, formulierst du zuerst einen ADT.

Der ADT soll folgende Operationen unterstützen. Es soll möglich sein, ...

- ... einen Timer mit einer Zeiteingabe zu starten – die Zeiteingabe soll aus Stunden und Minuten bestehen,
- die verbleibende Zeit auf dem Timer abzufragen, und
- zu prüfen, ob der Alarm bereits ausgelöst wurde. Der Alarm wird gelöst, sobald die verbleibende Zeit vorüber ist.
- Um das Vergehen von Zeit zu modellieren, soll der ADT weiter eine Funktion bereitstellen, die eine Sekunde vergehen lässt. Wenn keine Zeit mehr verblieben ist, dann hat diese Operation keine Auswirkung.

Gebe für jede Funktion mindestens ein Gesetz an.

---

Einen Lösungsvorschlag für diese Aufgabe findest du im Anhang dieses Dokument als Bemerkung 4. Weiter findest du in Bemerkung 5 auch Anmerkungen zu Fehlern, die in der Klausur des 1. Prüfungszeitraums WS25/26 übertragen auf diese Aufgabe gemacht wurden.

### CHALLENGE 9 ⌚

Gebe folgende abstrakte Datentypen an: Paar, Menge, stack, queue, double-ended queue, knotenbeschrifteter Binärbaum, priority queue.

Anschließend kannst du diese auch (naiv) implementieren und deine Implementierung testen, indem du deine formulierten Gesetze mit QuickCheck implementierst.

### TEST 91

Als Teil eines ADTs für Arrays soll eine Operation `reverse :: Array a → Array a` spezifiziert werden, die ein Array umdreht. Ihr Verhalten soll unter anderem durch das folgende Gesetz festgehalten sein:

$$\text{reverse (reverse a)} == a \quad \text{für alle Arrays } a$$

Warum ist dieses Gesetz problematisch? Wie können wir das Problem beseitigen?

### TEST 92

Als Teil eines ADTs für Array soll eine Operation `at :: Array a → Int → a` spezifiziert werden, die das Element an einer Position in einem Array zurückgibt. Weiter soll `update :: Int → a → Array a → Array a` einen Wert an eine Position in ein Array schreiben.

Wie können wir spezifizieren, dass durch ein `update` nur das Element an der gegebenen Position verändert wird?

### TEST 93

Eine Teilmenge der Operationen für eine Menge sind

- `insert :: a → Set a → Set a` zum Einfügen von einem Wert in eine Menge und
- `size :: Set a → Int` zum Bestimmen der Kardinalität einer Menge.

Gegeben ist folgendes Gesetz:

$$\text{size (insert y (insert x s))} == \text{size s} + 2 \quad \text{für alle Werte } x, y, \text{ alle Mengen } s.$$

Das Gesetz ist falsch. Warum und wie können es korrigieren?

### TEST 94

In ADT-Gesetzen sind Variablen allquantifiziert. Wie können wir gewährleisten, dass ein Wert bestimmte Bedingungen erfüllt, bevor wir ein entsprechendes Gesetz für solche Werte definieren?

### TEST 95

Warum benötigen wir Konstruktoren als Teil eines ADTs?

#### TEST 96

Im Kontext der objektorientierten Programmierung hast du wahrscheinlich das Konzept der Datenkapselung kennengelernt. Dabei geht es um das Verbergen von Daten, sodass Zugriffe von außen nicht möglich sind. In der objektorientierten Programmierung wird dies z.B. durch explizite Angabe von Zugriffsarten für Attribute oder Methoden erreicht.

In Haskell haben wir so etwas **private** und **public** nicht. Wie können wir aber trotzdem verhindern, dass bestimmte Operationen auf Werten eines Datentypen nicht möglich sind? Welche Rolle spielen smart constructors in diesem Zusammenhang?

### TEST 97 ☹

Kritisiere folgenden ADT für eine queue, so pingelig wie du kannst, und bessere ihn.

```
empty :: Queue a
isEmpty :: Queue a → Bool
enqueue :: a → Queue a → Queue a
dequeue :: Queue a → Queue a
front :: Queue a → a
rear :: Queue a → a
clear :: Queue a → Queue a
duplicate :: Queue a → Queue a
reverseQ :: Queue a → Queue a
toList :: Queue a → [a]
fromList :: [a] → Queue a

isEmpty empty = True
isEmpty (enqueue x q) = isEmpty q
isEmpty (clear q) = True
enqueue x empty = empty
enqueue x (enqueue y q) = enqueue y (enqueue x q)
dequeue empty = empty
dequeue (enqueue x q) = q
dequeue q = empty
front empty = front empty
front (enqueue x q) = x
front q = front (dequeue q)
front q = front (enqueue x q)
size empty = 1
size (enqueue x q) = size q
size (dequeue q) = size q + 1
reverseQ empty = empty
reverseQ (enqueue x q) = enqueue x (reverseQ q)
reverseQ (reverseQ q) = q
reverseQ q = q
duplicate empty = empty
duplicate q = enqueue (front q) q
clear q = q
clear q = empty
toList empty = []
toList (enqueue x q) = x : toList q
fromList [] = empty
fromList (x:xs) = enqueue x (fromList xs)
fromList (toList q) = empty
toList (fromList xs) = []
enqueue x q = q
enqueue x q = enqueue y q
dequeue (enqueue x empty) = enqueue x empty
```

### Selbstevaluation

Ich bin in der Lage, ...

- zu erklären, was ein abstrakter Datentyp ist und was er besteht,
- zu erklären, was der theoretische und praktische Nutzen von abstrakten Datentypen ist,
- abstrakte Datentypen anzugeben,
- keine Implementierung für einen ADT anzugeben, wenn nicht explizit danach gefragt ist.

## Typklassen und Überladung

### Referenzen

- Skript: Funktionale Programmierung, Typklassen und Überladung
- [Types and Typeclasses – Learn You a Haskell for Great Good!](#)
- [Making Our Own Types and Typeclasses – Learn You a Haskell for Great Good!](#) („Typeclasses 102“, „A yes-no typeclass“)

### TEST 98

Was sind Typklassen?

### TEST 99

Wie unterscheidet sich der Polymorphismus, der durch Typklassen ermöglicht wird, vom parametrischen Polymorphismus?

### TEST 100

In einem vorherigen Test wurdest du bereits gefragt, wieso `show` nicht als Funktion mit dem Typ `a → String` implementiert sein kann. Wieso wird die Funktion durch den Typ `Show a ⇒ a → String` gerettet?

### TEST 101

Welche Typklassen kennst du? Was ermöglichen sie konkret?

### TEST 102

Eine `Show`-Instanz für den Typ `data Tree a = Leaf a | Tree a :+: Tree a` könnte wie folgt aussehen:

```
instance Show a ⇒ Show (Tree a) where
  show (Leaf x)  = "Leaf " ++ show x
  show (l :+: r) = "(" ++ show l ++ " :+: (" ++ show r ++ ")"
```

Welche Werte vom Typ `Tree a` führen zur worst-case Laufzeit und welche zur best-case Laufzeit? Die Anzahl der Blätter soll hier frei sein. Welche Eigenschaften von `(++)` führen zu den jeweiligen Laufzeiten?

### TEST 103

Überlade die Operationen `(+)`, `(-)`, `(*)`, `abs`, `signum`, `fromInteger` für den Datentypen `data Mat22 a = Mat22 a a a a`, der  $(2 \times 2)$ -Matrizen repräsentieren soll – `abs`, `signum`, `fromInteger` kannst du z.B. komponentenweise implementieren.

Oft sind an Funktionen von Typklassen Bedingungen bzw. Gesetze, die erfüllt werden sollen, gekoppelt. Diese werden durch den Implementierungsvorschlag von `abs` und `signum` nicht erfüllt.

#### TEST 104

Mit

$$\begin{pmatrix} f_{n+1} & f_n \\ f_n & f_{n-1} \end{pmatrix}^n = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n$$

und der binären Exponentiation ([Test 12](#)) und **Mat22 Integer** ([Test 103](#)) aus vorherigen Tests kannst du die  $n$ -te Fibonacci-Zahl in logarithmischer Laufzeit in  $n$  berechnen. Implementiere das Verfahren.

Da du eine **Num**-Instanz auf **Mat22** definiert hast, kannst du den  $(^)$ -Operator zur binären Exponentiation nutzen.

#### TEST 105

Das Hamming-Gewicht lässt sich effizienter als in [Test 7](#) implementieren. Betrachte dafür folgende Haskell-Funktion.

```
popcnt :: Int → Int
popcnt 0 = 0
popcnt w = 1 + popcnt (w .&. (w - 1))
```

Wieso sind `(.&.)` oder `(-)` auf **Int**-Werte anwendbar?

#### TEST 106

Wieso gilt sowohl `14000605 :: Int` als auch `14000605 :: Float`?

#### TEST 107

Was ist der Unterschied zwischen den folgenden Ausdrücken?

```
-- Ausdruck 1
case x of
  0 → a
  _ → b

-- Ausdruck 2
if x == 0 then a else b
```

## CHALLENGE 10 🕒

Deep Dive

Hinweis 1

In dieser Challenge sollst du automatisches Differenzieren im Vorwärtsmodus mithilfe von (Operator-)Überladung implementieren. Dieser Ansatz des Differenzierens führt dabei das Differenzieren komplizierter Funktionen auf einfache, elementare Funktionen zurück.

Wir verwenden folgenden Datentyp: `data D a = D a a`. Ein Wert vom Typ `D a` enthält einen Funktionswert und die Ableitung an einer gegebenen Stelle.

Der Kern der Idee ist, Funktionen so zu überladen, dass sie auf `D a`-Werte angewendet werden können. Angenommen, es sei eine Funktion `f` und ihre Ableitungsfunktion `f'` gegeben. Dann soll ein überladenes `f` wie folgt funktionieren

```
f :: D a -> D a
f (D gx dgdx) = D (f gx) (dgdx * f' gx)
```

Der Wert `gx` ist das Ergebnis einer inneren Funktion `g`, und `dgdx` entspricht deren Ableitung `g'` an der Stelle `x` (bzw.  $\frac{dg}{dx}(x)$ ). Die Kettenregel führt dann zu

$$(f \circ g)' x = g' x * f' (g x) = dgdx * f' gx.$$

Nach dem Muster kannst du nun Standardfunktionen überladen. Für die arithmetischen Operatoren benötigst du an der Stelle deren Ableitungsregeln (Summenregel, Leibnizregel, usw.)

Implementiere die Typklasseninstanzen `Num`, `Fractional` und `Floating`.

Mit den folgenden Funktionen kannst du z.B. die erste oder zweite Ableitung bilden.

```
d1 :: Num a => (D a -> D b) -> a -> b
d1 f x = let (D _ d) = f (D x 1) in d

d2 :: Num a => (D (D a) -> D (D b)) -> a -> b
d2 f x = let (D (D _ _) (D _ d)) = f (D (D x 1) 1) in d
```

Hier ist eine [Zusammenfassung der Ableitungsregeln](#).

In [Bemerkung 2](#) kannst du eine allgemeinere Funktion zum Berechnen der Ableitung sehen.

An vielen Stellen in den bisherigen Selbsttests haben wir oft einen konkreten Typ (z.B. `Int`) genutzt, für den es bereits vorimplementierte Typklasseninstanzen gibt. Das ist meistens der Fall gewesen, wenn wir Gleichheit auf Werten oder eine Vergleichsoperation auf Werten brauchten. Schau dir die bisherigen Selbsttests gerne erneut an und überlege dir, wo du Typen verallgemeinern kannst.

## TEST 108

Welche Funktionen musst du implementieren, damit eine `Eq`-Instanz vollständig definiert ist?  
Welche Gesetze sollten die Funktionen einer `Eq`-Instanz erfüllen?

### TEST 109 🕒



Gegeben sei der Typ

```
data Tree a b c = Empty | Leaf a | Node (Tree a b c) Int c (Tree a b c).
```

Implementiere eine **Eq**-Instanz für diesen Typen. Die Gleichheit soll sich so verhalten, wie die die wir durch das Ableiten bekommen würden. Bevor du die Instanz implementierst, überlege dir:

- Wie viele Regeln brauchst du mindestens, um **(==)** zu definieren?
- Benötigst du für die Implementierung Typeinschränkungen? Wenn ja, für welche Typen?
- An welchen Stellen wirst du **(==)** rekursiv anwenden?
- Die Datenkonstruktoren sind auf den rechten Seiten der Regeln nicht relevant. Auf welchen Typen kannst du die Gleichheit für z.B. **Node** zurückführen, bzw. wenn du dir die rechte Seite der Regel für **Node** anschaust, welche Typen fallen dir ein, für die diese rechte Seite auch eine Gleichheit definieren würde?

### TEST 110

Welche Funktionen musst du implementieren, damit eine **Ord**-Instanz vollständig definiert ist? Welche Gesetze sollten die Funktionen einer **Ord**-Instanz erfüllen?

### TEST 111 🕒



Gegeben sei der Typ

```
data Tree a b c = Empty | Leaf a | Node (Tree a b c) Int c (Tree a b c).
```

Implementiere eine **Ord**-Instanz für diesen Typen. Die Ordnung soll sich so verhalten, wie die die wir durch das Ableiten bekommen würden. Bevor du die Instanz implementierst, überlege dir:

- Spielt die Reihenfolge, in der wir die Datenkonstruktoren definieren eine Rolle für die Ordnung? Wenn ja, wie?
- Wie viele Regeln brauchst du mindestens, um **compare** zu definieren? Wie auch bei der Typklasse **Eq** können wir eine Regel definieren, die alle Fälle abdeckt, in denen wir **GT** erhalten, wenn wir uns nur die Datenkonstruktoren anschauen. Welches Schema müssen wir für die anderen Regeln verwenden, damit das funktioniert?
- Benötigst du für die Implementierung Typeinschränkungen? Wenn ja, für welche Typen?
- An welchen Stellen wirst du **compare** rekursiv anwenden?
- Die Datenkonstruktoren sind auf den rechten Seiten der Regeln nicht relevant. Auf welchen Typen kannst du die Ordnung für z.B. **Node** zurückführen, bzw. wenn du dir die rechte Seite der Regel für **Node** anschaust, welche Typen fallen dir ein, für die diese rechte Seite auch eine Ordnung definieren würde?

Implementiere die Ordnung auch erneut mit **(≤)**.

### TEST 112 ☹

Die Typklasse `Ord` ist wie folgt definiert:

```
class Eq a => Ord a where
  compare      :: a -> a -> Ordering
  (<), (<=), (>), (>=) :: a -> a -> Bool
  max, min     :: a -> a -> a

  -- default definitions
  -- ...

{-# MINIMAL compare | (<=) #-}
```

Das `{-# MINIMAL compare | (<=) #-}` bedeutet, dass es genügt, entweder `compare` oder `(<=)` zu implementieren.

- Gebe Standarddefinitionen für die Funktionen der Typklasse an.
- Was ermöglicht es, eine Standarddefinition für `compare` angeben zu können?
- Deine Standarddefinition von `compare` ist voraussichtlich ineffizient – die Vordefinierte ist es auch. Woran liegt das? Welche der beiden Funktionen würdest du implementieren, wenn du nur eine implementieren dürftest und es um die beste Laufzeit ginge?

Standarddefinitionen sind für den Anfang hilfreich, um mit minimalem Aufwand alle Funktionen einer Typklasse verwenden zu können. Häufig findet man aber konkrete Implementierungen für mehr als nur die notwendigen Funktionen, da diese eine bessere Laufzeit haben.

### TEST 113 ☹

In nicht streng getypten Programmiersprachen haben wir es oft mit impliziter Typkonversion zu tun. Implementiere eine Funktion `ifThenElse`, die als Bedingung Werte beliebiger Typen entgegennehmen kann. Ziel ist es, dass der folgende Ausdruck ausgewertet werden kann.

```
let a = ifThenElse 0 3 4
    b = ifThenElse [5] 6 7
    c = ifThenElse Nothing 8 9
in a + b + c -- 19
```

Theoretisch könnten wir über eine Spracherweiterung des GHC sogar die Standardimplementierung von bedingten Ausdrücken ersetzen. Das wollen wir aber ganz schnell wieder vergessen, genauso wie den Inhalt dieses Tests, nachdem wir ihn bearbeitet haben.

### TEST 114 ☹

Eine Halbgruppe ist eine Struktur  $(H, *)$ , wobei  $H$  eine Menge ist und  $*$  eine assoziative, binäre Verknüpfung  $* : H \times H \rightarrow H$  ist. Ein Monoid erweitert die Halbgruppe um ein neutrales Element bzgl.  $*$ .

Definiere Typklassen `Semigroup` und `Monoid`, die diese Strukturen implementieren. Gebe auch beispielhaft ein paar Instanzen für diese an.

Diese Typklassen sind vorimplementiert in Haskell. Sie befinden sich in den Modulen `Data.Semigroup` und `Data.Monoid`.

#### TEST 115

Wo findest du das Konzept der Typklassen in Programmiersprachen wie z.B. Python oder Java wieder? Gibt es z.B. ein Pendant zur **Show**-Typklasse in diesen Programmiersprachen?

#### TEST 116

Mit welcher Typklasse bzw. mit welcher Funktion können wir durch Strings repräsentierte Werte parsen?

#### TEST 117

Gebe für die folgenden Funktionsdefinitionen den allgemeinsten Typ an.

- `f x [] = [x]`  
  `f x (y:ys)`  
    `| x < y = x : y : ys`  
    `| otherwise = y : f x ys`
- `f mmx = mmx >= id`
- `f = (.) . (.)`
- `f = uncurry const`

#### Selbstevaluation

Ich bin in der Lage, ...

- Typklasseninstanzen, **Show**, **Eq** und **Ord**, für Datentypen anzugeben,
- ad-hoc Polymorphismus zu erklären und er sich von parametrischen Polymorphismus unterscheidet.

## Lazy Evaluation

#### Referenzen

- Skript: Funktionale Programmierung, Lazy Evaluation

#### TEST 118

Was ist Lazy Evaluation?

#### TEST 119

Wie werden Berechnungen in Haskell angestoßen? Wie viel wird berechnet?

#### TEST 120

Gebe ein Beispiel an, das zeigt, dass die faule Auswertung berechnungsstärker ist.

#### TEST 121

Welche praktischen Vorteile ergeben sich aus der Lazy Evaluation?

#### TEST 122

Wie werden mehrfache Berechnungen in einer nicht-strikten Auswertungsstrategie vermieden?

#### TEST 123

Gegeben sei folgender Haskell-Ausdruck.

```
let c = x == 0
    a = u `div` x
    b = 0
in if c then b else a
```

Der Teilausdruck `a = u `div` x` erscheint auf dem ersten Blick problematisch, da `x` Null sein könnte. Wieso stellt das mit Lazy Evaluation kein Problem dar?

#### TEST 124

Wieso ist `length [loop, loop, loop]` berechenbar? `loop` ist definiert durch

```
loop :: a → a
loop = loop
```

#### TEST 125 ☹

Eine zyklische einfach-verkettete Liste können wir in Python z.B. so definieren.

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

one, two = Node(1), Node(2)
one.next, two.next = two, one
```

Wenn wir mit `one` starten, dann korrespondiert diese verkettete Liste mit der unendlichen Liste `[1, 2, 1, 2, ...]`.

Die Mutierbarkeit des `next`-Zeigers macht das Verlinken der Knoten in Python möglich. Wie können wir in Haskell, trotz der Abwesenheit von Mutierbarkeit, zyklische Datenstrukturen umsetzen? Versuche, dein Programm ähnlich zum Python-Programm aussehen zu lassen. Verwende dafür auch den (polymorphen) Datentyp `Node`.

---

Die Technik ist als Tying the Knot bekannt.

### CHALLENGE 11 ⌚

Gegeben sei der Datentyp

```
data Tree a = Empty | Node (Tree a) a (Tree a).
```

- Implementiere eine Funktion `preorder :: Tree a → [a]`, die Knotenwerte in pre-order zurückgibt. Das heißt, zuerst wird ein Knoten betrachtet und anschließend dessen linker und danach dessen rechter Teilbaum.
- Implementiere einen unendlichen Baum `tree :: Tree Int`, der die Menge

$$\{f(i, j) \mid i, j \in \mathbb{N}, i \leq j\} \text{ mit } f(i, j) = i + j + 2ij$$

darstellt. Die Wurzel soll den Wert  $f(1, 1)$  haben. Für einen beliebigen Knoten mit Beschriftung  $f(i, j)$  soll die Wurzel des linken Teilbaums mit  $f(i + 1, j)$  beschriftet sein und die Wurzel des rechten Teilbaums mit  $f(i, j + 1)$ . Falls  $i > j$  erreicht wird, soll in den Baum ein `Empty`-Knoten gesetzt werden.

- Wende `preorder` auf `tree` an. Welches Problem haben wir hinsichtlich der Werte, die wir den jeweiligen Teilbäumen von `tree` sehen und der Ergebnisliste von `preorder`? Wie hängt deine Beobachtung mit `[f i j | i <- [1..], j <- [i..]]` zusammen?
- Implementiere eine Abwandlung von `preorder`, die statt `(++)` die Funktion `merge` (aus z.B. `mergesort`) verwendet.
- Implementiere als Nächstes die Mengendifferenz als Funktion `diff :: Ord a ⇒ [a] → [a] → [a]`. Du darfst dabei annehmen, dass die Eingabelisten bereits sortiert sind.
- Was berechnet `2 : map (\x → 2 * x + 1) ([1..] `diff` preorder tree)`?

Das Verfahren ist als Sieb von Sundaram bekannt. Die Konstruktion der oben angegebenen Menge mithilfe von unendlichen Bäumen ist nicht Teil des Verfahrens.

### TEST 126 ⌚

Gegeben sei der Datentyp

```
data Doubly a = Null | Node (Doubly a) a (Doubly a).
```

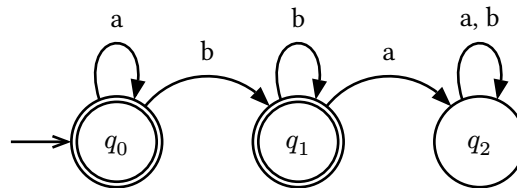
- Implementiere eine Funktion `fromList :: [a] → Doubly a`, die die gegebene Liste in eine doppelt-verkettete Liste umwandelt. `Null` soll sowohl das linke als auch das rechte Ende der Liste darstellen. Von diesem muss es nicht möglich sein, zum anderseitig verketteten Element zurückzukommen. `fromList` soll den Knoten zurückgeben, der mit dem ersten Listenelement korrespondiert.
- Weiter implementiere auch `prev :: Doubly a → Doubly a`, `value :: Doubly a → Maybe a` und `next :: Doubly a → Doubly a`, die den vorherigen Knoten, die Beschriftung eines Knoten, und den nächsten Knoten zurückgeben sollen.
- Angenommen du möchtest einen weiteren Wert in die doppelt-verkettete Liste einfügen, auf welches Problem stoßt du hinsichtlich Mutierbarkeit?

Der Wert des folgenden Ausdrucks soll `8` sein.

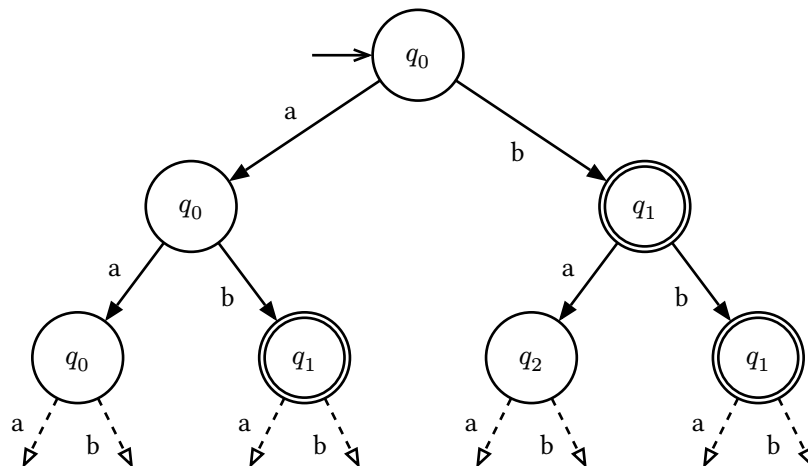
```
let xs = fromList [1, 6, 1, 8, 0, 3]
in value . prev . next . next . next . next $ xs
```

## CHALLENGE 12

Wir können endliche Automaten als unendliche Bäume darstellen. Betrachte z.B. den endlichen Automaten für die reguläre Sprache  $a^*b^*$ .



Diesen können wir als unendlichen Baum wie folgt darstellen.



- Konstruiere diesen Baum als `asbs :: State Char` mithilfe des Typs

```
data State a = State Bool [(a, State a)].
```

Der Boolean gibt an, ob der Zustand akzeptierend ist, und `[(a, State a)]` gibt die ausgehenden Transitionen an.

- Implementiere eine Funktion `accept :: Eq a => [a] -> State a -> Bool`, die bestimmt, ob eine Eingabe akzeptiert wird.
- Implementiere eine Funktion `language :: State a -> [[a]]`, die die akzeptierte Sprache des Automaten zurückgibt. (Du kannst davon ausgehen, dass die Sprache nicht leer ist – wenn du Entscheidungsproblem trotzdem lösen möchtest, halten wir dich nicht auf.)
- Warum funktioniert die folgende Implementierung der Funktion `language` nicht?

```
language :: State a -> [[a]]
language (State False ts) = [c:ws | (c, q) <- ts, ws <- language q]
language (State True ts) = [] : [c:ws | (c, q) <- ts, ws <- language q]
```

## TEST 127

Wieso können wir mit `foldl` auf unendlichen Listen mit keinem Ergebnis rechnen?

### TEST 128

`scanl` ::  $(b \rightarrow a \rightarrow b) \rightarrow b \rightarrow [a] \rightarrow [b]$  und `scanr` ::  $(a \rightarrow b \rightarrow b) \rightarrow b \rightarrow [a] \rightarrow [b]$  sind ähnlich zu `foldl` und `foldr`. Beide Funktionen speichern die Zwischenergebnisse der jeweiligen Funktion in Listen.

```
scanl :: (b -> a -> b) -> b -> [a] -> [b]
scanl _ e []      = [e]
scanl f e (x:xs) = e : scanl f (f e x) xs
```

```
-- scanl (+) 0 [1..4] = [0, 1, 3, 6, 10]
```

```
scanr :: (a -> b -> b) -> b -> [a] -> [b]
scanr _ e []      = [e]
scanr f e (x:xs) = f x y : ys
  where ys@(y:_) = scanr f e xs
```

```
-- scanr (+) (0) [1..4] = [10, 9, 7, 4, 0]
```

- Welche der beiden Funktionen kann auf unendlichen Listen arbeiten?
- (Implementiere die Fibonacci-Folge `fibs` :: `[Integer]` mithilfe von einer der beiden Funktionen.)

### CHALLENGE 13 ⌚

Fixpunktverfahren sind iterative Methoden, bei denen eine Funktion wiederholt auf einen Wert angewendet wird, bis sich ein stabiler Punkt (ein sogenannter Fixpunkt) ergibt, der sich durch weitere Anwendungen der Funktion nicht mehr verändert.

Dieses Berechnungsmuster wird durch die Funktion `iterate :: (a → a) → a → [a]` in der Prelude festgehalten. Sie berechnet eine unendliche Liste, bestehend aus den Ergebnissen der wiederholten Anwendungen der übergebenen Funktion auf den gegebenen Wert. Das erste Ergebnis ist der gegebene Wert, auf den die Funktion noch nicht angewendet wurde.

- Implementiere `iterate`.
- Ein klassisches Beispiel für ein Fixpunktverfahren aus der Numerik ist die Berechnung der Wurzel mithilfe des Heron-Verfahrens. Es ist gegeben durch

$$x_0 = a, \quad x_{n+1} = \frac{1}{2} \left( x_n + \frac{a}{x_n} \right)$$

Diese Folge nähert den Wert von  $\sqrt{a}$  mit jedem Folgenglied besser an. Implementiere das Verfahren mithilfe von `iterate`. Wähle die erste Näherung  $x_{n+1}$ , die  $|x_{n+1} - x_n| < \varepsilon$  erfüllt, für ein gegebenes  $\varepsilon > 0$ .

- Implementiere die Fibonacci-Folge als unendliche Liste `fibs :: [Integer]` mithilfe von `iterate`. Du brauchst eine Hilfsfunktion, die die Elemente der Ergebnisliste von `iterate` projiziert.
- Solange eine Liste Inversionen enthält, d.h., es existieren  $i, j$  mit  $i < j$ , sodass  $a_i > a_j$  gilt, gilt eine Liste als unsortiert. Das schrittweise Entfernen solcher Fehlstellungen führt zu einer sortierten Liste.
  - ▶ Implementiere eine Funktion `resolve :: Ord a ⇒ [a] → [a]`, die eine Fehlstellung findet und sie auflöst, indem sie die Elemente an den entsprechenden Positionen tauscht.
  - ▶ Implementiere das daraus resultierende Sortiervorgehen mithilfe von `iterate`.
- Mithilfe des Differenzenquotienten kannst du die erste Ableitung approximieren (oder mit der Lösung von [Challenge 2](#) oder [Challenge 10](#)). Diese benötigt man unter anderem für das [Gradientenverfahren](#). Eine vereinfachte Iterationsvorschrift des Verfahrens ist gegeben durch

$$x_{k+1} = x_k - 3 \cdot 10^{-4} \cdot f'(x_k) \quad \text{für alle } k \in \mathbb{N}$$

und einem Startpunkt  $x_0 \in \mathbb{R}$ . Implementiere das Verfahren mithilfe von `iterate`. Wähle als Ergebnis die erste Näherung  $x_{n+1}$ , die  $|x_{n+1} - x_n| < \varepsilon$  für ein festes  $\varepsilon > 0$  erfüllt (z. B.  $\varepsilon = 10^{-5}$ ).

### CHALLENGE 14

Deep Dive

Eine Editierdistanz zwischen zwei Wörtern  $u \in \Sigma^m, v \in \Sigma^n$  können wir mithilfe der folgenden Rekurrenz bestimmen:

$$\text{ed}(i, j) = \begin{cases} 0 & \text{falls } (i, j) = (0, 0), \\ i & \text{falls } j = 0, \\ j & \text{falls } i = 0, \\ \text{ed}(i-1, j-1) & \text{falls } u_{i-1} = v_{j-1}, \\ \min(\text{ed}(i-1, j-1), \text{ed}(i, j-1), \text{ed}(i-1, j)) + 1 & \text{sonst} \end{cases}$$

für alle  $0 \leq i \leq m, 0 \leq j \leq n$ . Um  $\text{ed}(m, n)$  effizient auszurechnen, nutzt man dynamische Programmierung. Das heißt, wir merken uns die Zwischenergebnisse und nutzen diese, wenn wir sie erneut brauchen, anstatt sie neu zu berechnen und ohne die Optimalität des Ergebnisses zu gefährden.

In Haskell können wir memoization mithilfe von Lazy Evaluation umsetzen. Hier ist ein unvollständiges Haskell-Programm, dass die Editierdistanz berechnen soll.

```
editdist :: Eq a => [a] -> [a] -> Int
editdist u v = table !! m !! n
  where
    (m, n) = (length u, length v)
    table = [[ed i j | j <- [0..n]] | i <- [0..m]]
```

Die Berechnungen sind in `table` gespeichert.

- Definiere die Funktion `ed :: Int -> Int -> Int` lokal in `editdist`. `ed` soll hier die zwischengespeicherten Ergebnisse aus `table` verwenden und dadurch die Berechnungen anstoßen.
- Überlege dir wie hier laziness und memoization zusammenspielen.
- Welche worst-case Laufzeit hat deine Lösung, wenn du annimmst, dass die Laufzeit von `(!!)` konstant ist?

### CHALLENGE 15

Deep Dive

Hinweis 1

Bevor du dich dieser Challenge stellst, bietet es sich an, sich [Challenge 14](#) anzunehmen, da in dieser der technische Teil der Lösungsidee vorgestellt wird.

Gegeben sei ein Gitter  $G \in \mathbb{Z}^{m \times n}$ . Ein Pfad durch das Gitter startet oben links und endet unten rechts. In jedem Schritt kannst du von einer Zelle in die rechts- oder darunteranschließende Zelle gehen. Die Pfadsumme ist die Summe aller Zellenwerte, durch die der Pfad führt.

Hier ist ein Beispiel für ein solches Gitter. Der Pfad der minimalen Pfadsumme ist durch die Pfeile angedeutet. Für dieses Beispiel ist die minimale Pfadsumme 6.

|    |     |     |     |
|----|-----|-----|-----|
| 1  | 2   | 3   | 4   |
| ↓  |     |     |     |
| -8 | → 4 | 6   | 1   |
|    | ↓   |     |     |
| 5  | 2   | → 3 | → 4 |

Implementiere eine Funktion `pathsum :: (Num a, Ord a) => [[a]] -> a`, die die minimale Pfadsumme berechnet.

### Selbstevaluation

Ich bin in der Lage, ...

- lazy evaluation in Haskell zu erklären,
- Funktionen auf unendliche Datenstrukturen zu definieren, und
- unendliche Datenstrukturen mithilfe von Funktionen zu erzeugen.

## Sequenzen

### Referenzen

- Skript: Funktionale Programmierung, Sequenzen
- Starting Out – Learn You a Haskell for Great Good! („Texas ranges“)

#### TEST 129

Ordne jeder der zur Typklasse `Enum` gehörenden Funktionen eine äquivalente `range` zu, also Ausdrücke der Form `[a, b..c]`.

- `enumFrom :: Enum a => a -> [a]`
- `enumFromThen :: Enum a => a -> a -> [a]`
- `enumFromTo :: Enum a => a -> a -> [a]`
- `enumFrom :: Enum a => a -> a -> a -> [a]`

#### TEST 130

Um eine `Enum`-Instanz anzugeben, benötigen wir mindestens Definitionen für `fromEnum :: a -> Int` und `toEnum :: Int -> a`. Wie können wir aus denen, Definitionen für die übrigen Funktionen der Typklasse gewinnen?

#### TEST 131

Gegeben sei der Datentyp

```
data Direction = North | East | South | West.
```

- Implementiere eine `Enum`-Instanz für `Direction`, die sich wie die vom GHC abgeleitete Instanz verhält.
- Implementiere eine `Bounded`-Instanz für `Direction`, die sich wie die vom GHC abgeleitete Instanz verhält.
- Implementiere die Funktionen `turnLeft :: Direction -> Direction` und `turnRight :: Direction -> Direction`, die die Himmelsrichtungen entsprechend ihrer Bezeichnung durchgehen.
- Implementiere eine Funktion `allDirections :: [Direction]`, die alle Himmelsrichtungen auflistet. Nutze dafür Funktionen, die dir durch die vorherigen Typklassen bereitgestellt werden.

#### TEST 132

Implementiere Funktion `cycleFrom :: (Enum a, Bounded a) => a -> [a]`, die ab einem gegebenen Wert alle Werte des Typen nicht absteigend durchläuft. Wenn der größte Wert erreicht ist, soll die Liste wieder beim kleinsten Wert des Typen beginnen.

### Selbstevaluation

Ich bin in der Lage, ...

- zu erklären, wie die Typklassen **Enum** und **Bounded** und list comprehensions zusammenhängen.

## List Comprehensions

### Referenzen

- Skript: Funktionale Programmierung, List Comprehensions
- [Starting Out – Learn You a Haskell for Great Good!](#) („I’m a list comprehension“)

#### TEST 133

An welches mathematische Konzept sind list comprehensions angelehnt?

#### TEST 134

Aus welchen Teilen besteht eine list comprehension?

#### TEST 135

Implementiere die Funktionen `map`, `filter` und `concatMap` mithilfe von list comprehensions (sowohl in Haskell als auch Python).

#### TEST 136

Übersetze die gegebenen Funktionen in eine äquivalente Funktion, die keine list comprehensions verwendet.

- `f xs = [x * 2 | x <- xs, x > 0]`
- `f xs ys = [x + y | x <- xs, y <- ys]`
- `f xs ys = [(x, y) | x <- xs, y <- ys, x < y]`
- `f xss = [x | xs <- xss, length xs > 2, x <- xs, even x]`
- `f xs = [(x, y) | x <- xs, y <- [1..x], even (x + y)]`
- `f xs = [(x, y) | x <- xs, let y = x * x, y `mod` 3 == 0]`

Versuche diese Aufgabe mit und ohne Listenmonade zu lösen.

### Selbstevaluation

Ich bin in der Lage, ...

- List Comprehensions konzeptionell zu verstehen und korrekt zu anwenden, einschließlich ihres mathematischen Hintergrunds, und
- Listenverarbeitungen mit List Comprehensions auszudrücken und äquivalent umzuformen, auch bei komplexeren Konstruktionen.

## Ein- und Ausgabe

### Referenzen

- Skript: Funktionale Programmierung, Ein- und Ausgabe
- [Input and Output – Learn You a Haskell for Great Good!](#)

#### TEST 137

Was ist referenzielle Transparenz?

#### TEST 138

Welche Rolle spielt der Typ `IO a` bzgl. Seiteneffekte? Was beschreibt ein Wert vom Typ `IO a`?

#### TEST 139

Wie können wir zwei `IO`-Aktionen zu einer neuen `IO`-Aktion kombinieren?

#### TEST 140

Level Up

Betrachte die `IO`-Aktion `act1 >> act2`.

- Welche der beiden Aktionen wird zuerst ausgeführt?
- Warum erscheint das bei Lazy Evaluation kontraintuitiv?
- Welche Rolle spielt der Typ `RealWorld -> (RealWorld, a)` bei der Sequenzierung?
- Wieso können `IO`-Berechnung in Haskell als „pure“ betrachtet werden?

#### TEST 141

Mit `getLine :: IO String` können Zeilen aus der Standardeingabe gelesen werden. Oft wollen wir den Wert haben, der durch die eingebene Zeichenkette repräsentiert wird. Wie können wir diesen Wert erhalten?

Implementiere eine Funktion `readInt :: IO Int`, die genau dies tut.

#### TEST 142

Implementiere ein Haskell-Programm, dass eine Eingabe wörterweise liest. Dabei soll gezählt werden, wie viele abwechselnde Vorkommen es der Wörter „Liebe“ und „Harry“ als Teilfolge in der Eingabe gibt. Jedes der Wörter soll einzeln gezählt werden. Zum Beispiel soll das Programm für folgende Eingabe

Liebe Harry Liebe Liebe Voldemort

3 ausgeben.

#### TEST 143

Wir betrachten die Dyck-Sprache, die durch die folgende kontextfreie Grammatik erzeugt wird:

$$S \rightarrow SS, \quad S \rightarrow (S), \quad S \rightarrow \varepsilon$$

Implementiere eine Funktion `dyck :: IO ()`, die die Standardeingabe liest und jedes Mal, wenn der aktuell gelesene Präfix der Standardeingabe ein Dyck-Wort ist, den String `"DYCK!"` ausgibt (auf der Standardausgabe). Du kannst davon ausgehen, dass die Eingabe immer ein Dyck-Wort ist.

Folgende Funktion testet, ob ein String ein Dyck-Wort der obigen Dyck-Sprache ist.

```
isDyck :: String -> Bool
isDyck = go 0
  where
    go _ "" = True
    go k '(' : s = go (k + 1) s
    go 0 ')' : _ = False
    go k ')' : s = go (k - 1) s
```

Eine Abwandlung dieser Funktion kannst du für deine Implementierung von `dyck` verwenden.

#### TEST 144

Implementiere ein IO-Programm `dan :: IO ()`, dass wie folgt abläuft:

- Es werden solange Namen eingegeben, bis eine leere Zeile eingegeben wird.
- Danach beginnt ein unendlicher Dialog zwischen den Personen, deren Namen eingegeben wurden. Dabei soll jede Person im Sekunden-Takt nacheinander „Nein, ich bin der dreckige Dan!“ sagen. Beim ersten Mal soll „Ich bin der dreckige Dan!“ ausgegeben werden. Die Pausen kannst du mithilfe der Funktion `threadDelay :: Int -> IO ()` aus dem Modul `Control.Concurrent` umsetzen.

Ein Programmablauf könnte wie folgt aussehen.

```
ghci> dan
Bob
Patrick

Bob: Ich bin der dreckige Dan!
Patrick: Nein, ich bin der dreckige Dan!
Bob: Nein, ich bin der dreckige Dan!
...
```

#### TEST 145

Implementiere ein Programm, das Zahlen aus einer Datei aufsummiert, bzw. implementiere eine Funktion `sumFile :: FilePath -> IO Int`. In jeder Zeile einer Datei steht eine nicht-negative Zahl.

#### TEST 146 ☹

Implementiere folgendes Rate-Spiel als IO-Programm. Es soll eine Zahl erraten werden.

- Du bekommst ein Orakel vom Typ `a → Ordering`, das dir verrät, ob dein Rateversuch kleiner als, gleich oder größer als der Wert ist, den das Orakel festgelegt hat.
- In jeder Runde des Spiels ratest du eine Zahl.
- Wenn die Zahl kleiner als die unbekannte Zahl ist, dann soll der Hinweis „Die gesuchte Zahl ist kleiner.“ ausgegeben werden. Wenn die unbekannte Zahl größer ist, soll ebenso eine entsprechende Nachricht ausgegeben werden.
- Wenn die korrekte Zahl erraten wurde, bricht das Spiel ab.

Implementiere das Spiel als Funktion `game :: Read a ⇒ (a → Ordering) → IO ()`.

Du kannst das Spiel gerne ausschmücken und erweitern. Zum Beispiel kannst du die Anzahl der Rateversuche begrenzen oder eine weitere Zahl festlegen, die vorzeitig das Spiel beendet und das Orakel gewinnen lässt.

#### 📦 Selbstevaluation

Ich bin in der Lage, ...

- referenzielle Transparenz und Seiteneffekte konzeptionell zu erklären und die Rolle des Typs `IO a` für die Modellierung von Effekten in Haskell einzuordnen,
- IO-Aktionen korrekt zu kombinieren, zu sequenzieren und auszuwerten, auch im Kontext von Lazy Evaluation und der zugrunde liegenden Weltzustands-Semantik,
- einfache bis mittelkomplexe IO-Programme entwerfen und implementieren, die Ein-/Ausgabe, Dateizugriff, Verarbeitung von Eingaben und kontrollierte Interaktion enthalten.

## Funktoren, Applicatives und Monaden

#### 📖 Referenzen

- Skript: Funktionale Programmierung, Funktoren und Monaden
- [Functors, Applicative Functors and Monoids – Learn You a Haskell for Great Good!](#) (ohne „Monoids“)
- [A Fistful of Monads – Learn You a Haskell for Great Good!](#)
- [The Absolute Best Intro to Monads For Software Engineers – YouTube](#) (nutzt TypeScript, aber die Ideen bleiben die gleichen)

Wenn du Zeit übrig hast und es dich interessiert: Die Monoiden-Sektion ist interessant.

#### TEST 147

Was ist der Unterschied zwischen Typklassen und Typkonstruktorklassen? Gebe Beispiele für beide an.

#### TEST 148

Gegeben sei folgende Typkonstruktorklasse.

```
class TCC t where  
  f :: t a b → b
```

Für welche Datentypen können wir Instanzen von **TCC** angeben? Beantworte die Frage möglichst allgemein.

#### TEST 149

Wie lauten die **Functor**-Gesetze?

#### TEST 150

Die Funktor-Typkonstruktorklasse ist wie folgt definiert.

```
class Functor f where  
  fmap :: (a → b) → f a → f b
```

Welchen kind hat **f**?

#### TEST 151

Wie sind die **Functor**-Instanzen für **Identity**, **Maybe**, **Either** **e**, **[]** und **((→) r)** definiert?

#### TEST 152

Wie lauten die **Applicative**-Gesetze?

#### TEST 153

Wie sind die **Applicative**-Instanzen für **Identity**, **Maybe**, **Either** **e**, **[]** und **((→) r)** definiert?

#### TEST 154

Wie lauten die **Monad**-Gesetze?

#### TEST 155

Wie sind die **Monad**-Instanzen für **Identity**, **Maybe**, **Either** **e**, **[]** und **((→) r)** definiert?

#### TEST 156 ☹

Die **Applicative**-Typkonstruktorklasse erlaubt es uns, `fmap` auf Funktionen mit mehreren Argumenten zu verallgemeinern. Dadurch können wir

`(+) <$> Just 1 <*> Just 2` oder `Just (+) <*> Just 1 <*> Just 2`

schreiben. Die Operatoren `(<$>)` und `(<*>)` funktionieren dabei ähnlich wie `(<$)` – mit `(<$>)` muss die Funktion nicht explizit in den entsprechenden **Applicative** gehoben werden.

Ein Typ, der uns konzeptionell auf Ebene der applikativen Funktoren näher an die gewöhnliche Funktionsapplikation heranführt, ist

```
newtype Identity a = Identity { runIdentity :: a }.
```

Implementiere **Functor**-, **Applicative**- und **Monad**-Instanzen für **Identity**.

Wenn du die Instanzen definierst, solltest du feststellen, dass du im Wesentlichen nur den enthaltenden Wert aus der **Identity** holst, verarbeitest und anschließend wieder hereinpackst.

#### TEST 157

Gegeben sei der Datentyp

```
data NonEmpty a = a :| [a].
```

Gebe **Functor**-, **Applicative**- und **Monad**-Instanzen für diesen Typen an.

#### TEST 158

Um zu verifizieren, dass die **Functor**-Gesetze für z.B. den Typ **Maybe** `a` gelten, müssen wir

- das Identitätsgesetz `fmap id = id` und
- das Kompositionsgesetz `fmap f . fmap g = fmap (f . g)`

zeigen. Wie gehen wir konkret für den gegebenen Typ vor? Wie zeigen wir die Gleichheit von Funktionen? Wenn die Gesetze nun für den Listendatentypen `[a]` zeigen wollen, was ändert sich an deinem Vorgehen?

#### TEST 159

Warum gilt

```
(1 +) <$> Just 1 == Just (1 +) <*> Just 1 == pure (1 +) <*> Just 1?
```

Wie ergibt sich aus deinen Beobachtungen eine Definition für `fmap`? Wie können wir die Berechnung für beliebige applikative Funktoren verallgemeinern? Wie wird dann festgelegt, was während der Berechnung tatsächlich passiert?

### TEST 160

Monaden sind ausdrucksstärker als applikative Funktoren, und applikative Funktoren sind ausdrucksstärker als Funktoren.

- Implementiere `fmap`, `pure` und `(<*>)` mithilfe von `return` und `(>=)`.
- Implementiere `fmap` mithilfe von `pure` und `(<*>)`.

Die Erkenntnis dieses Tests kannst du nutzen, wenn es für dich einfacher ist eine Monaden-Instanz anzugeben, anstatt Funktoren- und applikative Funktor-Instanzen anzugeben oder um Zeit zu sparen.

### TEST 161

Betrachten wir die Typen von `fmap` und `(>=)`, dann sehen wir gewisse Ähnlichkeiten.

```
fmap :: (a → b) → f a → f b
(>=) :: m a → (a → m b) → m b
```

Wie können wir bereits an den Typen sehen, dass `(>=)` die mächtigere Funktionen der beiden ist? Beziehe in deine Überlegungen ein, dass

```
fmap f (Right x) = Left y
```

nie gelten kann. Kann `fmap`, die Struktur in Abhängigkeit des Wertes vom Typ `a` verändern?

### TEST 162

Gegeben sei diese fehlerhafte Definition einer sicheren Division:

```
safeDiv :: Int → Int → Maybe Int
safeDiv x y = Just (div x) <*> Just y
```

Wie können wir diese Implementierung reparieren, ohne `(>=)` zu verwenden und die rechte Seite soweit wie möglich zu erhalten? Wie siehst du an diesem Beispiel, dass Monaden ausdrucksstärker als applikative Funktoren sind?

### TEST 163

Das Sequenzieren von Berechnung haben wir bereits in einfacheren Form als Funktionskomposition kennengelernt. Wenn wir die Fortsetzung einer Berechnung im Kontext von Monaden betrachten, dann stellen wir fest, dass sich zwei Funktionen des Typs `Monad m ⇒ a → m b` nicht im Sinne des `(>=)`-Operators hintereinanderausführen lassen.

Finde ein geeignetes Beispiel das zeigt, dass dieses Szenario illustriert.

### TEST 164

Monaden bieten uns die Möglichkeit, Berechnungen als Folge von kleineren Berechnungen zu sequenzieren. Je nachdem welche Monade wir betrachten, beobachten wir verschiedene Effekte.

- Welche Monade drückt eine Berechnung aus, die kein oder genau ein Ergebnis liefern kann?
- Welche Monade drückt eine Berechnung aus, die kein Ergebnis oder beliebig viele Ergebnisse liefern kann?
- Welche Monade drückt eine Berechnung aus, die entweder fehlschlägt oder erfolgreich ist? Welche kann im Fall eines Fehlschlag zusätzliche Information hervorbringen?

### TEST 165

Wir nutzen Monaden zur Sequenzierung von Berechnungen. Identifiziere diese Sequenzierung in den folgenden **Monad**-Instanzen bzw. stelle fest, dass `m` in `m >>= k` zuerst berechnet wird, bevor die Berechnung mit `k` fortgesetzt wird.

```
instance Monad Maybe where
  return x = Just x

  Nothing >>= _ = Nothing
  Just x  >>= k = k x

instance Monad [] where
  return x = [x]

  []      >>= _ = []
  (x:xs) >>= k = k x ++ (xs >>= k)
```

### TEST 166

Die **Reader**-Monade ermöglicht es, eine gemeinsame Umgebung mit vielen Berechnungen zu teilen.

```
newtype Reader r a = Reader { runReader :: r -> a }

instance Monad (Reader r) where
  return x = Reader (\_ -> x)

  r >>= k = Reader (\s -> let x = runReader r s
                           y = runReader (k y) s
                           in y)
```

Woran kannst du erkennen, dass die Berechnung `r` vor dessen Weiterführung mit `k` und dem Ergebnis `r` stattfindet?

### TEST 167 ☹

[Level Up](#)[Hinweis 1](#)

Das Pendant zur **Reader**-Monade aus [Test 166](#) ist die **Writer**-Monade. Eine Spezialisierung der Monade soll hier die **ListWriter**-Monade sein. Diese Monade kann genutzt werden, um Zwischenergebnisse oder Logging-Informationen einer Berechnung zu speichern.

```
newtype ListWriter w a = ListWriter { runListWriter :: (a, [w]) }
```

Implementiere diese Monade. Mache dir insbesondere Gedanken darüber, wie du Berechnungen dieses Typens sequenzierst und wie du die Zwischenergebnisse von zwei Berechnungen kombinierst.

### TEST 168

Argumentiere anhand der Gesetze, die für eine **Functor**-Instanzen gelten sollen, dass die folgenden **Functor**-Instanzen keine gültigen Instanzen sind. Gebe auch Beispiele an, die zeigen, dass die Gesetze nicht erfüllt sind.

```
instance Functor [] where
  fmap f [] = []
  fmap f (x:xs) = f x : f x : xs

data Tree a = Empty | Leaf a | Tree a :+: Tree a

instance Functor Tree where
  fmap _ Empty      = Empty
  fmap f (Leaf _)   = Empty
  fmap f (l :+: r) = fmap f l :+: fmap f r
```

Das Verwenden von monadischen Funktionen kommt einem zum Anfang möglicherweise als erbitterter Kampf gegen das Typsystem vor. Diese Situationen ergeben sich bereits aus den scheinbar unschuldigsten Absichten. Oft fehlt dann die Erfahrung, um die geeignete Funktion auszuwählen bzw. sogar die Kenntnis darüber, dass es Funktionen gibt, die einem helfen könnten. Hier betrachten wir ein paar solcher Szenarien.

- Angenommen du sollst eine Funktion `putStrs :: [String] → IO ()` implementieren, die eine Liste von Strings zeilenweise ausgeben soll, dann hast du die Möglichkeit, naiv und zielführend die Funktion per Induktion zu definieren. Als Nächstes könntest du die Beobachtung machen, dass du die gleiche Funktion für jeden String anwenden sollst, was dich an `map` erinnern könnte. Du erhältst den folgenden typinkorrekten Code.

```
putStrs :: [String] → IO ()
putStrs ss = map putStrLn ss -- :: [IO ()]
```

An der Stelle wirfst du entweder den Ansatz über den Haufen oder überlegst dir, wie eine Funktion vom Typ `[IO ()] → IO ()` definiert sein könnte, um den obigen Code zu reparieren. Implementiere allgemeiner eine Funktion `sequence :: Monad m ⇒ [m a] → m [a]` bzw. eine Funktion `sequence_ :: Monad m ⇒ [m ()] → m ()`, um die obige Implementierung von `putStrs` zu reparieren.

- Implementiere, basierend auf der vorherigen Teilaufgabe, eine Funktion `mapM :: (a → m b) → [a] → m [b]`. (Ein entsprechendes `mapM_` kannst du auch implementieren.)
- Andere hilfreiche Funktionen sind
  - `replicateM :: Monad m ⇒ Int → m a → m [a]`, die eine monadische Aktion eine feste Anzahl von Malen ausführt und dann eine Liste der Ergebnisse ausgibt.
  - `zipWithM :: Monad m ⇒ (a → b → m c) → [a] → [b] → m [c]`, die `zipWith` auf Monaden verallgemeinert,
  - `join :: Monad m ⇒ m (m a) → m a`, die eine monadische Struktur flacher klopft,
  - und viele weitere!

Implementiere die konkret genannten Funktionen.

- Implementiere eine Funktion `Int → String → IO ()`, die einen String beliebig häufig ausgibt.
- Was ist `zipWithM safeDiv [1, 2, 3, 4] [0, 1, 0, 1]`, wobei `safeDiv` wie folgt definiert sei.

```
safeDiv :: Integral a ⇒ a → a → Maybe a
safeDiv _ 0 = Nothing
safeDiv p q = Just (p `div` q)
```

- Was sind
  - `join [[1, 2, 3], [4, 5], [6]]`,
  - `join (Just Nothing)` und
  - `join (Left (Right True))`?

## CHALLENGE 16 ⌚

Level Up

Hinweis 1

Gegeben sei der Datentyp

```
newtype ZipList a = ZipList { getZipList :: [a] }.
```

Das Ziel ist es, `ZipList` als  $n$ -stellige Generalisierung von `zipWith` zu verwenden:

```
f <$> ZipList xs1 <*> ... <*> ZipList xsN
```

- Implementiere eine `Functor`-Instanz für `ZipList`.
- Bevor du eine `Applicative`-Instanz für `ZipList` implementierst, überlege warum

```
pure :: a -> ZipList a
pure x = ZipList [x]
```

keine gültige Definition ist? Welche Gesetze wären verletzt und warum, würde man `pure` so definieren?

- Implementiere eine `Applicative`-Instanz für `ZipList`.
- Zeige, dass sowohl die `Functor`- als auch die `Applicative`-Instanz die üblichen geforderten Gesetze erfüllen.

Das Umwickeln eines Typen mit `newtype`, für den wir bereits Typklasseninstanzen haben, ist ein gängiger Trick, um alternative Instanzen für diese Typklassen bereitzustellen.

`ZipList` hat keine `Monad`-Instanz.

## TEST 170 ⌚

Level Up

`guard :: MonadZero m => Bool -> m ()` kann genutzt werden, um eine Berechnung bedingt fehlschlagen zu lassen. Zum Beispiel können wir mithilfe von `guard` eine sichere Division definieren.

```
safeDiv :: (Integral a, MonadZero m) => a -> a -> m a
safeDiv a b = guard (b /= 0) >> return (a `div` b)
```

- Implementiere die Funktion `guard :: Bool -> Maybe ()` mit `Maybe ()` als konkreten Ergebnistypen.
- Die Typklasse `MonadZero` wird gängigerweise wie folgt definiert.

```
class Monad m => MonadZero m where
  mzero :: m a
```

Zu ihr gehören die folgenden Gesetze.

```
mzero >=> f      = mzero
m      >=> mzero = mzero
```

Damit verhält sich `mzero` wie eine „monadische Null“ bzw. absorbierend bzgl. der Operationen `(>=>)` und `(>>)`. Betrachte den folgenden Ausdruck.

```
m1 >> ... >> mzero >> ... >> mn
```

Was ist das Ergebnis dieses Ausdrucks?

- Implementiere `guard` mit den Gedanken der vorherigen Teilaufgabe.
- Berechne `1 `safeDiv` 0 :: m Int` für `Maybe` und `[]`. Bevor das möglich ist, benötigst du entsprechende `MonadZero`-Instanzen.

`guard` ist auf Basis von `Alternative` bzw. `MonadPlus` implementiert. `MonadZero` ist nicht Teil der Standardbibliothek, aber es ist definiert als Teil von `MonadPlus`.

### TEST 171

In [Test 170](#) haben wir eine Typkonstruktorklasse definiert, die es uns erlaubt hat, einen Fehlschlag bzw. die Abwesenheit eines Ergebnisses auszudrücken. Diese können wir auch eine Abstraktionsebene früher einführen. Definiere eine weitere Typkonstruktorklasse `AlternativeZero` auf Ebene der applikativen Funktoren mit einer Funktion `empty :: f a`.

Folgende Ausdrücken sollen die gegebenen Werte haben:

- `guard (y /= 0) *> pure y = pure y` für `y /= 0` und
- `guard (y /= 0) *> pure y = empty` für `y == 0`.

`(>*) :: Applicative f => f a -> f b -> f b` ist ein Kombinator, der sich wie `(>>)` für applikative Funktoren verhält.

### TEST 172 ☹

Level Up

Als motivierendes Beispiel für Monaden hast du die Auswertung eines arithmetischen Ausdrucks, gegeben als Termstruktur, kennengelernt. Dort haben wir die `Maybe`-Monade verwendet, um fehlschlagende Berechnung aufzufangen. Der Typ für die arithmetischen Ausdrücke ist gegeben durch:

```
data Exp a = Num a
           | Exp a :+: Exp a
           | Exp a :-: Exp a
           | Exp a *: Exp a
           | Exp a :/: Exp a
```

- Implementiere `eval :: Exp Int -> Maybe Int` mit dem `Maybe`-Applicative. Warum ist die Division, ohne eine weitere Hilfsfunktion nicht möglich? Was müsste diese Hilfsfunktion tun, damit die Regel für die Division funktioniert?
- Implementiere `eval :: Exp Int -> Maybe Int` mit der `Maybe`-Monade. Benötigst du hier die Hilfsfunktion aus der vorherigen Teilaufgabe? Warum ja oder nein?
- Wie kannst du mit [Test 170](#) `eval` zu einer Funktion `eval :: MonadZero m => Exp Int -> m Int` verallgemeinern?
- `MonadZero` erlaubt es uns, einen Fehlschlag allgemein auszudrücken. Allerdings können wir anhand des Fehlschlags alleine nicht feststellen, warum es zum Fehlschlag kam. Verallgemeinere die Typkonstruktorklasse `MonadZero` zu einer Typkonstruktorklasse `MonadFail`, die es erlaubt, eine Beschreibung des Fehlschlags anzugeben. Sie soll dafür eine Funktion `fail :: String -> m a` definieren. Verallgemeinere `eval` erneut mit der neuen Typkonstruktorklasse. Gebe zusätzlich eine Instanz für den Typ `Either String a` – alternativ, implementiere eine `Functor`-, `Applicative`-, `Monad`- und `MonadFail`-Instanz für den Typen `data Result a = Failure String | Success a`.

Der applikativen Funktor wird durch die Verwendung von Hilfsfunktion nicht ausdrucksstärker! Das, was der applikative Funktor nicht leisten kann, wird in die Hilfsfunktion ausgelagert. Teile der Hilfsfunktion werden durch die Monade übernommen.

### TEST 173

Wie hängen die Listenmonade und list comprehensions zusammen?

- Schreibe den folgenden Ausdruck

```
do
  x ← [1..10]
  y ← [1..10]
  guard (x + y == 10)
  return (x, y)
```

mithilfe von list comprehensions. Der Wert des ausgerechneten Ausdrucks ist

```
[(0, 10), (1, 9), (2, 8), (3, 7), (4, 6), (5, 5),
 (6, 4), (7, 3), (8, 2), (9, 1), (10, 0)]
```

Anhand des Ergebnisses kannst du dir die Semantik von `guard` herleiten (oder schaust dir vorher [Test 170](#) an).

- Schreibe den folgenden Ausdruck

```
[f | n ← [0..], let f = fib n, f `mod` 2 == 0]
```

mithilfe der Listenmonade.

### TEST 174 ☹

Die `do`-Notation erlaubt es uns, statements der Form `p ← e` zu schreiben, wobei `p` ein beliebiges Muster sein kann. Zum Beispiel ist folgender Ausdruck valide und berechenbar.

```
do
  Just x ← [Just 1, Nothing, Just 2]
  return x
```

Der Wert dieses Ausdrucks ist `[1, 2]`.

- In welches Problem läufst du, wenn du diesen Ausdruck mithilfe von `(>=)` und `(>>)` ausdrücken möchtest?
- Wie kannst du das Problem beheben in diesem konkreten Fall beheben?
- Fällt dir möglicherweise ein allgemeine Übersetzungsvorschrift für statements dieser Form unter der Verwendung von `MonadFail` aus [Test 172](#) ein?

---

Der Haskell Report spezifiziert in der [Sektion über die `do`-Notation](#), wie diese übersetzt wird.

### TEST 175

Wie übersetzen wir

```
do x ← e1; e2    und    do e1; e2
```

in einen äquivalente Ausdrücke mithilfe von `(>=)` und `(>>)`?

Wie übersetzen wir diesen etwas größeren Ausdruck?

```
do
  x ← getInt
  y ← getInt
  print (x + y)
  return (x + y)
```

### TEST 176

Wie übersetzen wir die Ausdrücke

```
eval e1 >>= \x → eval e2
>>= \y → if y == 0
        then Nothing
        else return (x + y)
```

und

```
getLine >> read <$> getLine
>>= \x → case f x of
        Nothing → return 0
        Just _  → return x
```

in einen äquivalenten Ausdruck mithilfe der `do`-Notation?

### TEST 177

Gegeben sei folgendes Haskell-Programm.

```
main :: IO ()
main = do
  putStr "Hello"
  return ()
  putStrLn ", world!"
  return ()
```

Was ist die Ausgabe des Programms und warum? Wie steht das im Konflikt mit dem, was du aus imperativen Programmiersprachen kennst?

### TEST 178

Gegeben sei folgendes fehlerhafte Haskell-Programm.

```
main :: IO ()
main = do
  q ← read <$> getLine
  if q == 0
  then main
  else
    p ← read <$> getLine
    print (p `div` q)
```

Warum ist das Programm fehlerhaft? Um den Fehler zu identifizieren, klammere alle validen Haskell-Teilausdrücke. (Du kannst davon ausgehen, dass alle Eingaben valide sind und deshalb keine weitere Fehlerbehandlung der Eingaben stattfinden muss.)

### TEST 179 ☹

Gegeben sei der Datentyp `Tree a = Leaf a | Tree a :+: Tree a`.

- Implementiere eine Funktion `splits :: [a] → [[a], [a]]`, die alle nicht-leeren Aufteilungen der Eingabeliste berechnet.

Zum Beispiel soll `splits [1..4]` die Liste

`[[1], [2, 3, 4]], ([1, 2], [3, 4]), ([1, 2, 3], [4])`

ergeben.

- Implementiere eine Funktion `allTrees :: [a] → [Tree a]`, die alle Binärbäume generiert, deren Blätter von links nach rechts die Eingabeliste lesen. Versuche, `allTrees` mithilfe von list comprehensions oder der Listenmonade zu implementieren.

### TEST 180

Gegeben ist der Typ `data Deep a b = Deep [Maybe (Either a (Deep a b))]`. Die Faltungsfunktion sieht im Wesentlichen so aus.

```
foldDeep :: ([Maybe (Either a r)] → r) → Deep a b → r
foldDeep fdeep (Deep x) = fdeep (f x)
  where fold = foldDeep fdeep
```

Wie können wir `f :: [Maybe (Either a (Deep a b))] → [Maybe (Either a r)]` definieren?

### TEST 181

Gegeben seien die Datentypen

```
data T1 a = A | B [T1 a]
data T2 a = A | B (Maybe a)
```

Gebe Faltungsfunktionen an, die bis auf Umbenennung der Funktionsbezeichner und Datentypen in der Signatur identisch sind.

---

Könntest du sogar eine für den Typ `data TX f a = A | B (f (TX f a))` angeben?

Oft kommt es vor, dass Berechnungen zustandsabhängig verschiedene Ergebnisse liefern. Zum Beispiel merken wir uns in einer Tiefensuche durch einen Graph, welche Knoten bereits besucht wurden, damit die Tiefensuche sich nicht in einem Kreis verläuft. Allgemein können wir solche Berechnungen als Funktionen vom Typ  $s \rightarrow (a, s)$  auffassen. Sie bekommen ein Zustand vom Typ  $s$  und liefern ein Ergebnis vom Typ  $a$  und einen (möglicherweise) neuen Zustand (ebenfalls vom Typ  $s$ ).

Implementiere eine Funktion `sequence :: [s → (a, s)] → s → ([a], s)`, die Liste zustandsabhängiger Berechnungen nimmt und der Reihe nach ausführt. Es wird dabei ein erster Zustand übergeben, der die erste Berechnung anstößt. Das Ergebnis soll der letzte Zustand mit allen Ergebnissen der Berechnungen sein.

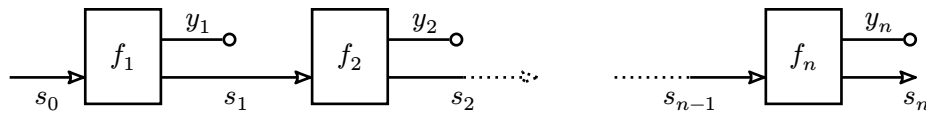
Betrachte folgendes Beispiel:

```
--      Zustand      Ergebnis  neuer Zustand
fib :: (Int, Int) → (Int      , (Int, Int) )
fib (f0, f1) = (f0, (f1, f0 + f1))

fibs :: Int → [Int]
fibs n = fst (sequence (replicate n fib) (0, 1))
```

Hier wird der erste Zustand mit den ersten beiden Fibonacci-Zahlen initialisiert, also  $(0, 1)$ . In jedem Schritt wird der Zustand mit der nächsten Fibonacci-Zahl aktualisiert und die kleinere Fibonacci-Zahl des Zustands zurückgegeben. Zuletzt projizieren wir mit `fst` das Ergebnis von `sequence` und verwerfen so den letzten Zustand.

Hier ist der Datenfluss von `sequence` nochmal visualisiert:



Es soll also `sequence [f1, f2, ..., fn] s0 = ([y1, y2, ..., yn], sn)` gelten.

Diese Implementierung von `sequence` ist ein Spezialfall für die [State-Monade](#). Mit der Intuition, dass wir hier Berechnungen sequenzieren, sollte es nicht überraschend sein, dass  $s \rightarrow (a, s)$  eine Monade ist. Alternativ kannst du die [State-Monade](#) implementieren und das vorimplementierte (oder von dir implementierte) Funktion `sequence` verwenden.

### CHALLENGE 17 ⌚

Im Folgenden modellieren wir einen Graph als Paar  $(V, E)$  mit einer Knoten- und Kantenbewertung  $w_v : V \rightarrow A$  und  $w_e : E \rightarrow B$ , und  $E \subseteq V \times V$ .

Gegeben sei der Datentyp

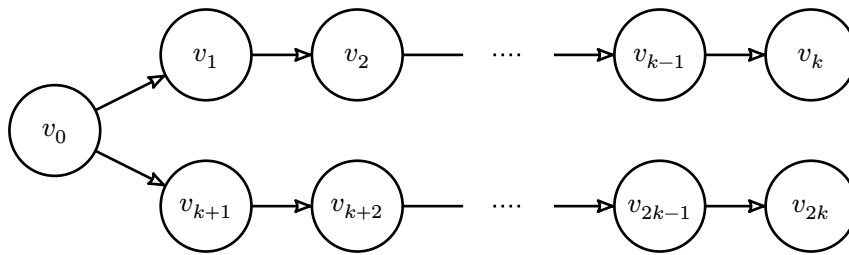
```
data Graph a b = Graph [(Int, a)] [(Int, b, Int)].
```

Hier entspricht der erste Parameter des Datenkonstruktors  $w_v$  und der zweite  $w_e$ .

- Implementiere eine Funktion `succs :: Int → Graph a b → [Int]`, die zu einem gegebenen Knoten die direkten Nachfolger berechnet.
- Implementiere eine Funktion `reachable :: Int → Int → Graph a b → Bool`, die entscheidet, ob zwischen zwei Knoten ein gerichteter Pfad existiert. Starte zuerst mit einfachen Graphen und betrachte immer komplexer werdende Graphen. Wie musst du (oder kannst du) deine Implementierung anpassen?
  - ▶ Nehme zuerst an, dass ein gegebener Graph immer ein Baum ist? Wenn also ein Pfad existiert, dann ist er eindeutig.
  - ▶ Nehme jetzt an, dass ein gegebener Graph immer azyklisch ist. Das heißt, falls ein Pfad existiert, muss dieser nicht mehr eindeutig sein. Nutze deine Erkenntnisse aus [Test 182](#), um eine Liste aller besuchten Knoten zu verwalten.
  - ▶ Was musst du in deiner Implementierung anpassen, wenn der gegebene Graph auch zyklisch sein kann?
- Je nachdem wie du `reachable` implementiert hast, könnte es denn Anschein erwecken, dass deine Lösung alle Knoten besucht. Wieso ist das nicht unbedingt der Fall? Wieso passiert das nur, wenn wir uns die Liste aller besuchten Knoten anschauen? Welche Beobachtungen machst du, wenn du `reachable 0 k (yGraph k)` berechnest?

```
yGraph :: Int → Graph () ()
yGraph k = Graph [(v, ()) | v <- [0..2 * k]]
              (let left = [(v, (), v + 1) | v <- [1..k]]
                 right = [(v, (), v + 1) | v <- [k + 1..2 * k]]
                 in [(0, (), 1) : (0, (), k + 1) : (left ++ right)])
```

Hier ist der Graph visualisiert.



Falls du es nicht geschafft hast, in [Test 182](#) `sequence` zu implementieren, kannst du diese Implementierung verwenden.

```
sequence :: [s → (a, s)] → s → ([a], s)
sequence [] s = ([], s)
sequence (f:fs) s = let (y, s') = f s
                      (ys, s'') = sequence fs s'
                      in (y:ys, s'')
```

## Selbstevaluation

Ich bin in der Lage, ...

- Funktoren zu definieren, die die Funktor-Gesetze erfüllen und das auch beweisen,
- applikative Funktoren zu definieren, die die Applicative-Gesetze erfüllen und das auch beweisen,
- Monaden zu definieren, die die Monaden-Gesetze erfüllen und das auch beweisen,
- Funktoren, Applicatives und Monaden zu nutzen,
- `do`-Notation zu nutzen, und
- zwischen `(>=>)`, `(>>)` und `do`-Notation zu übersetzen.

## Automatisches Testdatengenerierung

### Referenzen

- Skript: Funktionale Programmierung, Automatisches Testdatengenerierung
- [QuickCheck: a lightweight tool for random testing of Haskell programs](#)

### TEST 183

Es kommt häufiger vor, dass zufällig generierte Werte bestimmte Eigenschaften erfüllen sollen.

QuickCheck bietet auf Generatoren-Ebene z.B. die Funktion

`suchThat :: Gen a -> (a -> Bool) -> Gen a` dafür an. Diese nimmt einen Generator und generiert solange neue Werte, bis eine gewünschte Eigenschaft erfüllt ist. Diese Eigenschaft wird durch ein Prädikat formuliert.

Implementiere `suchThat`.

---

Es gibt ein paar QuickCheck-spezifische Fallstricke, die mit dem `size`-Parameter der Generatoren zu tun haben, die du bei deiner Implementierung nicht beachten brauchst. Falls es dich interessiert, ist hier die [suchThat-Implementierung von QuickCheck](#).

### TEST 184

Wenn du folgende (etwas künstliche) Eigenschaft mithilfe von QuickCheck prüfst, muss im Durchschnitt jeder zweite Test verworfen werden, weil die Vorbedingung der Eigenschaft nicht erfüllt ist.

```
prop_prop :: Int -> Property
prop_prop k = k > 0 ==> True
```

Um dieses Verhalten zu unterbinden, können wir die Eigenschaft leicht abändern.

```
prop_prop :: Positive Int -> Property
prop_prop (Positive k) = k > 0 ==> True
```

Jetzt werden keine Testeingaben verworfen.

Überlege dir, wie die Lösung des Problems funktioniert und implementiere sie.

---

QuickCheck hat viele solche [type-level modifiers](#), die das Generator-Verhalten verändern.

Die folgenden zwei Challenges steigen tiefer in die Funktionsweise von QuickCheck ein. Dabei werden zwar weiter viele Vereinfachungen gemacht, wir nähern uns dennoch der tatsächlichen Implementierung von QuickCheck stark an.

## CHALLENGE 18 ⌚

## Deep Dive

Eigenschaften lassen sich mit QuickCheck z.B. mithilfe der Funktion `quickCheck` prüfen.

Hier sind ein paar Eigenschaften.

```
prop_comm :: Int → Int → Bool
prop_comm x y = x + y == y + x
```

```
prop_inv :: Int → Bool
prop_inv x = x + (-x) = 0
```

- Welchen Typ muss `quickCheck` scheinbar haben, damit wir `prop_comm` prüfen können? Wie steht das im Konflikt mit `prop_inv`?
- Um `quickCheck` zu implementieren, benötigen wir die Möglichkeit, beliebig stellige Funktionen anwenden zu können. Das ist mithilfe von Typklassen möglich. Hier ist eine vereinfachte Variante der Typklasse `Testable`, wie sie in QuickCheck zu finden ist. Hier ist Typ `Property` ein einfacher Wrapper um den `Bool`-Typ. In QuickCheck ist dieser ein bisschen komplexer.

```
newtype Property = Property { unProperty :: Bool }
```

```
class Testable a where
  property :: a → Property
```

- ▶ Implementiere `Testable`-Instanzen für `Bool` und `Property`, und anschließend eine Instanz für den Typ `a → b`, wobei `a` eine `Arbitrary`-Instanz haben soll und `b` wieder `Testable` sein soll.
- ▶ Welche Instanzen übernehmen die Rolle eines Basisfalls und welche Instanzen übernehmen die Rolle einer induktiven Regel?
- ▶ Wie ist es möglich, dass wir Eigenschaften definieren können, die entweder ein `Bool` oder eine `Property` als Rückgabewert haben?
- ▶ Als Nächstes betrachten wir eine Vereinfachung der `Arbitrary`-Typklasse und eine Instanz für den Typ `Int`, damit wir `quickCheck` implementieren und verwenden können.

```
class Arbitrary a where
  arbitrary :: a
```

```
instance Arbitrary Int where
  arbitrary = 42 -- Hier werden normalerweise zufällige Werte generiert.
```

Implementiere eine Funktion `quickCheck :: Testable a ⇒ a → Bool`, die eine Eigenschaft nimmt und prüft.

In QuickCheck werden Generatoren genutzt, um zufällige Werte zu generieren. Diese nutzen alle einen Zufallszahlengenerator, der als Zustand mit weiteren Parametern durch alle `arbitrary`-Aufrufe durchgetragen wird. Das behandeln wir in der [Challenge 19](#), damit diese Challenge nicht zu sehr ausartet.

## CHALLENGE 19 ⌚

## Deep Dive

Bevor du diese Challenge bestreitest, sage [Challenge 18](#) den Kampf an!

Während QuickCheck `SplitMix64` als Pseudozufallszahlengenerator nutzt, nutzen wir ein einfacheres Verfahren, um solche Zahlen zu erzeugen. `Xorshift` erzeugt Pseudozufallszahlen nach dem folgenden Schema:

$$x \leftarrow x \oplus (x \ll 13)$$

$$x \leftarrow x \oplus (x \gg 7)$$

$$x \leftarrow x \oplus (x \ll 17)$$

wobei  $x$  eine 64-Bit Ganzzahl ist.

- Implementiere eine Funktion `xorshift64 :: Word64 → Word64`, die das obige Verfahren umsetzt. Folgende imports könnten hilfreich sein. Werte vom Typ `Word64` können direkt mit den Bit-Operationen verwendet werden. Wir nutzen den Typ `Word64`, weil für diesen im Gegensatz zum `Int` garantiert ist, dass dieser 64-Bit hat. Wir gehen danach aber wieder direkt zum Typ `Int`.

```
import Data.Bits (shiftL, shiftR, xor)
import Data.Word (Word64)
```

Eine zufällige Ganzzahl kann nun mithilfe der Funktion `nextInt` generiert werden. Ein Zufallszahlengenerator kann mit der Funktion `mkPRNG` erzeugt werden. Dieser muss mit einer Zahl (seed) initialisiert werden.

```
data PRNG = PRNG Word64
    deriving Show
```

```
mkPRNG :: Word64 → PRNG
mkPRNG s = PRNG (xorshift64 s)
```

```
nextInt :: PRNG → (Int, PRNG)
nextInt (PRNG s) = (fromIntegral r, PRNG r)
    where r = xorshift64 s
```

Damit der Zufallsgenerator bei der Definition von `arbitrary` zur Verfügung steht, passen wir die Definition der zugehörigen Typklasse an. Weiter erhält diese auch einen Größen-Parameter, um die generierten Zufallswerte in ihrer Größe zu kontrollieren. Andere Definitionen aus der vorherigen Challenge müssen wir auch anpassen.

```
newtype Property = Property { unProperty :: Gen Bool }
```

```
newtype Gen a = Gen { unGen :: PRNG → Int → (a, PRNG) }
```

```
class Arbitrary a where
    arbitrary :: Gen a
```

- Passe deine alte Implementierung aus [Challenge 18](#) bzgl. der neuen Typen an.
- Definiere `Functor`-, `Applicative`- und `Monad`-Instanzen für den Typkonstruktor `Gen`. (Hier passiert im Wesentlichen etwas Ähnliches wie in [Test 182](#).)
- Definiere eine Funktion `chooseInt :: (Int, Int) → Gen Int`, die eine zufällige Ganzzahl im übergebenen Intervall generiert. Um eine zufällige Zahl  $x \in \mathbb{Z}$  in ein Intervall  $[l, r)$  zu zwingen, können wir z.B.  $l + x \bmod (r - l)$  rechnen. Beachte, dass `mod` auch negativ sein kann.
- Definiere eine `Arbitrary`-Instanz für den Typ `Int`. Dabei soll eine generierte Ganzzahl im Intervall  $(-s, s)$  liegen, wobei  $s$  der übergebene Größen-Parameter der Generator-Funktion ist.
- Passe `quickCheck` so an, dass ein Zufallsgenerator und ein Größenwert an die Generator-Funktionen weitergegeben wird.

Die Vereinfachungen, die wir gemacht haben:

- einfacheren PRNG nutzen (anstatt `SplitMix64`)
- in `Gen` wird in `QuickCheck` der PRNG gesplittet, anstatt dass er durchgereicht wird
- `chooseInt`, so wie vorgeschlagen, erzeugt keine gleichverteilten Zufallswerte in dem Intervall. Für Genauerer:
  - [How to generate a random integer number from within a range – StackOverflow](#)
  - `nextIntInteger` aus `System.Random.SplitMax`

- wir haben einen festen seed gewählt (in SplitMix64 wird `initialSeed` basierend auf der aktuellen Uhrzeit gesetzt, damit immer neue zufällige Zahlen generiert werden, wenn das Programm erneut gestartet wird)
- und vieles mehr!

### Selbstevaluation

Ich bin in der Lage, ...

- einfache `Arbitrary`-Instanzen anzugeben.

---

Diese Aufgaben haben noch keinen Platz gefunden.

#### TEST 185

Implementiere Arrays als nicht-mutierende Datenstruktur in Python. Die Klasse soll so funktionieren, dass der folgende Beispielcode lauffähig ist und die angegebene Ausgabe erzeugt.

```
a = Array.fromList(5, [(2, 4), (3, 3)])
```

```
b = a.update([(1, 3), (2, 2)])  
for i in range(len(a)):  
    print(a[i], b[i])
```

Die Ausgabe des Programms soll

None None

None 3

4 2

3 3

None None

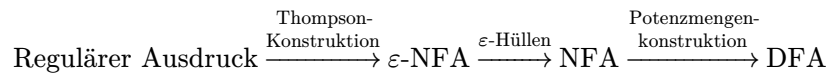
sein.

## CHALLENGE 20 🕒

## Deep Dive

Du hörst parallel zu diesem Modul „Berechnung und Logik“ und möchtest die Automaten-Konstruktionen für reguläre Sprachen in Haskell implementieren? Dann schau nicht weiter. Du hast die korrekte Challenge gefunden. Dieser Weg ist lang und das erste Mal etwas steinig, also schnappe dir genügend Proviant und bringe genügend Zeit mit.

In dieser Challenge wollen wir folgende Konstruktionen implementieren.



- Definiere einen Datentypen `RE`, um reguläre Ausdrücke darzustellen. Der reguläre Ausdruck  $(ab)^*|c$  könnte z.B. so dargestellt werden

```
kleene (Literal 'a' :* Literal 'b') :|: Literal 'c'.
```

Die leere Sprache und die Sprache, die nur das leere Wort enthält, sind über das Beispiel nicht abdeckt.

- Bevor wir mit den Konstruktionen beginnen, müssen wir uns überlegen, wie wir die Automaten darstellen wollen. Überlege dir Typen für nichtdeterministische endliche Automaten mit und ohne  $\varepsilon$ -Transitionen und deterministische endliche Automaten. Einen Zustand kannst du z.B. als Ganzzahl darstellen.
- Implementiere die Thompson-Konstruktion als Funktion `RE → EpsNFA`. Hier ist eine wesentliche Schwierigkeit, neue Zustände zu erzeugen, da die Bezeichnungen verschiedener Teilautomaten miteinander kollidieren können. Das Problem lässt sich z.B. mithilfe eines Zählers lösen, der durch die Konstruktion durchgeführt wird. Eine entsprechende Hilfsfunktion könnte folgenden Typ haben: `RE → Int → (EpsNFA, Int)`. Wenn du dich an die `State`-Monade wagen möchtest, könnte dies an dieser Stelle auch nützlich sein, um den Zähler mitzuführen – im Wesentlichen versteckst du damit das explizite Mitführen des Zählers.
- Implementiere eine Funktion `epsilonClosure :: EpsNFA → [Int] → [Int]`, die die  $\varepsilon$ -Hülle einer Menge von Zuständen berechnet.
- Implementiere eine Funktion `removeEpsilon :: EpsNFA → NFA`, die alle  $\varepsilon$ -Transitionen des übergebenen nichtdeterministischen endlichen Automaten entfernt. Hier brauchst du dir keine Gedanken über neue Bezeichner für Zustände machen, da du die Zustände des alten Automaten wiederverwenden kannst.
- Implementiere die Potenzmengenkonstruktion. Dafür bietet es sich an den NEA mit einer Tiefensuche zu durchlaufen, anstatt tabellarisch jede Teilmenge der Zustandsmenge entsprechend zu verbinden. So werden dann nur Zustände durchlaufen, die für den DEA wichtig sind. Hier ist eine wesentliche Schwierigkeit, die neuen Zustände zu benennen. Es bietet sich an, ein Wörterbuch der Form `[(Int, Int)]` zu verwalten, in dem die neuen Bezeichner nachgeschaut werden können.
- Zuletzt, um zu überprüfen, ob der DEA die korrekte Sprache erkennt, implementiere eine Funktion `member :: String → DFA → Bool`, die überprüft, ob ein Wort vom übergebenen Automaten erkannt wird.

Wenn du bis hierhin gekommen bist, könntest du weiter den DEA mithilfe von Hopcrofts Algorithmus minimieren.

## BEMERKUNG 1

In der Evaluation des Moduls im Wintersemester 2025/2026 hat sich eine Person gewünscht, dass das Prolog-Interpreter-Projekt erweitert wird, sodass auch der Parser selber gebaut werden muss.

Leider ist in diesem Modul zu wenig Zeit, um dieses Thema hinreichend zu behandeln. Das rabbit hole soll dir mit dieser Bemerkung allerdigns eröffnet werden.

- Der Parser aus dem Projekt basiert auf parsec, einer Bibliothek bestehend aus verschiedenen monadischen Parser-Kombinatoren. Die wesentlichen Ideen, die diese Bibliothek nutzt, findest du in dem paper Monadic parsing in Haskell beschrieben. Etwas weniger in die Tiefe geht einer der Co-Autoren in diesem Video Functional Parsing.
- Mit dieser Bibliothek lassen sich sogenannte recursive descent parser spezifizieren. Die Syntax der Kombinatoren ermöglicht es, dass entsprechende Grammatiken fast direkt aus der mathematischen Notation in ein Haskell-Programm überführt werden können.
- Als Nächstes benötigen wir eine Grammatik für die Syntax von Prolog. Nicht jede Grammatik ist dafür geeignet, insbesondere müssen bei rekursiven Abstiegsparsern darauf achten, dass die Grammatik nicht linksrekursiv ist.
- So können wir dann einen abstrakten Syntaxbaum berechnen, so wie du ihn aus dem Projekt kennst. Der Baum wird durch die vielen gegebenen Typen dargestellt.

Je nachdem wie dich das Thema interessiert, behandelt Frank z.B. in seiner Vorlesung „Funktionale Programmierung“ monadisches Parsen. Wenn du noch tiefer einsteigen willst, dann wird auch hin und wieder das Modul „Übersetzerbau“ angeboten. Für einen tieferen Einstieg wird oft folgende Literatur empfohlen.

- Compilers: Principles, Techniques, and Tools von Alfred V. Aho, Monica S. Lam, Ravi Sethi, und Jeffrey D. Ullman (auch dragon book genannt), und
- Engineering a Compiler von Keith D. Cooper und Linda Torczon (das Modul Übersetzerbau nutzt dieses Buch)

# Logische Programmierung

## Prolog, Prolog!

### Referenzen

- Skript: Einführung in die Logikprogrammierung, Motivation
- Skript: Einführung in die Logikprogrammierung, Syntax von Prolog

#### TEST 186

Wie unterscheiden sich Variablen in Haskell und Prolog?

#### TEST 187

Was ist das Berechnungsprinzip von Prolog bzw. wie leitet Prolog aus gegebenen Informationen neue Erkenntnisse ab?

#### TEST 188

Formuliere folgende Aussage als Prolog-Programm: Seien  $A$  und  $B$ . Dann gilt  $C$ , wenn  $A$  und  $B$  gelten.

#### TEST 189

Wie können wir Funktionen, wie wir sie aus Haskell kennen, in Prolog umsetzen? Erkläre es mithilfe eines Beispiels (wie z.B.  $(++) :: [a] \rightarrow [a] \rightarrow [a]$ ).

#### TEST 190

$n$ -stellige Funktionen können wir in Prolog als  $(n + 1)$ -stellige Relation umsetzen. Dabei nimmt die letzte Position der Relation die Rolle des Ergebnisses ein. Gehen wir mit Funktionen vom Typ  $a_1 \rightarrow \dots a_n \rightarrow \text{Bool}$  besonders um?

#### TEST 191

Wie hängen die Syntax von Prolog und die der Aussagenlogik zusammen? Welche Symbole entsprechen sich?

#### TEST 192

In Prolog können wir Termstrukturen erzeugen. Der Haskell-Typ

```
data Maybe a = Nothing | Just a
```

kann z.B. wie folgt in Prolog umgesetzt werden.

```
maybe(nothing).  
maybe(just(_)).
```

Übersetze mit der gleichen Idee den Datentyp

```
data Tree a = Empty | Node (Tree a) a (Tree a).
```

#### TEST 193



Wie viele Ergebnisse liefern die folgenden Anfragen?

- ?- `append(_, [X|_], [1, 2, 3, 4])`.
- ?- `append(_, [_|X|_], [1, 2, 3, 4])`.
- ?- `member(X, [1, 2, 3]), member(Y, [2, 3, 4]), X \= Y`.
- ?- `append(_, [X|Ys], [1, 2, 3, 4]), append(_, [Y|_], Ys)`.

#### TEST 194

Wie stehen die Begriffe Fakt, Regel, Klausel, Prädikat und Relation in Beziehung?

#### TEST 195

Gebe Beispiele für Formeln an, die zeigen, dass Prolog nur eine echte Teilmenge der Prädikatenlogik erster Stufe umsetzt.

#### TEST 196

Ordne die Begriffe Atom, Fakt, Regel und Variable dem folgenden Prolog-Programm zu?

```
semitone(c, cs). semitone(cs, d). semitone(d, ds). semitone(ds, e).  
semitone(e, f). semitone(f, fs). semitone(fs, g). semitone(g, gs).  
semitone(gs, a). semitone(a, as). semitone(as, b). semitone(b, c).
```

```
minor_third(P1, Mi3) :-  
    semitone(P1, Mi2), semitone(Mi2, Ma2), semitone(Ma2, Mi3).
```

```
major_third(P1, Ma3) :- minor_third(P1, Mi3), semitone(Mi3, Ma3).
```

```
chord(minor(P1, Mi3, P5)) :- minor_third(P1, Mi3), major_third(Mi3, P5).  
chord(major(P1, Ma3, P5)) :- major_third(P1, Ma3), minor_third(Ma3, P5).
```

#### TEST 197

In Test 196 sind die Halbtonschritte als Fakten definiert. Um diese etwas kompakter zu schreiben, können wir `append/3` verwenden. Implementiere `semitone/2` mithilfe von `append/3`.

### TEST 198

Warum ergibt es Sinn, beim Prolog-Programmieren in Relationen statt Funktionen zu denken? Betrachte z.B. das Prädikat `append/3` gemeinsam mit den Anfragen

- `?- append(Xs, Ys, [1, 2, 3]).`,
- `?- append(Xs, [2, 3], Zs).`,
- `?- append([1], [2, 3], Zs).`,
- `?- append([1, 2], Xs, Zs).` und
- `?- append([1, 2], Ys, [1, 2, 3]).`

Beschreibe, was jede dieser Anfragen berechnet. Welche der Anfragen entspricht der Anwendung von `append/3` als Funktion, so wie wir es bzgl. der Benennung des Prädikats erwarten würden?

### TEST 199

Wenn wir Haskell um die Möglichkeit erweitern könnten, mehrere Regeln auszuprobieren und mehrere Lösungen zu kombinieren, hätten wir trotz der Abwesenheit logischer Variablen bereits viele Möglichkeiten der Modellierung, wie wir sie in Prolog haben. Es stellt sich heraus, dass die Listenmonade den Nichtdeterminismus schon sehr gut abbilden kann, und erlaubt damit einen weiteren abstrakten Blick auf die Listenmonade - anstatt z.B. der Blick der imperativen Programmierung als Verschachtelung von Schleifen.

Als kleines Beispiel betrachten wir einen fairen Münzenwurf. Wir kodieren die Ereignisse binär, wobei 0 für Kopf und 1 für Zahl stehen soll. Weiter ist auch ein Beispiel angegeben für zwei unabhängige Münzenwürfe.

In Prolog:

```
coin(0).  
coin(1).  
  
coin2(X, Y) :-  
    coin(X),  
    coin(Y).
```

In Haskell:

```
coin :: [Int]  
coin = [0] ++ [1]  
  
coin2 :: [(Int, Int)]  
coin2 = do  
    x <- coin  
    y <- coin  
    return (x, y)
```

Bewaffnet mit diesen Ideen, modelliere einen fairen 6-seitigen Würfel. Berechne alle Möglichkeiten, wie man drei Würfel werfen kann, um die Augenzahl 11 zu erhalten. Gebe deine Lösung sowohl in Prolog als auch in Haskell an. Um zu prüfen, ob die Summe von drei Zahlen einer anderen Zahl entspricht, kannst du `sum3(X, Y, Z, S) :- S is X + Y + Z.` nutzen.

### TEST 200

Implementiere das Prädikat `zip/3`, dass zwei Liste bekommt und eine Liste von Paaren zurückliefert – so wie du es aus Haskell kennst. Es soll

```
?- zip([1, 2], [3, 4, 5], [(1, 3), (2, 4)]).
```

gelten. Wie gewinnst du aus deiner Implementierung das Prädikat `unzip/3`, also die Umkehrfunktion `zip/3`, wenn diese auf Listen gleicher Länge eingeschränkt ist.

### TEST 201 ☹

Mithilfe von `append/3` lassen sehr viele andere Prädikate auf Listen definieren. Überlege dir, wie du folgende Prädikate unter dessen Nutzung implementieren kannst. Die Prädikate sollen sich wie deren entsprechenden Haskell-Funktionen verhalten, wenn dir der Name des Prädikats bekannt vorkommt.

- `head/2` und `tail/2`,
- `last/2`,
- `member/2` (entspricht `elem` in Haskell)
- `dups/2` soll alle Elemente in einer Liste finden, die genau zweimal vorkommen. Du darfst `not_member/2` als Hilfsprädikat verwenden. Es ist definiert durch `not_member(X, L) :- \+ member(X, L)`. Als Zwischenschritt kannst du `dups/2` so implementieren, dass geprüft wird, ob ein Element mindestens zweimal vorkommt.
- `sublist/2` soll erfüllbar sein, wenn eine Liste in einer anderen ohne Lücken enthalten ist.

Warum ist `member/2` hier zweistellig?

### TEST 202

Das Prädikat `sublist/2` soll genau dann erfüllbar sein, wenn eine gegebene Liste in einer anderen gegebenen Liste enthalten ist. Eine mögliche Implementierung sieht wie folgt aus:

```
is_prefix([], _).  
is_prefix([X|Xs], [X|Ys]) :- is_prefix(Xs, Ys).  
  
sublist(Xs, Ys) :- is_prefix(Xs, Ys).  
sublist(Xs, [_|Ys]) :- sublist(Xs, Ys).
```

Was sind die Ergebnisse der Anfrage `?- sublist(Xs, [1, 2]).?`

### TEST 203

Warum ist `not_member(X, Xs) :- append(_, [Y|_], Xs), X \= Y` keine korrekte Implementierung des Prädikates, das testet, ob ein Element nicht einer Liste enthalten ist?

### TEST 204

In diesem Selbsttest wollen wir die Peano-Arithmetik wiederholen. Implementiere folgende Prädikate:

- `peano/1` soll beweisbar sein, wenn der übergebene Term eine Peano-Zahl ist, also z.B. die Form `o`, `s(o)`, `s(s(o))`, ... hat.
- `add/3` soll die Addition auf Peano-Zahlen implementieren.
- Die Multiplikation kann wie folgt definiert werden:

```
mult(o, _, o).  
mult(s(x), Y, Z) :- add(U, Y, Z), mult(X, Y, U).
```

Eine alternative Implementierung könnte so aussehen:

```
mult(o, _, o).  
mult(s(x), Y, Z) :- mult(X, Y, U), add(U, Y, Z).
```

Welcher der beiden Implementierungen ist besser? Betrachte während deiner Überlegungen auch folgende Anfrage `?- mult(s(s(o)), Y, s(s(s(s(o))))`. – also sinngemäß die Anfrage  $2 \cdot y = 4$ .

- Implementiere weiter die Prädikate `lt/2`, `eq/2` für Peano-Zahlen, also die  $<$ -Relation und Gleichheit auf Peano-Zahlen.

### TEST 205 ☹

Das Sieb des Eratosthenes ist ein Algorithmus zur Bestimmung von Primzahlen. Dieses wollen wir nun in Prolog implementieren – mit Peano-Zahlen natürlich.

- Implementiere zuerst ein Prädikat `range/3`, das eine Liste berechnet, die alle Ganzzahlen in einem vorgegebenen Intervall enthält. Zum Beispiel soll `?- range(s(o), s(s(s(o))), [s(o), s(s(o))])` beweisbar sein.
- Als Nächstes implementiere ein Prädikat `filter_by_prime/3`, das alle Elemente einer Liste entfernt, die durch eine gegebene Primzahl teilbar sind.
- Zuletzt benötigen wir noch `primes/2`, das alle Primzahlen bis zu einer angegebenen Zahl berechnen soll. Implementiere dieses mithilfe der bereits vorbereiteten Prädikate.
- Mit `Test 206` und einem weiteren Prädikat `map_to_nat/2` kannst du die Liste der Primzahlen in eine lesbare Form bringen – du kannst auch ein allgemeines `map/3` mithilfe des Prädikats höherer Ordnung `call` implementieren.

### TEST 206

Implementiere ein Prädikat `to_nat/2`, das eine Peano-Zahl in eine natürliche Zahl konvertiert. Nutze dafür `is/2`. Wieso terminiert die Anfrage `?- to_nat(P, 3)` nicht?

### TEST 207 ☹

Ein Graph sei dargestellt als eine Liste von Kanten. Die Kanten seien wiederum als Tupel dargestellt.

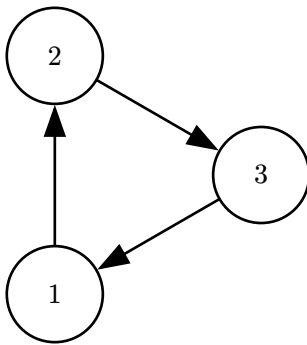
Implementiere ein Prädikat `reachable/3`, das bestimmt, ob ein Knoten von einem anderen Knoten aus erreichbar ist. Du benötigst voraussichtlich ein weiteres vierstelliges Hilfsprädikat.

Dein Programm soll sich wie folgt verhalten:

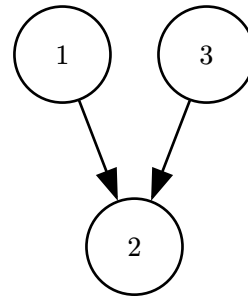
```
?- reachable(1, 3, [(1, 2), (2, 3), (3, 1)]).  
true ;  
false.
```

```
?- reachable(1, 3, [(1, 2), (3, 2)]).  
false.
```

Hier sind die Eingabe-Graphen nochmal visualisiert.



Eingabe-Graph der ersten Anfrage mit unendlich vielen gerichteten Pfaden von 1 bis 3



Eingabe-Graph der zweiten Anfrage mit keinem gerichteten Pfad von 1 nach 3

### TEST 208

Wie unterscheiden sich die Gleichheit `==/2` in Prolog und die Gleichheit in Haskell?

### TEST 209

Welche Konzepte, die Haskell verwendet, stecken hinter den Anfragen

- `?- just((X, Y)) = just((1, 2))` und
- `?- to(X, to(X, list(X))) = to(int, to(int, list(int)))?`

### TEST 210

Eine Dur-Tonleiter hat folgendes Muster relativ zum vorherigen Ton in Halbtonschritten:

+2, +2, +1, +2, +2, +2, +1

Zum Beispiel erhalten wir mit dem Grundton C, so die C-Dur Tonleiter

C - D - E - F - G - A - B - C.

D ist zwei Halbtöne von C entfernt, E ist zwei Halbtöne von D entfernt, F ist einen Halbton von E entfernt und so weiter.

Schreibe ein Prädikat `all_major/2`, dass alle diatonischen Dur-Akkorde (wie in [Test 196](#) definiert) einer Dur-Tonleiter mit gegebenen Grundton berechnet. Diatonisch bedeutet, dass der Akkord nur aus Tönen der gegebenen Tonleiter bestehen darf. Für die C-Dur Tonleiter sind die gesuchten Akkorde C-Dur, F-Dur und G-Dur.

### TEST 211

Entwickle ein Prädikat `nth/3`, dass das  $n$ -te Element einer Liste zurückgibt. Zum Beispiel soll folgende Anfrage beweisbar sein.

```
?- nth([3, 1, 4, 1, 5], s(s(o)), X).  
X = 4.
```

### TEST 212 ☹

Für die nächste Ausgabe des Mittelerde-Kuriers benötigst du noch ein Titelbild für den Artikel „Die Gefährten ziehen zum Schicksalsberg“. Dafür möchtest du die Gefährten der Größe nach aufstellen. Merry und Pippin sind sich nicht einig darüber, wer der größere der beiden ist. Du entscheidest dich, beide Varianten zu fotografieren.

Die Größen-Relation ist mithilfe des folgenden Prädikats festgehalten.

```
before(X, Y, Xs) :- append(_, [X|R], Xs), append(_, [Y|_], R).
```

```
smaller(X, Y) :- before(X, Y, [frodo, merry, pippin, sam]).  
smaller(pippin, merry).
```

- Implementiere ein Prädikat `insert/4`, dass ein Element bzgl. einer Ordnung in eine gegebene Liste einfügt.

```
?- insert(smaller, merry, [pippin], Fs).  
Fs = [merry, pippin] ;  
Fs = [pippin, merry].
```

Falls dir Prädikate höherer Ordnung Schwierigkeiten bereiten, kannst du auch zuerst versuchen, eine Implementierung mit einer festen `<`-Relation angeben.

- Implementiere mithilfe des `insert`-Prädikats Sortieren durch Einfügen. Das Prädikat sollte dann beide Sortierungen ausgeben.

```
?- isort(smaller, [sam, pippin, frodo, merry], Fs).  
Fs = [frodo, merry, pippin, sam] ;  
Fs = [frodo, pippin, merry, sam] ;
```

### **Selbstevaluation**

Ich bin in der Lage, ...

- das Berechnungs- und Ableitungsprinzip von Prolog informell zu erklären und anzuwenden, insbesondere Backtracking und Nichtdeterminismus,
- Programme relational zu modellieren, d.h., Fakten, Regeln und Termstrukturen zu entwerfen und Prolog bewusst nicht funktional sondern relational einzusetzen,
- Zusammenhänge zwischen Prolog und Haskell herzustellen, etwa Funktionen gegenüber Relationen, **Bool**-Funktionen, Listenverarbeitung, Nichtdeterminismus (Listenmonade) und Gleichheit,
- zentrale Prolog-Prädikate zu analysieren und nutzen, insbesondere `append/3`, `member/2`, Rekursion sowie deren Verhalten bei verschiedenen Anfrageformen,

## Elementare Programmiertechniken

### **Referenzen**

- Skript: Logische Programmierung, Elementare Programmiertechniken

#### TEST 213

Aus welchen Teilen besteht das generate-and-test Schema?

#### TEST 214

Wie hängen musterorientierte Prädikate und induktiv definierte Funktionen miteinander zusammen?

## CHALLENGE 21 ⌚

Das Erfüllbarkeitsproblem der Aussagenlogik fragt, ob es für eine gegebene aussagenlogische Formel eine Belegung der Variablen mit wahr oder falsch gibt, sodass die Formel insgesamt wahr ist.

Zur Erinnerung, die Syntax aussagenlogischer Formeln ist wie folgt beschrieben:

- $\top$  (true),  $\perp$  (false) und Variablen sind aussagenlogische Formeln.
- Seien  $F, G$  aussagenlogische Formeln, dann sind
  - $\neg F$  (Negation),
  - $F \wedge G$  (Konjunktion) und
  - $F \vee G$  (Disjunktion)

aussagenlogische Formeln.

Die Semantik der atomaren Formeln und der Junktoren wird durch die entsprechenden booleschen Funktionen und eine Belegung der Variablen definiert.

- Implementiere zuerst `bool/1`, das den Wertebereich festlegen soll, und die booleschen Funktionen `and/3`, `or/3`, `neg/2`.
- Implementiere ein Prädikat `get_vars/2`, das alle Bezeichner von (freien) Variablen einer gegebenen Formel zurückgibt. Eine Variable soll in einer Formel als Term `var(X)` gekennzeichnet werden.  $X$  ist dann der Bezeichner der Variable. Stelle dabei sicher, dass die Liste der Bezeichner keine Duplikate enthält – dafür kannst du z.B. `list_to_set/2` verwenden.
- Implementiere als Nächstes ein Prädikat `assignment/2`, das alle Belegungen generiert. Eine Zuweisung soll als Liste von Tupeln dargestellt werden (vgl. Beispielanfragen).
- Implementiere ein Prädikat `eval/3`, das beweisbar ist, wenn die gegebene Formel erfüllbar unter der gegebenen Belegung ist. Hier sind ein paar Beispielanfragen:

```
?- eval(and(true, var(x)), [(x, true)], false).  
false.
```

```
?- eval(and(true, var(x)), [(x, B)], false).  
B = false.
```

- Implementiere ein Prädikat `sat/2`, das alle belegten Formeln berechnet. Hier sind ein paar weitere Beispielanfragen:

```
?- sat(or(var(x), neg(var(x))), A).  
A = [(x, true)] ;  
A = [(x, false)] ;  
false.
```

```
?- sat(and(var(x), or(neg(var(y)), neg(var(z))))), A).  
A = [(x, true), (y, true), (z, false)] ;  
A = [(x, true), (y, false), (z, true)] ;  
A = [(x, true), (y, false), (z, false)] ;  
false.
```

```
?- sat(and(var(x), neg(var(x))), A).  
false.
```

#### TEST 215 ☹

In diesem Test ergänzen wir den SAT-Löser aus [Challenge 21](#) um die Implikation und Äquivalenz. Anstatt das `eval`-Prädikat entsprechend zu erweitern, wollen wir diese Terme in andere Terme übersetzen, die wir bereits auswerten können. Das können wir mit folgenden Regeln erreichen:

$$(A \Rightarrow B) \Leftrightarrow (\neg A \vee B) \quad \text{und} \quad (A \Leftrightarrow B) \Leftrightarrow (A \Rightarrow B \wedge B \Rightarrow A)$$

Implementiere dafür ein Prädikat `desugar/2`. Als Konstruktoren kannst du z.B. `impl/2` und `iff/2` verwenden.

#### TEST 216 ☹

Wir machen uns auf in die Kombinatorik und wollen ein paar nützliche Prädikate definieren, die uns helfen Suchräume zu durchlaufen.

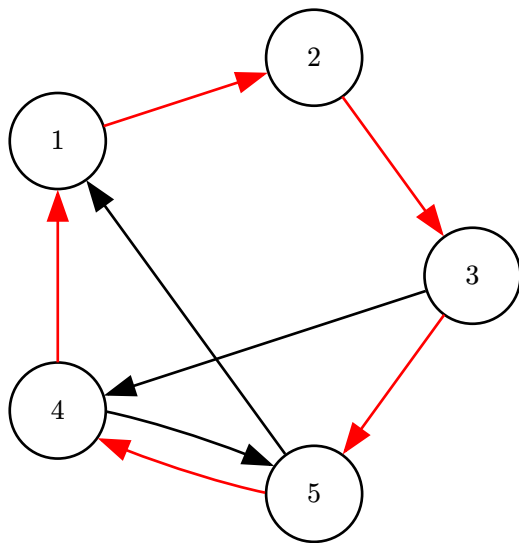
- Implementiere ein Prädikat `varia_rep/3`, das genau dann beweisbar ist, wenn eine über gegebene Liste eine Variation einer anderen ist und aus  $k$  Elementen besteht – es soll z.B.  
?- `varia_rep([0, 1], 4, [1, 0, 0, 1])`. beweisbar sein.
- Implementiere ein Prädikat `perms/2`, dass genau dann beweisbar ist, wenn zwei übergebene Listen Permutationen voneinander sind.

## CHALLENGE 22 ⌚

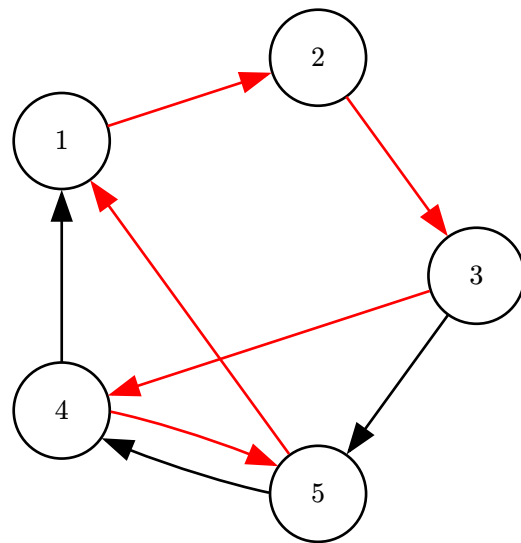
Oft eignet sich Prolog gut, um Algorithmen für Entscheidungsprobleme, die in NP liegen, zu implementieren. Als Methode dafür hast du generate-and-test kennengelernt – wir generieren mögliche (also korrekte oder falsche) Lösungen und entscheiden dann, ob sie korrekt sind.

Ein Hamiltonkreis ist ein Kreis in einem Graph, der jeden Knoten genau einmal enthält. Wir wollen diese als Listen darstellen. Durch Rotation dieser Liste erhalten wir äquivalente Hamiltonkreise. Diese wollen wir ignorieren und wählen stattdessen einen kanonischen Repräsentanten, indem wir uns auf die Liste beschränken, die mit dem kleinsten Knoten beginnt.

Im folgenden Graph gibt es zwei Hamiltonkreise.



Hamiltonkreis 1: (1, 2, 3, 5, 4)



Hamiltonkreis 2: (1, 2, 3, 4, 5)

Um Hamiltonkreise zu bestimmen, arbeiten wir uns von einem naiven Pfad-Generator zu einem, der versucht nach und nach den Suchraum durch vorhandene Informationen zu verkleinern (auch pruning genannt). Danach verifizieren wir, ob ein gegebener Pfad ein Hamiltonkreis ist. Graphen kannst du z.B. als Liste von Kanten darstellen.

```
graph([(1, 2), (2, 3), (3, 4), (3, 5), (4, 5), (4, 1), (5, 1), (5, 4)]).
```

Es lohnt sich, [Test 216](#) vor dieser Challenge gemacht zu haben, wenn du mit den naiveren Pfad-Generatoren starten möchtest.

- Implementiere zuerst einen (polynomiellen) Verifizierer, also ein Prädikat `is_hamilton/2`, das bestimmt, ob ein gegebener Pfad in einem gegebenen Graphen einem Hamiltonkreis entspricht.
- Implementiere ein Prädikat `path/2` und verbessere sukzessiv mit den folgenden Ideen:
  - Prüfe jede beliebige Knotenfolge bestehend aus  $|V|$  Knoten. Das heißt, der Suchraum entspricht zu Beginn  $V^{|V|}$  für einen Graphen  $(V, E)$ .
  - Jede zwei Knoten dieser Folge müssen paarweise unterschiedlich sein.
  - Jede zwei aufeinanderfolgenden Knoten entsprechen einer Kante in dem Eingabegraphen. Die Knotenfolge ist also ein Pfad.
- Implementiere zuletzt das Prädikat `hamilton/2`, das Hamiltonpfade bzw. -kreise berechnet.

Beobachte experimentell, wie sich die Laufzeit durch die einzelnen Pruning-Schritte verändert.

### **Selbstevaluation**

Ich bin in der Lage, ...

- einen Suchraum zu generieren,
- den gleichen Suchraum zu durchsuchen,
- und währenddessen (große) Teile des Suchraums, die keine Lösung enthalten können, kategorisch auszuschließen.

## Rechnen in der Logikprogrammierung

### **Referenzen**

- Skript: Logische Programmierung, Rechnen in der Logikprogrammierung

Im Folgenden stehen Großbuchstaben für Variablen und Kleinbuchstaben für atomare Ausdrücke – wenn nicht anders im Test oder in der Challenge eingeführt.

#### TEST 217

Was besagt die Abtrennungsregel (modus ponens)?

#### TEST 218

Was ist das (einfache) Resolutionsprinzip?

#### TEST 219

Wann ist eine Anfrage mithilfe des Resolutionsprinzips beweisbar?

#### TEST 220

Mithilfe welcher Methode stellen wir fest, ob ein Literal zu einer linken Regelseite passt?

#### TEST 221

Wie sind Terme definiert?

#### TEST 222

Welche der folgenden Terme sind syntaktisch korrekt?

- $X$
- $a$
- $XX$
- $f(X, a)$
- $f(f(X))$
- $g(f(X)(Y))$

#### TEST 223

Mithilfe welcher Methode ersetzen wir Variablen in Termen?

#### TEST 224

Wie ist die Substitution auf Termen definiert? Was wird insbesondere durch eine Substitution verändert und was nicht?

#### TEST 225

Falls du Challenge 21 gemeistert hast – an welcher Stelle deines Programms führst du eine Substitution durch?

#### TEST 226

Sei  $\sigma = \{X \mapsto 1, Y \mapsto 2\}$  eine Substitution. Welche Anwendungen oder Aussagen sind korrekt?

- $\sigma(\text{add}(X, Y)) = \text{add}(1, 2)$
- $\sigma(\text{eq}(X, X)) = \text{eq}(1, X)$
- $\sigma(f(g(X, Y), Z))$  ist nicht definiert.

#### TEST 227

Welche der folgenden Substitutionen sind wohldefiniert?

- $\sigma = \{X \mapsto 1\}$
- $\sigma = \{X \mapsto X\}$
- $\sigma = \{f(X) \mapsto f(Y)\}$
- $\sigma = \{X \mapsto Y, Y \mapsto X\}$
- $\sigma = \{[X|Y] \mapsto [1|[]]\}$

#### TEST 228

Sind  $[X]$  und  $[1, 2]$  unifizierbar?

#### TEST 229

Wende die Substitution  $\sigma = \{X \mapsto 1, Y \mapsto f(X)\}$  auf den Term  $g(X, h(Y))$  an, ohne einen Zwischenschritt auszulassen.

#### TEST 230

Was ist ein Unifikator? Was ist ein allgemeinster Unifikator?

#### TEST 231

Was sind die Ergebnisse der folgenden Kompositionen von Substitutionen?

- $\{Y \mapsto X\} \circ \{Z \mapsto 1\}$
- $\{Y \mapsto X\} \circ \{Z \mapsto Y\}$
- $\{Z \mapsto Y\} \circ \{Y \mapsto X\}$

#### TEST 232

Wie ist Unifizierbarkeit definiert?

#### TEST 233

Wir haben eine Substitution als eine Abbildung  $\mathcal{X} \rightarrow \mathcal{T}(\mathcal{X})$  definiert, wobei  $\mathcal{X}$  eine Menge von Variablen und  $\mathcal{T}(\mathcal{X})$  die Menge aller Terme über  $\mathcal{X}$  sind. Angenommen eine Substitution sei eine Abbildung  $\mathcal{T}(\mathcal{X}) \rightarrow \mathcal{T}(\mathcal{X})$ . Wieso ist dann der Begriff der Unifizierbarkeit weniger brauchbar?

#### TEST 234

Beim Durchführen des Unifikationsalgorithmus treten in zwei verschiedenen Iterationen  $i, j$  mit  $i < j$  die Substitutionen

$$\sigma_i = \{X \mapsto f(Y)\} \text{ und } \sigma_j = \{X \mapsto a\}$$

auf. Ist ein solcher Verlauf korrekt? Begründe deine Antwort. Dabei kann angenommen werden, dass  $X$  nicht in  $Y$  vorkommt.

#### TEST 235

Warum gilt  $\{Y \mapsto X\} \circ \{Z \mapsto Y\} = \{Z \mapsto Y\} \circ \{Y \mapsto X\}$  nicht?

#### TEST 236

Warum ist  $\{Y \mapsto X\} \circ \{Y \mapsto 2\} = \{Y \mapsto 2\}$ ?

#### TEST 237

Warum ist die Komposition von Substitutionen im Allgemeinen nicht die Vereinigung, also  $\varphi \circ \psi = \varphi \cup \psi$ ?

#### TEST 238

Wann existiert ein allgemeinsten Unifikator?

#### TEST 239

Was ist die Unstimmigkeitsmenge zweier Terme, was gibt sie an und wie ist sie definiert?

#### TEST 240

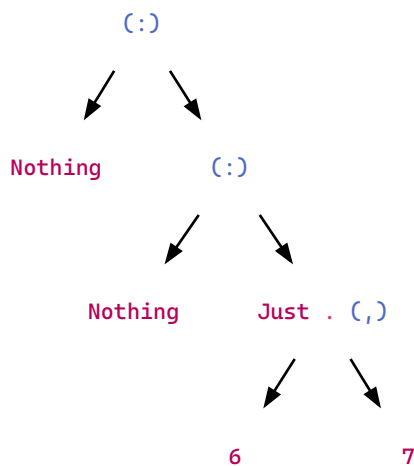
Welche Unstimmigkeitsmengen sind korrekt berechnet?

- $ds(f(X), f(1)) = \{X, 1\}$
- $ds(1, 2) = \{1, 2\}$
- $ds(g(1, 2), h(1, 2)) = \emptyset$
- $ds(X, Y) = \emptyset$
- $ds(f(X, Y), f(1, 2)) = \{Y, 2\}$

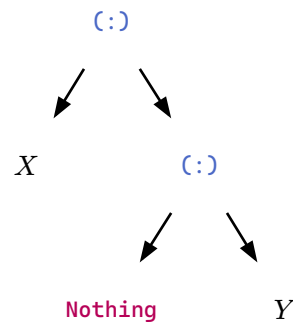
#### TEST 241

Finde die allgemeinsten Unifikatoren für die folgende Terme:

- $(X + 1) \cdot Y + Z$  und  $((3 + Z) + 1) \cdot Z + 3$ ,
- $f(X, Y)$  und  $f([Y|R], 1)$ ,
- $f([X|R], X, R)$  und  $f([1, 2, 3], [Z|S])$ , und
- 



und



#### TEST 242

Unter welchen Umständen terminiert der Unifikationsalgorithmus?

#### TEST 243

Wie können wir überprüfen, ob eine von uns berechnete Substitution (z.B. als Ergebnis des Unifikationsalgorithmus) ein Unifikator ist? Wie können wir Evidenz gewinnen, dass die Substitution wahrscheinlich keinen Fehler enthält?

#### TEST 244

Aus welcher Eigenschaft des Unifikationsalgorithmus folgt, dass ein berechneter Unifikator ein allgemeinsten Unifikator ist?

#### TEST 245

Seien  $t_1, t_2$  Terme. Wenn  $ds(\sigma(t_1), \sigma(t_2)) = \emptyset$  gilt, was können wir über  $\sigma$  folgern?

#### TEST 246

Welche Arten von Fehlschlägen können während des Unifikationsalgorithmus auftreten? Unter welchen Umständen treten diese auf?

#### TEST 247

In welches Problem laufen wir, wenn wir mit einer Substitution, die sich aus  $ds(t_1, t_2) = \{X, f(X)\}$  ergibt, naiv weiter rechnen würden, wobei  $t_1$  und  $t_2$  Terme sind?

#### TEST 248

Was bedeutet es, wenn der Vorkommenstest (occurs check) positiv ist?

#### TEST 249

Gebe ein Beispiel für eine Eingabe an, für das der Unifikationsalgorithmus exponentielle Laufzeit bzgl. der Größe der Eingabeterme hat.

Die Größe eines Terms  $|\cdot|$  wir z.B. wie folgt berechnen:

- $|X| = 1$ , falls  $X$  Variable ist,
- $|a| = 1$ , falls  $a$  Konstante ist und
- $|f(t_1, \dots, t_n)| = 1 + \sum_{i=1}^n |t_i|$  für Terme  $t_1, \dots, t_n$  und  $n$ -stelligen Funktor  $f$ .

#### TEST 250

Wieso kommt der Fall der exponentiellen Laufzeit in der Größe der Eingabeterme überhaupt zustande?

Zu den am häufigsten gemachten Fehlern beim Angeben eines SLD-Baums ist die fehlerhafte Anwendung des Unifikationsalgorithmus. Beim intuitiven Anwenden des Algorithmus suchen wir oftmals nur die Stellen in den Ausgangstermen, an denen Variablen auf Terme stoßen und vergessen, zuvor freie und bereits gebundene Variablen zu ersetzen. Als Unifikator für z.B.  $f(1, Y)$  und  $f(X, [X|R])$  wird oft

$$\sigma = \{X \mapsto 1, Y \mapsto [X|R]\}$$

angegeben. Das ist falsch, da

$$\sigma(f(1, Y)) = f(1, [X|R]) \neq f(1, [1|R]) = \sigma(f(X, [X|R]))$$

Auf der linken Seite könnten wir den Unifikator zwar erneut anwenden, um den korrekten Term zu erhalten, damit halten wir uns allerdings nicht an die Definition der Unifizierbarkeit.

Wenn wir den Unifikationsalgorithmus auf die beiden Terme rigoros anwenden, sehen wir in der ersten Iteration, dass der Teilterm  $[X|R]$  nicht mehr in  $\sigma_1(t_2) = f(1, [1|R])$  vorkommt.

| $k$ | $\sigma_k$                         | $\sigma_k(t_1)$ | $\sigma_k(t_2)$ | $\text{ds}(\sigma_k(t_1), \sigma_k(t_2))$ |
|-----|------------------------------------|-----------------|-----------------|-------------------------------------------|
| 0   | $\emptyset$                        | $f(1, Y)$       | $f(X, [X R])$   | $\{X, 1\}$                                |
| 1   | $\{X \mapsto 1\}$                  | $f(1, Y)$       | $f(1, [1 R])$   | $\{Y, [1 R]\}$                            |
| 2   | $\{X \mapsto 1, Y \mapsto [1 R]\}$ | $f(1, [1 R])$   | $f(1, [1 R])$   | $\emptyset$                               |

Der obige Fehler kommt zustande, da wir beim Berechnen der zweiten Unstimmigkeitsmenge noch die Ausgangsterme,  $t_1$  und  $t_2$ , anstatt der neuen Terme,  $\sigma_1(t_1)$  und  $\sigma_1(t_2)$ , nutzen, in denen  $X$  dann substituiert ist.

Wie können wir daran denken? Wenn eine Variable  $X$  in der  $k$ . Iteration gebunden wird, dann kann  $X$  weder in  $\sigma_k(t_1)$  noch in  $\sigma_k(t_2)$  vorkommen. Für dieses Beispiel bedeutet das, wenn wir kurz davor sind,  $Y \mapsto [X|R]$  hinzuschreiben, sollten wir uns vergewissern, ob bereits gebundene Variablen auf der rechten Seite vorkommen (hier  $X$ ) und entsprechend ersetzen (mit 1). In einem anderen Fall können wir Variablen haben, die bereits an Terme gebunden sind aber noch freie Vorkommen von Variablen haben – betrachte z.B.  $f(X, 1)$  und  $f([Y|R], Y)$  mit  $\{Y \mapsto 1\} \circ \{X \mapsto [Y|R]\}$ . Hier müssen wir andersherum schauen, ob die neue Belegung alte Belegungen verändert. Das geht aus der linken Menge der Definition der Komposition hervor.

$$\varphi \circ \psi = \{v \mapsto \varphi(t) \mid v \mapsto t \in \psi, \varphi(t) \neq v\} \cup \{v \mapsto t \mid v \mapsto t \in \varphi, v \notin D(\psi)\}.$$

Ein Unifikator  $\sigma$  erfüllt die Eigenschaft

$$\forall v \mapsto t \in \sigma : \text{Vars}(t) \cap D(\sigma) = \emptyset \text{ oder alternativ } \sigma \circ \sigma = \sigma.$$

Es dürfen also auf den rechten Seiten keine Variablen vorkommen können, die durch den Unifikator selber gebunden werden. Solange diese Eigenschaft nicht erfüllt ist, müssen wir solche Vorkommen durch die Belegungen ersetzen. In [Test 241](#) findest du Beispiele zum Üben.

Hier sind andere Fehler, die seltener geschehen, über die man sich trotzdem Gedanken machen kann.

- Basierend auf dem obigen Beispiel wird als Zwischenschritt manchmal

$$\{Y \mapsto [X|R], X \mapsto 1\} = \{Y \mapsto [1|R], X \mapsto 1\}$$

aufgeschrieben. Das ist falsch, denn hier besteht keine Gleichheit. Die linke Substitution bildet  $Y$  auf  $[X|R]$  ab, während die rechte  $Y$  auf  $[1|R]$  abbildet. Die Terme  $[X|R]$  und  $[1|R]$  sind nicht gleich – selbst wenn in den beiden Substitutionen  $X$  auf 1 abgebildet wird.

- In den Unifikatoren müssen alle Belegungen stehen, die benötigt werden, um die linke Seite einer Regel mit dem derzeitig selektierten Literal zu unifizieren. Das ist unabhängig davon, ob diese Belegungen am Ende zur Berechnung einer Lösung benötigt werden. Es geht um die korrekte Anwendungen eines Algorithmus und nicht darum, dass ihr logisch schließen könnt, dass manche Belegungen irrelevant sind.

#### TEST 251

Aus welchen Komponenten setzt sich das allgemeine Resolutionsprinzip zusammen? Wie wird es auch genannt?

#### TEST 252

SLD-Resolution-Aufgaben findest du den Übungsserien sowohl als Präsenz- und Hausaufgaben. Darüber findest du sie auch den Altklausuren des Moduls. Diese kannst du auf der Webseite der Fachschaft oder ggf. im aktuellen Moodle-Kurs herunterladen.

#### TEST 253

Was legt die Selektionsfunktion der SLD-Resolution fest? Welche verwendet Prolog?

#### TEST 254

Gegeben sei die Anfrage  $?- A_1, \dots, A_m$ . Du stellst fest, dass  $A_1$  mit der linken Seite der Regel  $L :- L_1, \dots, L_n$  unifizierbar ist. Welche Anfrage ist in einem SLD-Resolutionsschritt daraus ableitbar.

#### TEST 255

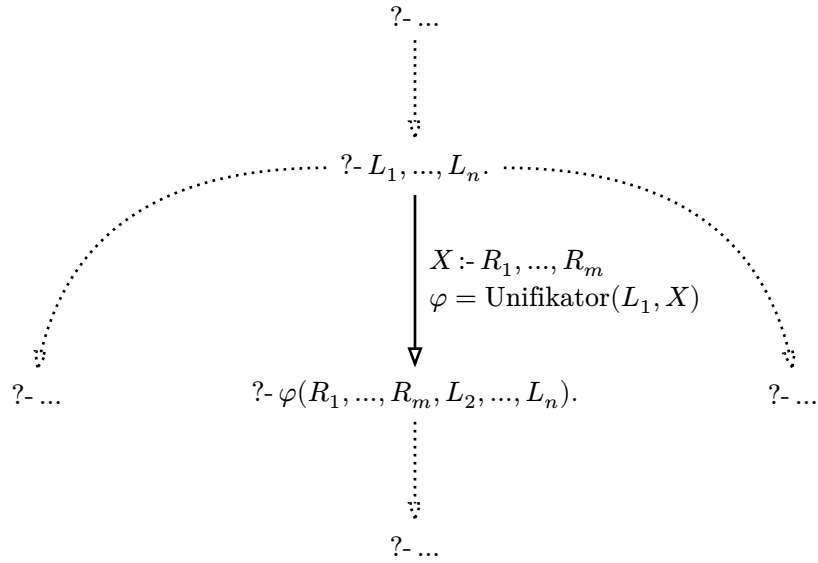
Wieso benennen wir Variablen einer Regel um, bevor wir eine Unifikation als Teil eines SLD-Resolutionsschritt durchführen?

#### TEST 256

Wodurch ergibt sich die Struktur eines SLD-Baums?

Ein SLD-Baum stellt einzelne SLD-Resolutionsschritte graphisch dar – betrachte parallel beim Lesen den unten abgebildeten Ausschnitt eines SLD-Baums. Die Wurzel des Baumes ist mit einer gegebenen zu beweisenden Anfrage beschriftet. Jeder Resolutionsschritt beginnt bei einem Knoten, der mit einer Anfrage  $?- L_1, \dots, L_n$  ( $n \geq 1$ ) beschriftet ist – wenn  $n = 0$  ist, sind wir bereits fertig. Basierend auf der Auswertungsstrategie von Prolog wählen wir das erste Literal  $L_1$  dieser Anfrage aus und versuchen dieses zu beweisen. Dafür schauen wir uns gegebene (frisch umbenannte!) Klauseln der Reihe nach an. Wenn  $X$  die nächste linke Seite einer Klausel  $X :- R_1, \dots, R_m$  ( $m \geq 0$ ) mit  $L_1$  unifizierbar ist, d.h., es existiert  $\varphi = \text{Unifikator}(L_1, X)$ , dann ersetzen wir das Literal  $L_1$  mit der rechten Seite  $R_1, \dots, R_m$  und wenden auf die neu gewonnene Anfrage  $R_1, \dots, R_m, L_2, \dots, L_n$  den Unifikator an, den wir aus der Unifikation des Literals mit der linken Seite erhalten, also  $\varphi(R_1, \dots, R_m, L_2, \dots, L_n)$ . Die umbenannte Klausel geben wir neben dem berechneten Unifikator als Teil der Kantenbeschriftung als Hilfestellung mit an. Wenn die Anfrage bewiesen ist, also die Anfrage aus keinem Literal mehr besteht ( $?- .$ ), dann haben wir einen Beweis für die Anfrage gefunden und geben das zugehörige Ergebnis  $\sigma$  an. Das heißt, wir berechnen die Komposition der Unifikatoren von der Lösung bis zur Wurzel und schränken sie danach die freien Variable der ursprünglichen Anfrage ein. Wenn  $A$  die zu beweisende Aussage ist und  $\varphi_1, \dots, \varphi_l$  Unifikatoren sind, die wir über die Resolutionsschritte von  $A$  nach  $?- .$  erhalten haben, dann ist die Lösung in dem Fall  $\sigma = (\varphi_l \circ \dots \circ \varphi_1) \upharpoonright_{\text{vars}(A)}$ . Im anderen Fall führen wir weitere SLD-Resolutionsschritte durch. Wenn die Anfrage leer ist oder keine weiteren Klauseln zum betrachteten Literal passen, backtracken wir. Das heißt, wir betrachten die zuvor bearbeitete Anfrage erneut und versuchen, einen alternativen Beweis zu finden, indem wir alternative Resolutionsschritte auf Basis von bisher nicht ausprobierten Klausuren durchführen. So können wir z.B. zu der Zwischenanfrage  $?- L_1, \dots, L_n$  zurückkommen und finden neben der Klausel  $X :- R_1, \dots, R_m$  möglicherweise weitere passende Klauseln. Das ist hier angedeutet durch die rechteste Kante. Im anderen Fall backtracken wir weiter.

Wenn das aktuelle Literal ein cut ist, dann betrachten wir im nächsten Schritt die Anfrage ohne den cut. Wichtig ist, dass das Beweisen des cuts dazu führt, dass sowohl für die einführende Regel des cuts als auch für alle Literale die danach auf der linken Seite vom cut stehen keine alternativen Resolutionsschritte durchgeführt werden, sobald wir einen Beweis für die linke Seite gefunden haben.



#### TEST 257

Gegeben sei folgendes Prolog-Programm.

```
upTo(o, -, []).  
upTo(s(N), A, [A|Xs]) :- upTo(N, A, Xs).  
upTo(s(N), A, []).
```

Betrachte die Anfrage `?- upTo(s(s(o)), a, Xs), Xs = [X, Y]`. Die erste Lösung ergibt sich aus der Komposition der folgenden Unifikatoren und dessen anschließlichen Einschränkung auf die freien Variablen der Anfrage. Für die erste Lösung  $\sigma_1$  der obigen Anfrage lauten die Unifikatoren z.B.

- $\varphi_1 = \{N_1 \mapsto s(o), A_1 \mapsto a, Xs \mapsto [a|Xs_1]\}$ ,
- $\varphi_2 = \{N_2 \mapsto o, A_2 \mapsto a, Xs_1 \mapsto [a|Xs_2]\}$ ,
- $\varphi_3 = \{Xs_2 \mapsto []\}$  und
- $\varphi_4 = \{X \mapsto a, Y \mapsto a, X_4 \mapsto [a, a]\}$ .

$\varphi_4 \circ \varphi_3 \circ \varphi_2 \circ \varphi_1$  rigoros auszurechnen, ist müheselig und zeitaufwendig. In diesem Test versuchen wir, einen schnelleren Weg zur Berechnung einer Lösung zu ergründen, wenn wir die Unifikatoren bereits haben.

- Sei  $G = (V, E)$  ein gerichteter Graph mit

$$V = \bigcup_{k=1}^4 D(\varphi_k) \text{ und } E = \{(V, W) \mid k \in \{1, \dots, 4\}, V \mapsto T \in \varphi_k, W \in \text{Vars}(T)\}.$$

Stelle diesen Graph dar.

- Wieso ist  $G$  azyklisch? Wieso haben die freien Variablen der Anfrage keine eingehenden Kanten in dem Graphen?
- Betrachte die Knoten  $X$ ,  $Y$  und  $Xs$ . Wie hängen ihre Nachfolger mit ihnen zusammen?
- Wie erhalten wir  $Xs \mapsto [a, a]$  aus dem Teilgraphen von  $G$ , der  $Xs$  enthält und alle von diesem Knoten aus erreichbaren Knoten?
- Wie ergibt sich aus der vorherigen Betrachtung ein schnelleres Berechnungsverfahren?
- Betrachten  $G$  ohne die von  $X$ ,  $Y$  und  $Xs$  und die von diesen erreichbaren Knoten. Welche Bedeutung haben diese bei der Berechnung der Lösung  $\sigma_1 = (\varphi_4 \circ \varphi_3 \circ \varphi_2 \circ \varphi_1)|_{\{X, Y, Xs\}}$ ?

#### TEST 258

Welche Auswertungsstrategie findet immer eine Lösung, falls eine existiert?

#### TEST 259

Warum wird die Tiefensuche als Auswertungsstrategie der Breitensuche bevorzugt?

#### TEST 260

Wie ergibt sich die Reihenfolge der Kindknoten eines Knoten in einem SLD-Baum?

#### TEST 261

Welche Rolle spielt Backtracking in Prolog?

#### TEST 262

Wie werden Variablen in Prolog gebunden?

#### TEST 263

Wieso wird empfohlen, dass Klauseln für Spezialfälle vor allgemeineren Klauseln stehen sollten?

#### TEST 264

Gegeben sei das Prolog-Programm.

```
fruit(apple).  
fruit(banana).  
fruit(cranberry).
```

```
salad(F1, F2, F3) :- fruit(F1), fruit(F2), fruit(F3).
```

Was sind die ersten vier Lösungen, die Prolog berechnet? Warum verändern sich die Belegungen für F2 und F3 zuerst?

#### TEST 265

Gegeben sei das Prolog-Programm:

```
nth(0, [X|_], X).  
nth(N, [_|Xs], X) :- N > 0, M is N - 1, nth(M, Xs, X).
```

Wir beginnen, eine Lösungen zu berechnen, wie Prolog sie berechnen würde, für die Anfrage

```
?- nth(1, [3, 1, 4, 1]).
```

```
?- nth(1, [3, 1, 4, 1]).
```

```
| - (2. Regel)
```

```
?- 1 > 0, M is 1 - 1, nth(M, [1, 4, 1], X).
```

```
| - (2. Regel)
```

```
?- 1 > 0, M is 1 - 1, M > 0, M is M - 1, nth(M, [4, 1], X).
```

```
| -
```

```
...
```

Was ist hier schief gelaufen?

#### TEST 266

Begründe warum die folgende Aussage korrekt oder falsch ist: Beim Berechnen der Unstimmigkeitsmenge kann folgendes Ergebnis herauskommen:

$$\{g(X, Y), g(Y, X)\}.$$

#### TEST 267

Gegeben sei folgendes Prolog-Programm.

```
append([], L, L).  
append([E|R], L, [E|RL]) :- append(R, L, RL).
```

Es wird die Anfrage `?- append(X, Y, [1, 2]).` gestellt. Beim Anwenden der zweiten Regel wurde die Substitution

$$\sigma_1 = \{X \mapsto [E_1|R_1], Y \mapsto L_1, E_1 \mapsto 1, RL_1 \mapsto [2]\}$$

berechnet. Ist diese Substitution ein Unifikator für `append(X, Y, [1, 2])` und `append([E1|R1], L1, [E1|RL1])`, der aus dem Unifikationsalgorithmus entstanden ist?

#### TEST 268

Gegeben sei folgendes Prolog-Programm.

```
f(X, 2).
```

Welcher Schritt der SLD-Resolution ist hier essentiell, um eine korrekte Lösung zu erhalten. Betrachte dafür die folgende Anfrage `f(1, X).`

#### TEST 269

Welche der folgenden Listen entsprechen syntaktisch korrekten Listen in Prolog. Darüber hinaus, welche der Listen entsprechen der Liste  $(1, 2, 3)$ .

- `[1, 2, 3]`
- `[1, [2, 3]]`
- `[1|[2, 3]]`
- `[1, 2|[3]]`
- `[1, 2|3]`
- `[1|2|3]`
- `[1|[2|[3|[]]]]`

#### TEST 270

Die Komposition von Substitutionen ist definiert durch

$$\varphi \circ \psi = \{v \mapsto \varphi(t) \mid v \mapsto t \in \psi, \varphi(t) \neq v\} \cup \{v \mapsto t \mid v \mapsto t \in \varphi, v \notin D(\psi)\}.$$

Wieso spielen  $\varphi(t) \neq v$  und  $v \notin D(\psi)$  keine Rolle, wenn wir den Unifikationsalgorithmus akribisch anwenden? Wieso spielt es aber eine Rolle, wenn wir z.B.  $\{Y \mapsto X\} \circ \{X \mapsto Y\}$  oder  $\{X \mapsto y\} \circ \{X \mapsto x\}$  betrachten?

#### TEST 271

Wieso tritt das Szenario, in dem wir z.B.  $f(X, 1)$  und  $f(1, X)$  unifizieren müssten, beim Angeben eines SLD-Baums nicht auf?

## Selbstevaluation

Ich bin in der Lage, ...

- Aussagen mithilfe von Modus Ponens und dem Resolutionsprinzip logisch abzuleiten,
- Terme zu erkennen, Substitutionen korrekt anzuwenden und allgemeinste Unifikatoren zu bestimmen,
- SLD-Resolution anzuwenden, Variablen korrekt zu binden und Backtracking nachzuvollziehen, und
- Nichtdeterminismus in Prolog zu verstehen und mehrere Lösungen systematisch zu berechnen (SLD-Bäume mit cuts).

## Negation

### Referenzen

- Skript: Einführung in die Logikprogrammierung, Negation

#### TEST 272

Was verbirgt sich hinter dem Begriff „Negation als Fehlschlag“?

#### TEST 273

Wann ist  $\neg p$  beweisbar?

#### TEST 274

Wieso stimmt die Negation als Fehlschlag nicht mit der prädikatenlogischen Negation überein?

#### TEST 275

Warum sollte  $p$  keine freien Variablen enthalten, wenn wir  $\neg p$  beweisen wollen? Wieso ergibt sich daraus die Empfehlung, dass Negationen soweit wie möglich rechts in einer Regel stehen sollten?

#### TEST 276

Wie sind die Variablen  $x$ ,  $y$  in der folgenden Regel quantifiziert?

$p(x) :- q(x, y).$

#### TEST 277

Nutze  $\forall x : p \Leftrightarrow \neg(\exists x : \neg p)$ , um ein Prädikat `forall/2` zu definieren.

Mithilfe dieses Prädikates soll es möglich sein, folgende Anfrage auszudrücken.

`?- forall(member(X, [1, 2, 3]), X > 0).`

Die Anfrage bedeutet, für alle  $x \in \{1, 2, 3\}$  gilt  $x > 0$ .

Wie können wir mit `forall` die Definition von `not_member` aus [Test 203](#) reparieren?

### Selbstevaluation

Ich bin in der Lage, ...

- die Negation zu nutzen,
- Fallstricke der Negation zu erklären.

## Der Cut-Operator

### Referenzen

- Skript: Einführung in die Logikprogrammierung, Der „Cut“-Operator

#### TEST 278

Wofür verwenden wir den Cut-Operator konzeptionell?

#### TEST 279

Wann ist  $p$  in  $p :- q, !, r.$  beweisbar? Was ist insbesondere der Fall, wenn  $q$  beweisbar ist?

#### TEST 280

Was sind die Ausgaben der jeweiligen Prolog-Programme, wenn die Anfrage  $?- a(1).$  bewiesen wird.

Programm 1:

```
a(1) :- writeln(a).  
a(2) :- writeln(b).  
a(1) :- writeln(c), !.  
a(1) :- writeln(d).  
a(2) :- writeln(e).
```

Programm 2:

```
a(1) :- writeln(a).  
a(1) :- false, !, writeln(b).  
a(2) :- writeln(c).  
a(1) :- !, writeln(d).  
a(1) :- writeln(e), !.  
a(1) :- writeln(f).  
a(2) :- writeln(g).
```

#### TEST 281

Wir ergibt sich aus der Semantik des Cut-Operators ein Fallunterscheidung-Konstrukt?

#### TEST 282

Wie können wir mithilfe des Cut-Operators die Negation als Fehlschlag implementieren?

#### TEST 283

Wie wirkt sich der Cut-Operator auf die Struktur eines SLD-Baums aus?

#### TEST 284

Wie können wir mithilfe des Cut-Operators im folgenden Prädikat Berechnungen sparen?

```
max(X, Y, X) :- X >= Y.  
max(X, Y, Y) :- X < Y.
```

#### TEST 285

Gegeben sei folgendes Prolog-Programm.

```
member(X, [X|_]).  
member(X, [_|R]) :- member(X, R).  
  
member_cut(X, [X|_]) :- !.  
member_cut(X, [_|R]) :- member_cut(X, R).
```

Wie verändert der Cut-Operator das Verhalten in `member_cut/2` von `member/2`?

#### Selbstevaluation

Ich bin in der Lage, ...

- zu erklären, was der Cut-Operator in Prolog macht und wie er das Backtracking beeinflusst,
- die Semantik des Cut-Operators in SLD-Bäumen kenntlich zu machen,
- anhand eines Prolog-Programms vorherzusagen, welche Lösungen durch einen Cut verhindert werden, und
- den Cut gezielt einzusetzen, um unnötiges Backtracking zu vermeiden, ohne die logische Bedeutung eines Programms zu verändern.

## Prolog-Arithmetik und nicht-relationale Prädikate

#### Referenzen

- Skript: Einführung in die Logikprogrammierung, Programmieren mit Constraints, Arithmetik in Prolog (Constraint-Programmierung mit CLP-Bibliotheken entfällt)

#### TEST 286

Wofür wird das Prädikat `is/2` verwendet?

#### TEST 287



Welche Anfragen sind valide?

- `X is 42 - 3`
- `X is Y + 1`
- `Y = 1, X is Y + 1`
- `32 + 10 is X`
- `42 is 40 + Y`

#### TEST 288

Warum können wir `=/2` nicht für Arithmetik verwenden bzw. wieso gilt `42 = 40 + 2` nicht?

#### TEST 289

Welche weiteren Prädikate kennst du neben `is/2` die auch Terme ausrechnen?

#### TEST 290

Das Prädikat `is/2` muss in seinem zweiten Argument hinreichend instanziiert sein, damit wir es auswerten können. Als Faustregel kann man sich merken, dass Prädikate, die arithmetische Ausdrücke oder aussagenlogische Formeln auswerten hinreichend instanziierte Argumente benötigen. Welche Prädikate gibt es, neben `is/2`, die solche Argumente benötigen? Warum ist die Anfrage `?- 42 >= X, 42 <= X.` nicht beweisbar?

---

Ein Teil der hier angesprochenen Probleme werden durch die Constraint-Programmierung gelöst, die wir nicht behandelt haben.

#### TEST 291

Ist `X =:= 4 + 7.` eine valide Anfrage in Prolog?

#### TEST 292

Benenne Vor- und Nachteile der Prolog-Arithmetik gegenüber Arithmetik mit Peano-Zahlen.

Betrachte dafür insbesondere `?- 4 is X + 2.` und `?- add(X, s(s(o)), s(s(s(s(o))))).`

#### TEST 293

Implementiere ein Prädikat `count_nodes/2`, dass die Anzahl von internen Knoten in einem Binärbaum zählt. Hier ist eine Beispielanwendung des Prädikats.

```
?- count_nodes(branch(empty, branch(empty, empty)), N).  
N = 2.
```

Warum die Anfrage `?- count_nodes(T, 2).` problematisch? Wie kannst du `count_nodes` anpassen, sodass das erwartete Ergebnis herauskommt?

#### TEST 294

Wann nennen wir einen Term vollständig instanziiert?

#### TEST 295

Wann nennen wir ein Prädikat relational, multimodal bzw. multidirektional?

## TEST 296

## Level Up

Du hast Prolog lieben gelernt und liest die Dokumentation von SWI-Prolog gerne vor dem Schlafen gehen. Du entdeckst sonderliche Sonderzeichen vor den Variablen eines Prädikats und fragst dich, wofür diese stehen. Das sind sogenannte argument mode indicators und deuten an, wie Informationen durch ein Prädikat fließen.

Was bedeuten die Indikatoren des `is/2`-Prädikats (siehe [is/2-Dokumentation](#))?

Uns ist bisher kein nicht vollständig instanziiertes Ausdruck eingefallen, den wir unter der Verwendung von ausschließlich vorlesungsbekannten Prädikaten konstruieren können. Um dennoch ein Beispiel gesehen zu haben, warum `Expr` mit `+` annotiert ist und nicht mit `++`, betrachte folgendes Programm:

```
:- arithmetic_function(mult/2).
mult(0, _, 0) :- !.
mult(_, 0, 0) :- !.
mult(X, Y, Z) :- Z is X * Y.

?- X is mult(0, Y).
X = 0.
```

Die rechte Seite muss so also nur hinreichend instanziiert sein, damit `is/2` beweisbar ist.

## TEST 297 ☹️



Gegeben sei das Prädikat

```
f(X, Y) :- Y is 2 * X + 1.
```

das die Funktion  $f(x) = 2x + 1$  in Prolog modelliert.

- Welche Anfragen der Form `?- f(X, Y)`, wobei `X` und `Y` frei oder gebunden sein können, sind nicht beweisbar und warum?
- Verbessere `f/2` so, dass `f/2` als  $f^{-1}$  verwendet werden kann. Als Hilfsprädikate kannst du die Prädikate `var/1` und `nonvar/1` verwenden.
- Warum ist `?- f(X, Y)` mit freien Variablen `X` und `Y` weiterhin problematisch?
- Wie könnte das Prädikat

```
int(0).
int(N) :- int(N, 1).
int(N, N).
int(N, M) :- N is -M.
int(N, M) :- K is M + 1, int(N, K).
```

das Problem der vorherigen Teilaufgabe lösen?

## TEST 298

Löse [Test 297](#) erneut, aber betrachte die Funktion  $f(x) = x^2$  für alle  $x \in \mathbb{Z}$ .

### Selbstevaluation

Ich bin in der Lage, ...

- das Prädikat `is/2` zu verwenden, um arithmetische Ausdrücke auszuwerten,
- gültige von ungültigen arithmetischen Anfragen zu unterscheiden,
- zu erklären, warum `=/2` für Arithmetik ungeeignet ist und wann `is/2` verwendet werden muss,
- alternative arithmetische Prädikate in Prolog einzusetzen (Peano-Arithmetik),
- die Vor- und Nachteile der eingebauten Prolog-Arithmetik gegenüber Peano-Arithmetik zu erläutern
- Prädikate relational zu definieren und Prädikate zu erkennen, die nicht relational sind.

## Prädikate höherer Ordnung

### Referenzen

- Skript: Einführung in die Logikprogrammierung, Meta-Programmierung, Prädikate höherer Ordnung

#### TEST 299

Erkläre, wie das Meta-Prädikat `call` verwendet wird. Welche Stelligkeit hat `call`?

#### TEST 300



Implementiere die Prädikate `map/3`, `filter/3`, `foldr/4`, so wie du sie erwarten würdest. Implementiere darüber hinaus das Prädikat `all/2`, dass genau dann erfüllt ist, wenn das ein übergegebene Prädikat für alle Elemente einer Liste erfüllt ist.

#### TEST 301

Du möchtest die Faltungsfunktion aus Test 300 in Aktion sehen und betrachtest die folgende Anfrage `?- foldr(+, 0, [1, 2, 3], X)`.

- Was ist das Ergebnis bzw. wie ist `X` belegt?
- Wieso funktioniert die Anfrage, sowie sie angegeben ist, nicht? Wie könntest du es ermöglichen, dass diese Anfrage im „Term-Sinne“ beweisbar wird?

### Selbstevaluation

Ich bin in der Lage, `call` zu nutzen.

## Kapselung des Nichtdeterminismus

### Referenzen

- Skript: Einführung in die Logikprogrammierung, Meta-Programmierung, Kapselung des Nichtdeterminismus

### TEST 302 ☹

Implementiere ein Prädikat `all_trees/2`, das jeden möglichen blätterbeschrifteten Binärbaum erzeugt, deren Blätter von links nach rechts die Eingabeliste lesen.

```
?- all_trees([1, 2, 3], Ts).  
Ts = [  
  branch(leaf(1), branch(leaf(2), leaf(3))),  
  branch(branch(leaf(1), leaf(2)), leaf(3))  
].
```

- Implementiere zuerst eine Variante, in der du auf keine besonderen Hilfsprädikate zurückgreifst.
- Gegeben sei folgende fehlerhafte Implementierung des Prädikats.

```
all_trees([], []).  
all_trees([X], [leaf(X)]) :- !.  
all_trees(Xs, Ts) :-  
  member(branch(Lt, Rt), Ts),  
  append([L|Ls], [R|Rs], Xs),  
  all_trees([L|Ls], Lts),  
  member(Lt, Lts),  
  all_trees([R|Rs], Rts),  
  member(Rt, Rts).
```

Auf dem ersten Blick scheint eine elegante Idee hinter dieser Definition zu stecken. Wieso funktioniert diese Idee allerdings nicht?

- Korrigiere die fehlerhafte Definition unter Verwendung von `findall/2` und erhalte dabei grundsätzliche Idee.
- Staune darüber, wie vergleichsweise müheselig das Implementieren der ersten Variante ohne Hilfsprädikate war, und sei dankbar für `findall/3`.

### TEST 303

Implementiere ein Prädikat `all_btrees/2`, dass alle Binärbäume mit einer festen internen Knotenanzahl berechnet.

---

Es bietet sich an, [Test 293](#) vorher bearbeitet zu haben.

### Selbstevaluation

Ich bin in der Lage, ...

- `findall/3` oder ähnliche Mechanismen einzusetzen, um systematisch alle möglichen Ergebnisse zu kapseln.

## Logik-Puzzles

Hier sind ein paar Logik-Puzzles, die zu deiner Belustigung dienen. Der Lerneffekt ist voraussichtlich sehr gering.

### CHALLENGE 23 ⌚

Das Zebra-Rätsel ist ein Logikpuzzle.

So wird es auf Wikipedia wiedergegeben:

1. Es gibt fünf Häuser.
2. Der Engländer wohnt im roten Haus.
3. Der Spanier hat einen Hund.
4. Kaffee wird im grünen Haus getrunken.
5. Der Ukrainer trinkt Tee.
6. Das grüne Haus ist direkt rechts vom weißen Haus.
7. Der Raucher von Old-Gold-Zigaretten hält Schnecken als Haustiere.
8. Die Zigaretten der Marke Kools werden im gelben Haus geraucht.
9. Milch wird im mittleren Haus getrunken.
10. Der Norweger wohnt im ersten Haus.
11. Der Mann, der Chesterfield raucht, wohnt neben dem Mann mit dem Fuchs.
12. Die Marke Kools wird geraucht im Haus neben dem Haus mit dem Pferd.
13. Der Lucky-Strike-Raucher trinkt am liebsten Orangensaft.
14. Der Japaner raucht Zigaretten der Marke Parliaments.
15. Der Norweger wohnt neben dem blauen Haus.

Wer trinkt Wasser? Wem gehört das Zebra?

Löse dieses Puzzle mithilfe von Prolog und übersetze dabei so wenig logische Schlüsse, die du selbst in deinem Kopf machst, in deinem Programm.

### CHALLENGE 24 ⌚

Ein Logikpuzzle, das einst viral ging, ist bekannt als „Cheryl’s Geburtstag“. Es ist das erste Puzzle aus dem paper Cheryl’s Birthday. Löse dieses Puzzle mithilfe von Prolog und übersetze dabei so wenig logische Schlüsse, die du selbst in deinem Kopf machst, in deinem Programm.

---

Diese Aufgaben haben noch keinen Platz gefunden.

## CHALLENGE 25

## Deep Dive

Psst! Ja, du! Möchtest du ein Werkzeug zum Generieren von SLD-Bäumen?

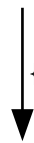
Erweitere den Prolog-Interpreter um eine Funktion `graphviz :: SLDTree → String`, die einen SLD-Baum nimmt und ihn z.B. mithilfe von `Graphviz` visualisiert.

Das Erzeugen des folgenden DOT-Codes ist vergleichsweise unkompliziert aus dem Projekt-Code heraus möglich. Wenn du dir mehr anzeigen lassen möchtest, musst du ein paar Stellen anpassen. Der `SLDTree` speichert nicht, welche Regel verwendet wurde, bzw. die umbenannte Regel, die für die Unifikation verwendet wurde. Dafür müsste der Typ entsprechend angepasst werden. Je mehr du dir anzeigen lassen möchtest, desto zeitaufwendiger oder schwieriger wird diese Challenge. Eine prototypische Implementierung, die das untere Beispiel umsetzt, ist zügig möglich.

Folgender DOT-Code erzeugt den folgenden SLD-Baum. Graphviz unterstützt von Haus aus nicht das Setzen von mathematischen Formeln. Entweder du verzichtest darauf oder du schaust dir z.B. `dot2tex` oder `diagraph` an, je nach dem ob du `LaTeX` oder `Typst` nutzt bzw. nutzen möchtest. Wenn du keinen DOT-Compiler hast, kannst du zum Debuggen deines generierten DOT-Codes [diese Seite](#) nutzen.

```
digraph {
  node [shape=none];
  1 [label="?- =(X, 1)."];
  2 [label="?- ."];
  1 → 2 [label="{X1 |→ 1, X |→ 1}"];
}
```

$?- = (X, 1).$



$\{X_1 \mapsto 1, X \mapsto 1\}$

$?- .$

## Hinweise zu Tests und Challenges

### HINWEIS ZU CHALLENGE 1

- Zur Darstellung der Multimengen eignen sich sortierte Listen gut.
- Zur Berechnung des Schnittes können zwei sortierte Listen parallel durchlaufen werden. Wenn zwei gleiche Elemente zu Beginn der Liste stehen, wird eines der Elemente zum Ergebnis hinzugefügt. Im anderen Fall überspringen wir das jeweils kleinere Element der beiden.

### HINWEIS ZU CHALLENGE 5

Nach  $n$  enqueue- Operationen in eine leere queue sieht diese wie folgt aus

$$Q [] [x_n, \dots, x_1].$$

Wenn als Nächstes dequeue-Operationen naiv ausgeführt werden, müssten wir die enqueue-Liste jedes Mal vollständig durchlaufen. Wenn wir die dequeue-Liste entsprechend verändern, können wir mindestens die nächsten  $n$  dequeue-Operationen effizient durchführen.

### HINWEIS ZU CHALLENGE 10

Wenn du Anlaufschwierigkeiten hast, helfen dir möglicherweise diese ersten Implementierungen weiter.

```
instance Num a => Num (D a) where
  D x1 d1 + D x2 d2 = D (x1 + x2) (d1 + d2) -- Summenregel
  -- ...

instance Fractional a => Fractional (D a) where
  -- ...

instance Floating a => Floating (D a) where
  exp (D x d) = D (exp x) (d * exp x) -- Kettenregel und exp'(x) = exp(x)
  -- ...
```

### HINWEIS ZU CHALLENGE 15

Die Rekursionsvorschrift ist gegeben durch

$$\text{pathsum}(i, j) = \begin{cases} G_{0,0} & \text{falls } (i, j) = (0, 0), \\ G_{i,0} + \text{pathsum}(i-1, 0) & \text{falls } j = 0, \\ G_{0,j} + \text{pathsum}(0, j-1) & \text{falls } i = 0, \\ G_{i,j} + \min(\text{pathsum}(i-1, j), \text{pathsum}(i, j-1)) & \text{sonst} \end{cases}$$

für  $i \in \{0, \dots, m-1\}, j \in \{0, \dots, n-1\}$ . Die minimale Pfadsumme ist dann  $\text{pathsum}(m-1, n-1)$ .

### HINWEIS ZU TEST 167

Die Implementierung dieser Monade ist sehr ähnlich zu der Implementierung der **Reader**-Monade, wie sie in [Test 166](#) angegeben ist.

#### HINWEIS ZU CHALLENGE 16

Das Identitätsgesetz wäre verletzt, wenn man `pure` wie gegeben definieren würde. Betrachte folgendes Gegenbeispiel.

```
pure id      <*> ZipList [1, 2]
== ZipList [id] <*> ZipList [1, 2]
== ZipList [1]
≠ ZipList [1, 2]
```

Es muss also dafür gesorgt sein, dass es genügend `ids` in der linken `ZipList` gibt.

## Weitere Ressourcen

Wenn du auf der Suche nach weiteren Übungsaufgaben bist, mit denen du deine Programmierkenntnisse in Prolog verbessern möchtest, bietet sich die Liste [P-99: Ninety-Nine Prolog Problems](#) an. Lösungen sind ebenso auf der Seite verfügbar. Für Haskell gibt es eine ähnliche Seite [H-99: Ninety-Nine Haskell Problems](#). Je nachdem wie fortgeschritten das Modul ist, kannst du versuchen, die ersten Probleme des vergangenen oder diesjährigen [Advent of Code](#) zu lösen – sehe diesen Verweis allerdings mehr als Herausforderung und nicht als Empfehlung zum Aufarbeiten von Grundkenntnissen.

Weitere Links:

- [Haskell 2010 Language Report](#): Spezifikation von Haskell
- [Learn You A Haskell](#)
- [Haskelite](#): Ein Schritt-für-Schritt Interpreter für (eine Teilmenge von) Haskell
- [Functors, Applicatives, And Monads In Pictures](#)
- [Haskell Cheatsheet](#)
- [Prolog Cheatsheet](#)

Awesome-Listen sind eine Kuration von „awesome“ Inhalten zu einem bestimmten Thema. Möglicherweise findest du hier etwas, was dich anspricht.

- [Awesome Haskell](#)
- [Awesome Prolog](#)

# Häufige Bearbeitungsfehler in Klausuren

## Klausur 1.PZ WS25/26

### • Aufgabe 1

- Partiell definierte Funktionen (wie z.B. `init`, `tail`, `div`, `mod`, `maximum`, ...) wurden häufig nicht erkannt. Hier solltet ihr euch immer Fragen: Welche Eingabe könnte zu keinem Ergebnis führen? Das sind oft null-artige Werte wie `0` oder `[]`. (Siehe [Test 45](#))
- Korrekte Substitutionen bzw. Unifikatoren können mithilfe einer Probe identifiziert werden. Wenn Zweifel besteht, können diese schnell auf einem Schmierzettel durchgeführt werden. Zwei Terme sind unifizierbar, wenn die Substitution durch *einmaliges* Anwenden zwei gleiche Terme erzeugt.
- ?-  $X = Y$ ,  $Y = X$  ist im Vergleich zu ?-  $X \geq Y$ ,  $Y \geq X$  beweisbar. In der ersten Anfrage geht es um die Unifizierbarkeit von Variablen, im zweiten zuerst darum, dass  $X$  und  $Y$  berechnet werden können. Zum Zeitpunkt der Berechnung sind diese aber noch nicht hinreichend gebunden.

### • Aufgabe 2

- Eine Zählvariable, so wie du sie aus imperativen Programmiersprachen kennst, kannst du als Parameter einer Hilfsfunktion mitführen. So etwas wie `count = count + 1` gibt es in Haskell nicht. (Siehe [Test 7](#))
- `getLine` entfernt den Zeilenumbruch am Ende einer Zeile. Technisches Detail, das man sich merken muss.
- ADT: siehe [Bemerkung 5](#) bzw. zuerst [Test 90](#)

### • Aufgabe 3

- In `instance Functor Fiction where` wurde häufig noch eine Typvariable ergänzt, obwohl in den Typkonstruktorklassen der letzte Parameter eines Typs eben nicht appliziert werden darf.
- Dasselbe gilt auch für die `Monad`-Instanz.
- In `fmap f (Fake fx) = Fake (fmap f fx)` wurde häufiger entweder der Konstruktor `Fake` oder `fmap` vergessen.
- Ähnlich auch bei der `Fake`-Regel für das `(>=>)`.
- In beiden Fällen sollte man die Identitätsgesetze auf Plausibilität prüfen, `fmap id = id` und `m >=> return = m`. So sieht man, ob z.B. in `Fake fx >=> f = fx >=> f` ein Konstruktor fehlt oder nicht.
- In der letzten Teilaufgabe wurden häufig `Fake`-Konstruktoren weggelassen. Hier bietet es sich an zu raten, wenn man keine `Monad`-Instanz angeben konnte, oder einen mit der eigenen `Monad`-Instanz konsistenten Wert schrittweise auszurechnen.

```
instance Monad Fiction where
  Real x  >=> f = f x           -- (1)
  Fake fx >=> f = Fake (fx >=> f) -- (2)

  Fake (Real 42) >=> \x → Fake (Real (x + 31)) -- (2)
= Fake (Real 42 >=> \x → Fake (Real (x + 31))) -- (1)
= Fake ((\x → Fake (Real (x + 31))) 42)       -- Applikation
= Fake (Fake (Real (42 + 31)))                -- Definition (+)
= Fake (Fake (Real 73))
```

So ist die Berechnung dann im schlechtesten Fall ein Folgefehler aus der Definition der `Monad`-Instanz.

### • Aufgabe 4

- „mithilfe eines Aufrufs von `foldr`“ bedeutet, die Definition besteht aus einer Regel, die so startet `listToN_fold xs = foldr ... xs` (oder  $\eta$ -reduziert).
- `[]` ist kein Fall, der im Lambda behandelt werden muss. Dafür ist der zweite Parameter der `foldr`-Funktion da.
- Mit der natürlichen Faltung ist das Verfahren aus der Vorlesung gemeint, um eine Faltungsfunktion für einen gegebenen Datentypen anzugeben – nichts anders.

- Häufig wurde eine Funktion für den **Nested**-Konstruktor vergessen.
- Variablen in Pattern werden kleingeschrieben.
- Prüft die Stelligkeiten und Typen der Parameter der Datenkonstruktoren!
- Schaut insbesondere nochmal nach, wie sich die Signatur der Faltungsfunktion aus dem Datentypen ergibt.
- **Aufgabe 5**
  - Arithmetik nicht in richtiger Reihenfolge (auf der rechten Seite muss alles hinreichend instanziiert zum Rechnen sein)
  - `f(X + 1, Y div 2)` gibt Terme an das Prädikat `f`, und nicht die ausgerechneten Ergebnisse. Diese müssen immer mit `is/2` berechnet werden.
- **Aufgabe 6**
  - $\sigma_1 = \{X \mapsto Y, Y \mapsto 1\} \neq \{X \mapsto 1, Y \mapsto 1\} = \sigma_2$ , denn  $\sigma_1((X, Y)) = (Y, 1) \neq (1, 1) = \sigma_2((X, Y))$ .
  - Insbesondere kann  $\{X \mapsto Y\} \circ \{Y \mapsto 1\}$  nicht während der Durchführung des Unifikationsalgorithmus auftreten.
  - Andersherum,  $\{Y \mapsto 1\} \circ \{X \mapsto Y\} = \{Y \mapsto 1, X \mapsto 1\}$  und nicht  $\{Y \mapsto 1, X \mapsto Y\}$ .
  - Im Kapitel Rechnen in der Logikprogrammierung findest du ganz viele Selbsttests, die dich für diese Fehler sensibilisieren können, sodass du sie rechtzeitig erkennst.

## Appendix

Bisher findest du hier im Anhang des Dokuments Bemerkungen zu verschiedenen Tests oder Challenges. Sie sind aus unserer Neugier entstanden und stecken dich möglicherweise an. Du kannst aber auch alles ignorieren, was hier steht, und verpasst nichts.

### BEMERKUNG 2

In [Challenge 10](#) wurden Funktionen `d1` und `d2` definiert. Diese ver- und entschachteln `D`-Werte unterschiedlich tief, um die jeweilige Ableitung zu berechnen. Ein allgemeines `d` für beliebige Ableitung zu definieren, stellt sich als schwierig heraus. Mit verschiedenen Haskell-Spracherweiterungen können wir eine allgemeinere Funktion angeben. Das ist in [AutoDiff.hs](#) im GitHub-Repository demonstriert. Mit `derive @3 cos (2.0 :: Double)` können wir jetzt z.B. die dritte Ableitung des Cosinus an der Stelle 2 berechnen.

### BEMERKUNG 3

In [Test 82](#) haben wir gesehen, wie deklarative Programmierkonzepte aus Haskell in Python verwendet werden können. In Java haben diese über die letzten Jahre auch einen Platz gefunden. Hier ist das Programm in Java (ohne gecurryte Funktionen und partielle Applikation).

```
import java.util.*;
import java.util.function.*;

interface Numeric<T> {
    T zero();
    T add(T a, T b);

    public static Numeric<Integer> integer() {
        return new Numeric<>() {
            public Integer zero() { return 0; }
            public Integer add(Integer a, Integer b) { return a + b; }
        };
    }
}

interface Foldable<T> {
    <R> R foldr(R initial, BiFunction<T, R, R> function);

    default List<T> toList() {
        return foldr(new LinkedList<>(), (value, list) -> {
            list.add(value);
            return list;
        });
    }

    default int length() { return foldr(0, (_, result) -> result + 1); }

    default boolean contains(T value) {
        return foldr(false, (other, result) -> result || other.equals(value));
    }

    default T sum(Numeric<T> numeric) {
        return foldr(numeric.zero(), numeric::add);
    }
}

sealed interface Tree<T> extends Foldable<T>
```

```

    permits Empty, Node {}

record Empty<T>() implements Tree<T> {
    @Override
    public <R> R foldr(R initial, BiFunction<T, R, R> function) {
        return initial;
    }
}

record Node<T>(Tree<T> left, T value, Tree<T> right) implements Tree<T> {
    @Override
    public <R> R foldr(R initial, BiFunction<T, R, R> function) {
        R x = right.foldr(initial, function);
        R y = function.apply(value, x);
        return left.foldr(y, function);
    }
}

public class Main {
    public static void main(String[] args) {
        Tree<Integer> tree = new Node<>(
            new Empty<>(),
            3,
            new Node<>(new Node<>(new Empty<>(), 7, new Empty<>()), 4, new Empty<>())
        );

        System.out.println(tree.sum(Numeric.integer())); // 14
        System.out.println(tree.toList()); // [4, 7, 3]
        System.out.println(tree.length()); // 3
        System.out.println(tree.contains(3)); // true
        System.out.println(tree.contains(9)); // false
    }
}

```

#### BEMERKUNG 4

Das ist ein Lösungsvorschlag für Test 90.

Beschreibe **Timer** den zu spezifizierenden ADT. Dann sollen für diesen folgende Funktionen existieren und Gesetze gelten.

##### Konstruktoren

- `new :: Int → Int → Timer` erzeugt einen neuen Timer mit einer Zeiteingabe bestehend aus Stunden und Minuten.

##### Selektoren

- `display :: Timer → (Int, Int)` gibt die verbleibende auf dem Timer an.

##### Testfunktionen

- `wentOff :: Timer → Bool` gibt an, ob der Timer bereits klingelte

##### Operationen

- `tick :: Timer → Timer` lässt eine Sekunde auf einem Timer vergehen.

##### Gesetze

Seien  $h \in \mathbb{N}_0$ ,  $m \in \{0, \dots, 59\}$  und  $t$  einer beliebiger Timer.

- `display (new h s) == (h, s)`
- `wentOff t ⇔ display t == (0, 0)`
- `s > 0 ⇔ display (tick (new h s)) == (h, s - 1)`
- `s == 0 && h > 0 ⇔ display (tick (new h s)) == (h - 1, 59)`
- `s == 0 && h == 0 ⇒ display (tick (new h s)) == (0, 0)`

#### BEMERKUNG 5

In der Klausur des 1. Prüfungszeitraums WS25/26 gab es eine ADT-Aufgabe, in der eine Uhr modelliert werden sollte. Die häufigsten Fehler sind hier auf Test 90 bzw. Bemerkung 4 übertragen.

- Die Definition von Konstruktoren als Teil **data**-Deklaration des ADT.  
`data Timer = Timer Int Int` ist bereits eine Implementierung. Eine wesentliche Eigenschaft eines ADTs ist, sich über Implementierungsdetails keine Gedanken gemacht werden muss (und soll). Dazu gehört die konkrete Darstellung des Typen, die wir mit der obigen Deklaration angeben. Stattdessen geben wir eine Funktion mit dem Typ `Int → Int → Timer` als Konstruktor an. So ist die interne Darstellung weiterhin offen.

Die nächsten Anmerkungen fallen in die Kategorie „Nicht unbedingt falsch, aber fehl am Platz“. Der ADT soll die Funktionweise einer Schnittstelle angeben und sich nicht mit Implementierungsdetails aufhalten.

- Es kam häufiger vor, dass Hilfsfunktionen angegeben wurden, die die Konsistenz des internen Zustands eines Wertes überprüfen sollten – möglicherweise aus Motivation heraus, dass so etwas wie z.B. `display (new 0 73) == (0, 73)` nicht passieren kann.
- Unter der Annahme einer Vergleichsoperation auf Timern gab es z.B. auch das Gesetz `t1 == t2 ⇔ display t1 == display t2`. Es gilt zwar, wann zwei Timer gleich sind, aber sagt nichts darüber aus, wie ein Timer funktionieren soll.